HelixTerminator — DevOps & Infrastructure Specification

Document ID: `HT-DEVOPS-004`
Version: `1.0.0`
Status: Production-Ready Draft
Last Updated: 2026-06-28
Owned by: Platform Engineering Team
Classification: Internal — Engineering

Table of Contents

1. Repository Structure
2. Kubernetes Manifests
3. Helm Charts
4. Docker & Podman Build
5. CI/CD Pipelines
6. Terraform Infrastructure-as-Code
7. Observability Stack
8. Disaster Recovery
9. Local Development Environment
10. Security Hardening
11. Cost and FinOps Governance

---

1. Repository Structure

1.1 Overview and Philosophy

The HelixTerminator monorepo follows a domain-driven layout that co-locates infrastructure definitions with the services they govern. Every directory has an explicit owner, naming convention, and

lifecycle policy. Git submodules pin external platform libraries at tested versions; the `helix-deps.yaml` manifest documents every external dependency with its version, license, and approved use.

The monorepo is the single source of truth for:

- All 25 backend microservices (Go 1.25)
- The Flutter client (web + iOS + Android)
- All Kubernetes, Helm, and Terraform declarations
- CI/CD pipeline definitions
- Operational runbooks

The repo enforces branch protection on `main` and `release/*`. Every merge to `main` triggers a full CI pipeline; every merge to `release/*` triggers a production deployment pipeline with canary promotion.

1.2 Top-Level Directory Tree

```
helix-terminator/                          # Root monorepo
├── .github/                               # GitHub-specific
configuration
│   ├── workflows/                         # GitHub Actions
workflow YAML
│   │   ├── pr.yml                         # Pull request
validation
│   │   ├── main.yml                       # Main branch
integration
│   │   ├── release.yml                    # Production release +
canary
│   │   ├── nightly.yml                    # Nightly security +
performance scans
│   │   └── dependency-update.yml          # Renovate / Dependabot
integration
│   ├── CODEOWNERS                         # Team ownership
mapping
│   ├── PULL_REQUEST_TEMPLATE.md           # PR checklist
│   ├── ISSUE_TEMPLATE/
│   │   ├── bug_report.md
│   │   ├── feature_request.md
│   │   └── security_vulnerability.md
│   └── dependabot.yml                     # Dependency update
schedule
│
├── constitution/                          # Git submodule:
HelixConstitution
```

```
│   │                                   # Enforces API contract
standards,
│   │                                   # naming conventions,
and governance rules
│   ├── README.md
│   ├── schemas/                        # JSON Schema / OpenAPI
schema registry
│   ├── policies/                       # OPA Rego policies for
compliance checks
│   └── linters/                        # Custom linter rules
for constitution compliance
│
├── submodules/                         # Platform library
submodules
│   ├── containers/                     #
digital.vasic.containers
│   │   │                               # ContainerRuntime
interface abstracting
│   │   │                               # Docker / Podman /
Kubernetes
│   │   ├── go.mod
│   │   ├── runtime.go                  # ContainerRuntime
interface definition
│   │   ├── docker/                     # Docker adapter
│   │   ├── podman/                     # Podman adapter
│   │   ├── kubernetes/                 # Kubernetes adapter
│   │   └── testutil/                   # Test helpers + mock
runtime
│   │
│   ├── docs_chain/                     #
digital.vasic.docs_chain
│   │   │                               # Documentation chain-
of-custody system;
│   │   │                               # all docs/ files are
managed through this
│   │   ├── go.mod
│   │   ├── chain.go
│   │   └── verifier/
│   │
│   ├── security/                       #
digital.vasic.security
│   │   │                               # Shared crypto
primitives, mTLS helpers,
│   │   │                               # RBAC enforcement
```

```
middleware
│   │   ├── go.mod
│   │   ├── mtls/
│   │   ├── rbac/
│   │   ├── crypto/
│   │   └── audit/
│   │
│   ├── auth/                          # digital.vasic.auth
│   │   │                              # JWT issuance/
validation, OAuth2 flows,
│   │   │                              # OIDC provider
integration
│   │   ├── go.mod
│   │   ├── jwt/
│   │   ├── oauth2/
│   │   └── oidc/
│   │
│   └── helixqa/                       # HelixDevelopment/
helixqa
│       │                              # Integration test
framework,
│       │                              # testcontainers
helpers, contract test tooling
│       ├── go.mod
│       ├── framework/
│       ├── containers/
│       └── pact/
│
├── services/                          # All 25 backend
microservices
│   ├── gateway/                       # API Gateway — entry
point for all external traffic
│   ├── auth-service/                  # Authentication &
session management
│   ├── vault-service/                 # Secret/credential
vault (Shamir-based)
│   ├── ssh-proxy/                     # SSH connection broker
and proxy
│   ├── session-recorder/              # SSH session recording
+ replay
│   ├── credential-manager/            # Credential lifecycle
management
│   ├── user-service/                  # User accounts and
profiles
```

```
│   ├── organization-service/             # Multi-tenant
organization management
│   ├── rbac-service/                     # Role-Based Access
Control engine
│   ├── audit-service/                    # Immutable audit log
ingestion
│   ├── notification-service/             # Email, Slack, webhook
notifications
│   ├── scheduler-service/                # Cron and one-time
task scheduling
│   ├── inventory-service/                # Server/host inventory
management
│   ├── key-rotation-service/             # Automated SSH key
rotation
│   ├── compliance-service/               # Compliance rule
evaluation engine
│   ├── reporting-service/                # Async report
generation
│   ├── billing-service/                  # Usage metering and
billing
│   ├── search-service/                   # Full-text search
(Elasticsearch-backed)
│   ├── webhook-service/                  # Outbound webhook
delivery
│   ├── approval-workflow-service/        # Multi-step approval
flows
│   ├── certificate-service/              # X.509 certificate
management
│   ├── tunnel-service/                   # TCP tunnel management
│   ├── metrics-aggregator/               # Internal metrics
rollup
│   ├── policy-engine/                    # OPA-based policy
evaluation
│   └── file-transfer-service/            # SCP/SFTP file
transfer brokering
│
├── clients/
│   └── flutter/                          # Single Flutter
codebase (web + iOS + Android)
│       ├── lib/
│       │   ├── core/
│       │   ├── features/
│       │   └── shared/
│       ├── test/
```

```
│       ├── integration_test/
│       ├── pubspec.yaml
│       └── Dockerfile                    # Flutter web
production build
│
├── infrastructure/
│   ├── kubernetes/                       # Raw Kubernetes
manifests (kustomize base)
│   │   ├── base/                         # Base resources
│   │   ├── overlays/
│   │   │   ├── development/
│   │   │   ├── staging/
│   │   │   └── production/
│   │   └── namespaces/
│   │
│   ├── helm/                             # Helm charts
│   │   ├── helixterm/                    # Umbrella chart
│   │   └── charts/                       # Service sub-charts
│   │
│   ├── terraform/                        # Terraform IaC
│   │   ├── modules/
│   │   ├── environments/
│   │   │   ├── production/
│   │   │   └── staging/
│   │   └── shared/
│   │
│   └── docker/                           # Dockerfiles + Compose
files
│       ├── compose/
│       └── images/
│
├── scripts/                              # Operational and
developer scripts
│   ├── setup-dev.sh                      # Developer environment
bootstrap
│   ├── seed-db.sh                        # Database seeding
│   ├── rotate-secrets.sh                 # Manual secret
rotation
│   ├── health-check.sh                   # Cluster health
verification
│   └── generate-certs.sh                 # Local TLS cert
generation
│
├── docs/                                 # docs_chain-managed
```

```
documentation
│   ├── 01_architecture.md
│   ├── 02_api_reference.md
│   ├── 03_security_model.md
│   ├── 04_devops_infrastructure.md        # This document
│   └── runbooks/
│       ├── incident-response.md
│       ├── failover-procedure.md
│       └── key-rotation.md
│
├── helix-deps.yaml                        # Dependency manifest
(all submodules + external)
├── go.work                                # Go workspace file
├── go.work.sum
├── Makefile                               # Top-level developer
shortcuts
├── .gitmodules                            # Submodule
declarations
├── .gitignore
└── README.md
```

1.3 Service Directory Convention

Every service under `services/` follows an identical internal layout:

```
services/<service-name>/
├── cmd/
│   └── <service-name>/
│       └── main.go                # Entrypoint — wires DI, starts
server
├── internal/
│   ├── api/                       # HTTP/gRPC handlers
│   │   ├── http/
│   │   └── grpc/
│   ├── domain/                    # Pure business logic (no I/O)
│   ├── repository/                # Database/cache access
│   ├── service/                   # Application services (use
cases)
│   ├── config/                    # Configuration loading
│   └── middleware/                # Service-specific middleware
├── migrations/                    # SQL migration files (golang-
migrate)
│   ├── 000001_init.up.sql
│   └── 000001_init.down.sql
```

```
├── api/
│   └── proto/                    # Protobuf definitions
│       └── v1/
│           └── <service>.proto
├── test/
│   ├── unit/
│   ├── integration/              # testcontainers-based
integration tests
│   └── contract/                 # Pact consumer/provider tests
├── Dockerfile                    # Production multi-stage
Dockerfile
├── Dockerfile.dev                # Development image (with delve
debugger)
├── go.mod
├── go.sum
└── README.md                     # Service-specific
documentation
```

1.4 Infrastructure Directory Convention

```
infrastructure/kubernetes/base/
├── namespaces/
│   ├── helixterm-prod.yaml
│   ├── helixterm-staging.yaml
│   ├── helixterm-dev.yaml
│   └── helixterm-monitoring.yaml
├── network-policies/
│   ├── default-deny-all.yaml
│   └── allow-<service>-<target>.yaml    # One file per allowed
connection
├── rbac/
│   ├── cluster-roles.yaml
│   └── service-accounts.yaml
└── <service-name>/
    ├── deployment.yaml
    ├── service.yaml
    ├── hpa.yaml
    ├── pdb.yaml
    ├── configmap.yaml
    └── serviceaccount.yaml
```

1.5 Naming Conventions

Resource	Convention	Example
Go packages	`lowercase`, no hyphens	`authservice`
Service directories	`kebab-case`	`auth-service`
Docker images	`ghcr.io/helixdevelopment/<service>:<semver>`	`ghcr.io/helixdevelopment/auth-service:1.4.2`
K8s namespaces	`helixterm-<env>`	`helixterm-prod`
K8s labels	`app`, `version`, `team`, `component`	`app: auth-service`
Helm releases	`helixterm-<env>`	`helixterm-prod`
Terraform workspaces	`<env>-<region>`	`prod-us-east-1`
Git branches	`feature/<ticket>-<slug>`, `fix/<ticket>-<slug>`	`feature/HT-123-add-vault-unseal`
Git tags	`v<MAJOR>.<MINOR>.<PATCH>`	`v1.4.2`

1.6 `helix-deps.yaml` Format

```yaml
# helix-deps.yaml — canonical dependency manifest for HelixTerminator
# Every external dependency requires an entry here before use.
# Fields: module, version, license, approved-by, approved-date, usage

apiVersion: helix/v1
kind: DependencyManifest

submodules:
  - name: digital.vasic.containers
    path: submodules/containers
    ref: v2.3.1
    license: MIT
    approved-by: platform-lead
    approved-date: "2026-01-15"
    usage: "ContainerRuntime abstraction for Docker/Podman/K8s"

  - name: digital.vasic.docs_chain
    path: submodules/docs_chain
    ref: v1.1.0
    license: MIT
```

```yaml
    approved-by: platform-lead
    approved-date: "2026-01-15"
    usage: "Documentation chain-of-custody and integrity
        verification"

  - name: digital.vasic.security
    path: submodules/security
    ref: v3.0.2
    license: Apache-2.0
    approved-by: security-lead
    approved-date: "2026-02-01"
    usage: "Shared crypto, mTLS, RBAC middleware"

  - name: digital.vasic.auth
    path: submodules/auth
    ref: v2.8.0
    license: Apache-2.0
    approved-by: security-lead
    approved-date: "2026-02-01"
    usage: "JWT, OAuth2, OIDC integration"

  - name: HelixDevelopment/helixqa
    path: submodules/helixqa
    ref: v1.5.3
    license: MIT
    approved-by: platform-lead
    approved-date: "2026-01-20"
    usage: "Integration test framework and testcontainers helpers"

  - name: HelixDevelopment/helixconstitution
    path: constitution
    ref: v4.1.0
    license: Proprietary
    approved-by: cto
    approved-date: "2026-01-01"
    usage: "API contract governance and compliance policies"

infrastructure-dependencies:
  - name: kafka
    chart: bitnami/kafka
    version: "26.8.5"
    license: Apache-2.0

  - name: postgresql
```

```yaml
    chart: bitnami/postgresql
    version: "13.4.4"
    license: Apache-2.0

  - name: redis
    chart: bitnami/redis
    version: "18.19.4"
    license: Apache-2.0

  - name: rabbitmq
    chart: bitnami/rabbitmq
    version: "14.6.6"
    license: Apache-2.0

  - name: cert-manager
    chart: jetstack/cert-manager
    version: "v1.14.5"
    license: Apache-2.0

  - name: ingress-nginx
    chart: ingress-nginx/ingress-nginx
    version: "4.10.1"
    license: Apache-2.0

  - name: prometheus-stack
    chart: prometheus-community/kube-prometheus-stack
    version: "58.7.2"
    license: Apache-2.0
```

1.7 CODEOWNERS

```
# .github/CODEOWNERS
# Each service is owned by its functional team.
# Platform infrastructure is owned by the platform team.

# Global fallback
*                                   @helixterm/platform-
engineering

# Core platform services
/services/gateway/                  @helixterm/platform-
engineering
/services/auth-service/             @helixterm/security-team
/services/vault-service/            @helixterm/security-team
```

```
/services/rbac-service/             @helixterm/security-team
/services/certificate-service/      @helixterm/security-team
/services/policy-engine/            @helixterm/security-team

# SSH infrastructure
/services/ssh-proxy/                @helixterm/ssh-team
/services/session-recorder/         @helixterm/ssh-team
/services/tunnel-service/           @helixterm/ssh-team
/services/file-transfer-service/    @helixterm/ssh-team
/services/key-rotation-service/     @helixterm/ssh-team

# User-facing services
/services/user-service/             @helixterm/product-engineering
/services/organization-service/     @helixterm/product-engineering
/services/billing-service/          @helixterm/product-engineering
/services/notification-service/     @helixterm/product-engineering
/services/approval-workflow-service/ @helixterm/product-
engineering
/services/reporting-service/        @helixterm/product-engineering

# Data and operations
/services/audit-service/            @helixterm/compliance-team
/services/compliance-service/       @helixterm/compliance-team
/services/inventory-service/        @helixterm/platform-
engineering
/services/scheduler-service/        @helixterm/platform-
engineering
/services/credential-manager/       @helixterm/security-team
/services/search-service/           @helixterm/platform-
engineering
/services/webhook-service/          @helixterm/platform-
engineering
/services/metrics-aggregator/       @helixterm/platform-
engineering

# Infrastructure
/infrastructure/                    @helixterm/platform-
engineering
/.github/                           @helixterm/platform-
engineering
/constitution/                      @helixterm/architecture

# Flutter client
/clients/flutter/                   @helixterm/frontend-team
```

```
# Security-sensitive files require security team review
/submodules/security/               @helixterm/security-team
@helixterm/platform-engineering
/infrastructure/terraform/          @helixterm/platform-
engineering @helixterm/sre-team
```

---

2. Kubernetes Manifests

2.1 Namespace Definitions

```yaml
# infrastructure/kubernetes/base/namespaces/helixterm-prod.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: helixterm-prod
  labels:
    environment: production
    team: platform
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: v1.31
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
---
apiVersion: v1
kind: Namespace
metadata:
  name: helixterm-staging
  labels:
    environment: staging
    team: platform
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: v1.31
---
apiVersion: v1
kind: Namespace
metadata:
  name: helixterm-dev
  labels:
    environment: development
    team: platform
    pod-security.kubernetes.io/enforce: baseline
```

```yaml
---
apiVersion: v1
kind: Namespace
metadata:
  name: helixterm-monitoring
  labels:
    environment: production
    team: platform
    purpose: observability
```

2.2 Default NetworkPolicy (Deny-All)

```yaml
# infrastructure/kubernetes/base/network-policies/default-deny-
        all.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: helixterm-prod
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
---
# Allow DNS resolution for all pods
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
  namespace: helixterm-prod
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress:
  - ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
```

2.3 Gateway Service

```yaml
# infrastructure/kubernetes/base/gateway/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gateway
  namespace: helixterm-prod
  labels:
    app: gateway
    version: v1
    team: platform
    component: ingress
spec:
  replicas: 3
  selector:
    matchLabels:
      app: gateway
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: gateway
        version: v1
        team: platform
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "9090"
        prometheus.io/path: "/metrics"
    spec:
      serviceAccountName: gateway
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        runAsUser: 65532
        runAsGroup: 65532
        fsGroup: 65532
        seccompProfile:
          type: RuntimeDefault
      affinity:
```

```yaml
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels:
            app: gateway
        topologyKey: kubernetes.io/hostname
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
              app: gateway
          topologyKey: topology.kubernetes.io/zone
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        app: gateway
  terminationGracePeriodSeconds: 30
  containers:
  - name: gateway
    image: ghcr.io/helixdevelopment/gateway:1.0.0
    imagePullPolicy: Always
    ports:
    - containerPort: 8080
      name: http
      protocol: TCP
    - containerPort: 9090
      name: metrics
      protocol: TCP
    resources:
      requests:
        memory: "256Mi"
        cpu: "250m"
      limits:
        memory: "512Mi"
        cpu: "1000m"
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop:
```

```
    - ALL
livenessProbe:
  httpGet:
    path: /healthz/live
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 15
  failureThreshold: 3
  timeoutSeconds: 5
readinessProbe:
  httpGet:
    path: /healthz/ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3
  timeoutSeconds: 5
startupProbe:
  httpGet:
    path: /healthz/live
    port: 8080
  initialDelaySeconds: 3
  periodSeconds: 5
  failureThreshold: 12
envFrom:
- configMapRef:
    name: gateway-config
- secretRef:
    name: gateway-secrets
env:
- name: POD_NAME
  valueFrom:
    fieldRef:
      fieldPath: metadata.name
- name: POD_NAMESPACE
  valueFrom:
    fieldRef:
      fieldPath: metadata.namespace
- name: POD_IP
  valueFrom:
    fieldRef:
      fieldPath: status.podIP
volumeMounts:
- name: tmp
```

```yaml
          mountPath: /tmp
        - name: tls-certs
          mountPath: /etc/tls
          readOnly: true
      volumes:
      - name: tmp
        emptyDir: {}
      - name: tls-certs
        secret:
          secretName: gateway-tls
      imagePullSecrets:
      - name: ghcr-pull-secret
---
apiVersion: v1
kind: Service
metadata:
  name: gateway
  namespace: helixterm-prod
  labels:
    app: gateway
    team: platform
spec:
  selector:
    app: gateway
  ports:
  - name: http
    port: 80
    targetPort: 8080
    protocol: TCP
  - name: https
    port: 443
    targetPort: 8443
    protocol: TCP
  type: ClusterIP
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: gateway
  namespace: helixterm-prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
```

```yaml
    name: gateway
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
      - type: Pods
        value: 4
        periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
      - type: Pods
        value: 2
        periodSeconds: 60
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: gateway
  namespace: helixterm-prod
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: gateway
---
apiVersion: v1
kind: ConfigMap
metadata:
```

```yaml
  name: gateway-config
  namespace: helixterm-prod
data:
  ENVIRONMENT: "production"
  LOG_LEVEL: "info"
  LOG_FORMAT: "json"
  HTTP_PORT: "8080"
  METRICS_PORT: "9090"
  AUTH_SERVICE_URL: "http://auth-service:8080"
  VAULT_SERVICE_URL: "http://vault-service:8080"
  RATE_LIMIT_RPS: "1000"
  RATE_LIMIT_BURST: "2000"
  CORS_ALLOWED_ORIGINS: "https://app.helixterm.io"
  REQUEST_TIMEOUT_SECONDS: "30"
  OTEL_EXPORTER_OTLP_ENDPOINT: "http://otel-collector:4317"
  OTEL_SERVICE_NAME: "gateway"
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: gateway
  namespace: helixterm-prod
  labels:
    app: gateway
automountServiceAccountToken: false
```

2.4 Auth Service

```yaml
# infrastructure/kubernetes/base/auth-service/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
  namespace: helixterm-prod
  labels:
    app: auth-service
    version: v1
    team: security
spec:
  replicas: 3
  selector:
    matchLabels:
      app: auth-service
  strategy:
```

```yaml
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
template:
  metadata:
    labels:
      app: auth-service
      version: v1
      team: security
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/port: "9090"
  spec:
    serviceAccountName: auth-service
    automountServiceAccountToken: false
    securityContext:
      runAsNonRoot: true
      runAsUser: 65532
      runAsGroup: 65532
      fsGroup: 65532
      seccompProfile:
        type: RuntimeDefault
    affinity:
      podAntiAffinity:
        requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchLabels:
              app: auth-service
          topologyKey: kubernetes.io/hostname
    terminationGracePeriodSeconds: 30
    containers:
    - name: auth-service
      image: ghcr.io/helixdevelopment/auth-service:1.0.0
      imagePullPolicy: Always
      ports:
      - containerPort: 8080
        name: http
      - containerPort: 9090
        name: grpc
      - containerPort: 9091
        name: metrics
      resources:
        requests:
```

```yaml
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "1000m"
        securityContext:
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          capabilities:
            drop:
            - ALL
        livenessProbe:
          httpGet:
            path: /healthz/live
            port: 8080
          initialDelaySeconds: 10
          periodSeconds: 15
          failureThreshold: 3
        readinessProbe:
          httpGet:
            path: /healthz/ready
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 10
          failureThreshold: 3
        envFrom:
        - configMapRef:
            name: auth-service-config
        - secretRef:
            name: auth-service-secrets
        volumeMounts:
        - name: tmp
          mountPath: /tmp
      volumes:
      - name: tmp
        emptyDir: {}
      imagePullSecrets:
      - name: ghcr-pull-secret
---
apiVersion: v1
kind: Service
metadata:
  name: auth-service
  namespace: helixterm-prod
```

```yaml
spec:
  selector:
    app: auth-service
  ports:
  - name: http
    port: 8080
    targetPort: 8080
  - name: grpc
    port: 9090
    targetPort: 9090
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: auth-service
  namespace: helixterm-prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: auth-service
  minReplicas: 3
  maxReplicas: 15
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: auth-service
  namespace: helixterm-prod
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: auth-service
---
apiVersion: v1
kind: ConfigMap
```

```yaml
metadata:
  name: auth-service-config
  namespace: helixterm-prod
data:
  ENVIRONMENT: "production"
  LOG_LEVEL: "info"
  LOG_FORMAT: "json"
  HTTP_PORT: "8080"
  GRPC_PORT: "9090"
  DB_HOST: "postgresql-primary"
  DB_PORT: "5432"
  DB_NAME: "helixterm_auth"
  DB_POOL_MAX_CONNS: "25"
  REDIS_HOST: "redis-master"
  REDIS_PORT: "6379"
  JWT_ISSUER: "https://auth.helixterm.io"
  JWT_AUDIENCE: "helixterm-api"
  JWT_ACCESS_TOKEN_TTL: "900"
  JWT_REFRESH_TOKEN_TTL: "86400"
  OIDC_PROVIDER_URL: "https://sso.helixterm.io"
  MFA_TOTP_ISSUER: "HelixTerminator"
  OTEL_SERVICE_NAME: "auth-service"
  OTEL_EXPORTER_OTLP_ENDPOINT: "http://otel-collector:4317"
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: auth-service
  namespace: helixterm-prod
automountServiceAccountToken: false
```

2.5 Vault Service

```yaml
# infrastructure/kubernetes/base/vault-service/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vault-service
  namespace: helixterm-prod
  labels:
    app: vault-service
    version: v1
    team: security
spec:
```

```yaml
replicas: 3
selector:
  matchLabels:
    app: vault-service
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
template:
  metadata:
    labels:
      app: vault-service
      version: v1
      team: security
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/port: "9091"
  spec:
    serviceAccountName: vault-service
    automountServiceAccountToken: false
    securityContext:
      runAsNonRoot: true
      runAsUser: 65532
      runAsGroup: 65532
      fsGroup: 65532
      seccompProfile:
        type: RuntimeDefault
    affinity:
      podAntiAffinity:
        requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchLabels:
              app: vault-service
          topologyKey: kubernetes.io/hostname
    containers:
    - name: vault-service
      image: ghcr.io/helixdevelopment/vault-service:1.0.0
      imagePullPolicy: Always
      ports:
      - containerPort: 8080
        name: http
      - containerPort: 9090
        name: grpc
```

```
    - containerPort: 9091
      name: metrics
    resources:
      requests:
        memory: "512Mi"
        cpu: "500m"
      limits:
        memory: "1Gi"
        cpu: "2000m"
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop:
        - ALL
    livenessProbe:
      httpGet:
        path: /healthz/live
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 15
      failureThreshold: 3
    readinessProbe:
      httpGet:
        path: /healthz/ready
        port: 8080
      initialDelaySeconds: 10
      periodSeconds: 10
      failureThreshold: 3
    envFrom:
    - configMapRef:
        name: vault-service-config
    - secretRef:
        name: vault-service-secrets
    volumeMounts:
    - name: tmp
      mountPath: /tmp
    - name: vault-data
      mountPath: /data
  volumes:
  - name: tmp
    emptyDir: {}
  - name: vault-data
    persistentVolumeClaim:
```

```yaml
          claimName: vault-service-data
      imagePullSecrets:
      - name: ghcr-pull-secret
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: vault-service-data
  namespace: helixterm-prod
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: gp3-encrypted
---
apiVersion: v1
kind: Service
metadata:
  name: vault-service
  namespace: helixterm-prod
spec:
  selector:
    app: vault-service
  ports:
  - name: http
    port: 8080
    targetPort: 8080
  - name: grpc
    port: 9090
    targetPort: 9090
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: vault-service
  namespace: helixterm-prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: vault-service
  minReplicas: 3
```

```yaml
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: vault-service
  namespace: helixterm-prod
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: vault-service
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: vault-service-config
  namespace: helixterm-prod
data:
  ENVIRONMENT: "production"
  LOG_LEVEL: "info"
  HTTP_PORT: "8080"
  GRPC_PORT: "9090"
  DB_HOST: "postgresql-primary"
  DB_NAME: "helixterm_vault"
  DB_POOL_MAX_CONNS: "20"
  SHAMIR_THRESHOLD: "3"
  SHAMIR_SHARES: "5"
  ENCRYPTION_KEY_ROTATION_DAYS: "90"
  OTEL_SERVICE_NAME: "vault-service"
  OTEL_EXPORTER_OTLP_ENDPOINT: "http://otel-collector:4317"
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: vault-service
```

```
  namespace: helixterm-prod
automountServiceAccountToken: false
```

2.6 SSH Proxy Service

```
# infrastructure/kubernetes/base/ssh-proxy/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ssh-proxy
  namespace: helixterm-prod
  labels:
    app: ssh-proxy
    version: v1
    team: ssh-team
spec:
  replicas: 5
  selector:
    matchLabels:
      app: ssh-proxy
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: ssh-proxy
        version: v1
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "9090"
    spec:
      serviceAccountName: ssh-proxy
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        runAsUser: 65532
        runAsGroup: 65532
        fsGroup: 65532
        seccompProfile:
          type: RuntimeDefault
      affinity:
```

```yaml
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels:
            app: ssh-proxy
        topologyKey: kubernetes.io/hostname
  containers:
  - name: ssh-proxy
    image: ghcr.io/helixdevelopment/ssh-proxy:1.0.0
    imagePullPolicy: Always
    ports:
    - containerPort: 2222
      name: ssh
      protocol: TCP
    - containerPort: 8080
      name: http
      protocol: TCP
    - containerPort: 9090
      name: metrics
      protocol: TCP
    resources:
      requests:
        memory: "512Mi"
        cpu: "500m"
      limits:
        memory: "2Gi"
        cpu: "4000m"
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop:
        - ALL
    livenessProbe:
      httpGet:
        path: /healthz/live
        port: 8080
      initialDelaySeconds: 10
      periodSeconds: 15
      failureThreshold: 3
    readinessProbe:
      tcpSocket:
        port: 2222
      initialDelaySeconds: 5
```

```yaml
          periodSeconds: 10
          failureThreshold: 3
        envFrom:
        - configMapRef:
            name: ssh-proxy-config
        - secretRef:
            name: ssh-proxy-secrets
        volumeMounts:
        - name: tmp
          mountPath: /tmp
        - name: host-keys
          mountPath: /etc/ssh/host-keys
          readOnly: true
      volumes:
      - name: tmp
        emptyDir: {}
      - name: host-keys
        secret:
          secretName: ssh-proxy-host-keys
      imagePullSecrets:
      - name: ghcr-pull-secret
---
apiVersion: v1
kind: Service
metadata:
  name: ssh-proxy
  namespace: helixterm-prod
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
    service.beta.kubernetes.io/aws-load-balancer-cross-zone-load-
        balancing-enabled: "true"
spec:
  selector:
    app: ssh-proxy
  ports:
  - name: ssh
    port: 22
    targetPort: 2222
    protocol: TCP
  - name: http
    port: 8080
    targetPort: 8080
    protocol: TCP
  type: LoadBalancer
```

```yaml
  externalTrafficPolicy: Local
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ssh-proxy
  namespace: helixterm-prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ssh-proxy
  minReplicas: 5
  maxReplicas: 50
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 65
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 75
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: ssh-proxy
  namespace: helixterm-prod
spec:
  minAvailable: 3
  selector:
    matchLabels:
      app: ssh-proxy
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: ssh-proxy-config
  namespace: helixterm-prod
```

```yaml
data:
  ENVIRONMENT: "production"
  SSH_PORT: "2222"
  HTTP_PORT: "8080"
  MAX_CONCURRENT_SESSIONS: "10000"
  SESSION_TIMEOUT_SECONDS: "3600"
  IDLE_TIMEOUT_SECONDS: "300"
  AUTH_SERVICE_URL: "http://auth-service:8080"
  VAULT_SERVICE_URL: "http://vault-service:8080"
  SESSION_RECORDER_URL: "http://session-recorder:8080"
  KAFKA_BROKERS: "kafka-headless:9092"
  KAFKA_TOPIC_SESSIONS: "ssh.sessions"
  KAFKA_TOPIC_EVENTS: "ssh.events"
  OTEL_SERVICE_NAME: "ssh-proxy"
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: ssh-proxy
  namespace: helixterm-prod
automountServiceAccountToken: false
```

2.7 Session Recorder Service

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: session-recorder
  namespace: helixterm-prod
  labels:
    app: session-recorder
    version: v1
    team: ssh-team
spec:
  replicas: 3
  selector:
    matchLabels:
      app: session-recorder
  template:
    metadata:
      labels:
        app: session-recorder
      annotations:
        prometheus.io/scrape: "true"
```

```yaml
    prometheus.io/port: "9090"
spec:
  serviceAccountName: session-recorder
  automountServiceAccountToken: false
  securityContext:
    runAsNonRoot: true
    runAsUser: 65532
    runAsGroup: 65532
    fsGroup: 65532
    seccompProfile:
      type: RuntimeDefault
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels:
            app: session-recorder
        topologyKey: kubernetes.io/hostname
  containers:
  - name: session-recorder
    image: ghcr.io/helixdevelopment/session-recorder:1.0.0
    imagePullPolicy: Always
    ports:
    - containerPort: 8080
      name: http
    - containerPort: 9090
      name: metrics
    resources:
      requests:
        memory: "512Mi"
        cpu: "500m"
      limits:
        memory: "2Gi"
        cpu: "2000m"
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: [ALL]
    livenessProbe:
      httpGet:
        path: /healthz/live
        port: 8080
      periodSeconds: 15
```

```yaml
        readinessProbe:
          httpGet:
            path: /healthz/ready
            port: 8080
          periodSeconds: 10
        envFrom:
        - configMapRef:
            name: session-recorder-config
        - secretRef:
            name: session-recorder-secrets
        volumeMounts:
        - name: tmp
          mountPath: /tmp
        - name: recordings-buffer
          mountPath: /recordings
      volumes:
      - name: tmp
        emptyDir: {}
      - name: recordings-buffer
        emptyDir:
          sizeLimit: 10Gi
      imagePullSecrets:
      - name: ghcr-pull-secret
---
apiVersion: v1
kind: Service
metadata:
  name: session-recorder
  namespace: helixterm-prod
spec:
  selector:
    app: session-recorder
  ports:
  - name: http
    port: 8080
    targetPort: 8080
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: session-recorder
  namespace: helixterm-prod
spec:
  scaleTargetRef:
```

```yaml
    apiVersion: apps/v1
    kind: Deployment
    name: session-recorder
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: session-recorder
  namespace: helixterm-prod
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: session-recorder
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: session-recorder-config
  namespace: helixterm-prod
data:
  ENVIRONMENT: "production"
  HTTP_PORT: "8080"
  S3_BUCKET: "helixterm-session-recordings-prod"
  S3_REGION: "us-east-1"
  UPLOAD_CONCURRENCY: "10"
  BUFFER_FLUSH_INTERVAL_SECONDS: "5"
  KAFKA_BROKERS: "kafka-headless:9092"
  KAFKA_CONSUMER_GROUP: "session-recorder"
  KAFKA_TOPIC_SESSIONS: "ssh.sessions"
  DB_HOST: "postgresql-primary"
  DB_NAME: "helixterm_recordings"
  OTEL_SERVICE_NAME: "session-recorder"
---
apiVersion: v1
```

```yaml
kind: ServiceAccount
metadata:
  name: session-recorder
  namespace: helixterm-prod
automountServiceAccountToken: false
```

2.8 Remaining Services (Abbreviated — Full Pattern Applies)

The following 20 services follow the identical Deployment/Service/HPA/PDB/ConfigMap/ServiceAccount pattern shown above. Key differentiators are documented per service.

credential-manager — Manages credential lifecycle. Resources: 256Mi/512Mi RAM, 250m/1000m CPU. Replicas: 3/10. Talks to: vault-service, postgresql, kafka.

user-service — User accounts and profiles. Resources: 256Mi/512Mi RAM, 250m/1000m CPU. Replicas: 3/15. Exposes HTTP + gRPC on 8080/9090.

organization-service — Multi-tenant org management. Resources: 256Mi/512Mi RAM, 250m/1000m CPU. Replicas: 3/10.

rbac-service — Real-time RBAC evaluation with Redis caching. Resources: 512Mi/1Gi RAM, 500m/2000m CPU. Replicas: 3/20. High-traffic service; scale aggressively.

audit-service — Write-only immutable audit ingestion. Resources: 512Mi/1Gi RAM, 500m/2000m CPU. Replicas: 3/20. Kafka consumer + PostgreSQL writer.

notification-service — Email/Slack/webhook. Resources: 256Mi/512Mi RAM, 250m/500m CPU. Replicas: 2/10. RabbitMQ consumer.

scheduler-service — Singleton-style leader election for cron jobs. Resources: 256Mi/512Mi RAM, 250m/500m CPU. Replicas: 3 (only leader active). Uses distributed lock via Redis.

inventory-service — Server inventory. Resources: 256Mi/512Mi RAM, 250m/1000m CPU. Replicas: 3/10.

key-rotation-service — Async key rotation via Kafka. Resources: 256Mi/512Mi RAM, 250m/500m CPU. Replicas: 2/5.

compliance-service — OPA-based evaluation. Resources: 512Mi/2Gi RAM, 500m/4000m CPU. Replicas: 3/15.

reporting-service — Async PDF/CSV report generation. Resources: 512Mi/2Gi RAM, 1000m/4000m CPU. Replicas: 2/10. Long-running jobs; terminationGracePeriodSeconds: 300.

billing-service — Usage metering. Resources: 256Mi/512Mi RAM, 250m/1000m CPU. Replicas: 3/10. Stripe integration via secrets.

search-service — Elasticsearch client. Resources: 256Mi/512Mi RAM, 250m/1000m CPU. Replicas: 3/10.

webhook-service — Outbound webhook delivery with retry queue. Resources: 256Mi/512Mi RAM, 250m/1000m CPU. Replicas: 3/10. RabbitMQ consumer.

approval-workflow-service — State machine for multi-step approvals. Resources: 256Mi/512Mi RAM, 250m/1000m CPU. Replicas: 3/10. PostgreSQL-backed state.

certificate-service — X.509 lifecycle, integrates with cert-manager and AWS ACM. Resources: 256Mi/512Mi RAM, 250m/1000m CPU. Replicas: 3/5.

tunnel-service — TCP tunnel management. Resources: 512Mi/2Gi RAM, 500m/4000m CPU. Replicas: 3/20. LoadBalancer service on port 443.

metrics-aggregator — Rolls up internal metrics for billing/reporting. Resources: 256Mi/512Mi RAM, 250m/1000m CPU. Replicas: 2/5.

policy-engine — OPA evaluation endpoint. Resources: 512Mi/1Gi RAM, 500m/2000m CPU. Replicas: 3/15. High-availability critical path.

file-transfer-service — SCP/SFTP brokering via SSH subsystem. Resources: 512Mi/2Gi RAM, 500m/2000m CPU. Replicas: 3/10. S3 streaming upload.

2.9 StatefulSet — Kafka

```
# infrastructure/kubernetes/base/kafka/statefulset.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: kafka
  namespace: helixterm-prod
```

```yaml
  labels:
    app: kafka
    component: messaging
spec:
  serviceName: kafka-headless
  replicas: 3
  selector:
    matchLabels:
      app: kafka
  podManagementPolicy: Parallel
  updateStrategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: kafka
        component: messaging
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "9308"
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 1001
        fsGroup: 1001
        seccompProfile:
          type: RuntimeDefault
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchLabels:
                app: kafka
            topologyKey: kubernetes.io/hostname
      containers:
      - name: kafka
        image: bitnami/kafka:3.9.0
        ports:
        - containerPort: 9092
          name: internal
        - containerPort: 9094
          name: controller
        resources:
          requests:
```

```
        memory: "2Gi"
        cpu: "1000m"
      limits:
        memory: "8Gi"
        cpu: "4000m"
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: false
      capabilities:
        drop: [ALL]
    env:
    - name: KAFKA_CFG_PROCESS_ROLES
      value: "broker,controller"
    - name: KAFKA_CFG_NODE_ID
      valueFrom:
        fieldRef:
          fieldPath:
    metadata.annotations['statefulset.kubernetes.io/pod-name']
    - name: KAFKA_CFG_CONTROLLER_QUORUM_VOTERS
      value: "0@kafka-0.kafka-headless:9094,1@kafka-1.kafka-
    headless:9094,2@kafka-2.kafka-headless:9094"
    - name: KAFKA_CFG_LISTENERS
      value: "PLAINTEXT://:9092,CONTROLLER://:9094"
    - name: KAFKA_CFG_LOG_RETENTION_HOURS
      value: "168"
    - name: KAFKA_CFG_LOG_SEGMENT_BYTES
      value: "1073741824"
    - name: KAFKA_CFG_NUM_PARTITIONS
      value: "12"
    - name: KAFKA_CFG_DEFAULT_REPLICATION_FACTOR
      value: "3"
    - name: KAFKA_CFG_MIN_INSYNC_REPLICAS
      value: "2"
    volumeMounts:
    - name: kafka-data
      mountPath: /bitnami/kafka
  - name: kafka-exporter
    image: danielqsj/kafka-exporter:v1.7.0
    ports:
    - containerPort: 9308
      name: metrics
    resources:
      requests:
        memory: "64Mi"
        cpu: "50m"
```

```
          limits:
            memory: "128Mi"
            cpu: "200m"
  volumeClaimTemplates:
  - metadata:
      name: kafka-data
    spec:
      accessModes: [ReadWriteOnce]
      storageClassName: gp3-encrypted
      resources:
        requests:
          storage: 200Gi
---
apiVersion: v1
kind: Service
metadata:
  name: kafka-headless
  namespace: helixterm-prod
spec:
  clusterIP: None
  selector:
    app: kafka
  ports:
  - name: internal
    port: 9092
  - name: controller
    port: 9094
---
apiVersion: v1
kind: Service
metadata:
  name: kafka
  namespace: helixterm-prod
spec:
  selector:
    app: kafka
  ports:
  - name: internal
    port: 9092
    targetPort: 9092
```

2.10 StatefulSet — PostgreSQL

```yaml
# infrastructure/kubernetes/base/postgresql/statefulset.yaml
# Note: In production, PostgreSQL is managed by AWS RDS Multi-AZ.
# This StatefulSet is used for staging and integration test
       environments.
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgresql
  namespace: helixterm-prod
  labels:
    app: postgresql
    component: database
spec:
  serviceName: postgresql-headless
  replicas: 3
  selector:
    matchLabels:
      app: postgresql
  template:
    metadata:
      labels:
        app: postgresql
        component: database
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 1001
        fsGroup: 1001
        seccompProfile:
          type: RuntimeDefault
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchLabels:
                app: postgresql
            topologyKey: kubernetes.io/hostname
      initContainers:
      - name: init-data-dir
        image: busybox:1.36
        command: ['sh', '-c', 'chown -R 1001:1001 /bitnami/
        postgresql']
        securityContext:
```

```
      runAsUser: 0
    volumeMounts:
    - name: pg-data
      mountPath: /bitnami/postgresql
  containers:
  - name: postgresql
    image: bitnami/postgresql:17.2.0
    ports:
    - containerPort: 5432
      name: postgresql
    resources:
      requests:
        memory: "2Gi"
        cpu: "1000m"
      limits:
        memory: "8Gi"
        cpu: "4000m"
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop: [ALL]
    env:
    - name: POSTGRESQL_REPLICATION_MODE
      value: "master"
    - name: POSTGRESQL_REPLICATION_USER
      value: "replicator"
    - name: POSTGRESQL_REPLICATION_PASSWORD
      valueFrom:
        secretKeyRef:
          name: postgresql-secrets
          key: replication-password
    - name: POSTGRESQL_PASSWORD
      valueFrom:
        secretKeyRef:
          name: postgresql-secrets
          key: postgres-password
    - name: POSTGRESQL_SHARED_PRELOAD_LIBRARIES
      value: "pg_stat_statements,pgaudit"
    - name: POSTGRESQL_MAX_CONNECTIONS
      value: "500"
    - name: POSTGRESQL_WAL_LEVEL
      value: "replica"
    - name: POSTGRESQL_MAX_WAL_SENDERS
      value: "10"
```

```
        livenessProbe:
          exec:
            command: ['pg_isready', '-U', 'postgres']
          initialDelaySeconds: 30
          periodSeconds: 15
        readinessProbe:
          exec:
            command: ['pg_isready', '-U', 'postgres']
          initialDelaySeconds: 15
          periodSeconds: 10
        volumeMounts:
        - name: pg-data
          mountPath: /bitnami/postgresql
  volumeClaimTemplates:
  - metadata:
      name: pg-data
    spec:
      accessModes: [ReadWriteOnce]
      storageClassName: gp3-encrypted
      resources:
        requests:
          storage: 500Gi
---
apiVersion: v1
kind: Service
metadata:
  name: postgresql-primary
  namespace: helixterm-prod
spec:
  selector:
    app: postgresql
    role: primary
  ports:
  - name: postgresql
    port: 5432
    targetPort: 5432
---
apiVersion: v1
kind: Service
metadata:
  name: postgresql-headless
  namespace: helixterm-prod
spec:
  clusterIP: None
```

```
  selector:
    app: postgresql
  ports:
  - port: 5432
```

2.11 StatefulSet — Redis

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
  namespace: helixterm-prod
  labels:
    app: redis
    component: cache
spec:
  serviceName: redis-headless
  replicas: 3
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
        component: cache
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 1001
        fsGroup: 1001
        seccompProfile:
          type: RuntimeDefault
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchLabels:
                app: redis
            topologyKey: kubernetes.io/hostname
      containers:
      - name: redis
        image: bitnami/redis:8.0.0
        ports:
```

```
      - containerPort: 6379
        name: redis
      resources:
        requests:
          memory: "1Gi"
          cpu: "500m"
        limits:
          memory: "4Gi"
          cpu: "2000m"
      securityContext:
        allowPrivilegeEscalation: false
        capabilities:
          drop: [ALL]
      env:
      - name: REDIS_REPLICATION_MODE
        value: "master"
      - name: REDIS_PASSWORD
        valueFrom:
          secretKeyRef:
            name: redis-secrets
            key: redis-password
      - name: REDIS_AOF_ENABLED
        value: "yes"
      - name: REDIS_MAXMEMORY
        value: "3gb"
      - name: REDIS_MAXMEMORY_POLICY
        value: "allkeys-lru"
      livenessProbe:
        exec:
          command: ['redis-cli', 'ping']
        initialDelaySeconds: 30
        periodSeconds: 10
      readinessProbe:
        exec:
          command: ['redis-cli', 'ping']
        initialDelaySeconds: 5
        periodSeconds: 10
      volumeMounts:
      - name: redis-data
        mountPath: /bitnami/redis/data
volumeClaimTemplates:
- metadata:
    name: redis-data
  spec:
```

```
      accessModes: [ReadWriteOnce]
      storageClassName: gp3-encrypted
      resources:
        requests:
          storage: 50Gi
---
apiVersion: v1
kind: Service
metadata:
  name: redis-master
  namespace: helixterm-prod
spec:
  selector:
    app: redis
    role: master
  ports:
  - port: 6379
    targetPort: 6379
```

2.12 Ingress Configuration

```
# infrastructure/kubernetes/base/ingress/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: helixterm-ingress
  namespace: helixterm-prod
  annotations:
    kubernetes.io/ingress.class: nginx
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-body-size: "100m"
    nginx.ingress.kubernetes.io/rate-limit: "on"
    nginx.ingress.kubernetes.io/limit-rps: "100"
    nginx.ingress.kubernetes.io/limit-connections: "25"
    nginx.ingress.kubernetes.io/enable-modsecurity: "true"
    nginx.ingress.kubernetes.io/enable-owasp-core-rules: "true"
    nginx.ingress.kubernetes.io/configuration-snippet: |
      more_set_headers "X-Frame-Options: DENY";
      more_set_headers "X-Content-Type-Options: nosniff";
      more_set_headers "X-XSS-Protection: 1; mode=block";
```

```
      more_set_headers "Referrer-Policy: strict-origin-when-cross-
        origin";
      more_set_headers "Permissions-Policy: camera=(),
        microphone=(), geolocation=()";
      more_set_headers "Strict-Transport-Security: max-
        age=31536000; includeSubDomains; preload";
spec:
  tls:
  - hosts:
    - app.helixterm.io
    - api.helixterm.io
    secretName: helixterm-tls
  rules:
  - host: api.helixterm.io
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: gateway
            port:
              number: 80
  - host: app.helixterm.io
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: flutter-web
            port:
              number: 80
---
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: platform@helixterm.io
    privateKeySecretRef:
      name: letsencrypt-prod-key
    solvers:
```

```yaml
    - http01:
        ingress:
          class: nginx
    - dns01:
        route53:
          region: us-east-1
          hostedZoneID: ZXXXXXXXXXXXX
---
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-staging
spec:
  acme:
    server: https://acme-staging-v02.api.letsencrypt.org/directory
    email: platform@helixterm.io
    privateKeySecretRef:
      name: letsencrypt-staging-key
    solvers:
    - http01:
        ingress:
          class: nginx
```

---

3. Helm Charts

3.1 Umbrella Chart Structure

```
infrastructure/helm/helixterm/
├── Chart.yaml                    # Umbrella chart metadata
├── values.yaml                   # Default values (all
environments)
├── values-production.yaml        # Production overrides
├── values-staging.yaml           # Staging overrides
├── values-development.yaml       # Development overrides
├── templates/
│   ├── _helpers.tpl              # Named template helpers
│   ├── namespace.yaml
│   ├── imagepullsecret.yaml
│   └── NOTES.txt
└── charts/                       # Sub-charts (one per service)
    ├── gateway/
    ├── auth-service/
```

```
    ├── vault-service/
    ├── ssh-proxy/
    ├── session-recorder/
    ├── credential-manager/
    ├── user-service/
    ├── organization-service/
    ├── rbac-service/
    ├── audit-service/
    ├── notification-service/
    ├── scheduler-service/
    ├── inventory-service/
    ├── key-rotation-service/
    ├── compliance-service/
    ├── reporting-service/
    ├── billing-service/
    ├── search-service/
    ├── webhook-service/
    ├── approval-workflow-service/
    ├── certificate-service/
    ├── tunnel-service/
    ├── metrics-aggregator/
    ├── policy-engine/
    └── file-transfer-service/
```

3.2 Chart.yaml

```yaml
# infrastructure/helm/helixterm/Chart.yaml
apiVersion: v2
name: helixterm
description: HelixTerminator — Privileged Access Management
        Platform
type: application
version: 1.0.0
appVersion: "1.0.0"
keywords:
  - pam
  - ssh
  - privileged-access
  - security
home: https://helixterm.io
sources:
  - https://github.com/helixterm/helix-terminator
maintainers:
  - name: HelixTerminator Platform Team
```

```
    email: platform@helixterm.io

dependencies:
  - name: kafka
    version: "26.8.5"
    repository: "https://charts.bitnami.com/bitnami"
    condition: kafka.enabled

  - name: postgresql
    version: "13.4.4"
    repository: "https://charts.bitnami.com/bitnami"
    condition: postgresql.enabled

  - name: redis
    version: "18.19.4"
    repository: "https://charts.bitnami.com/bitnami"
    condition: redis.enabled

  - name: rabbitmq
    version: "14.6.6"
    repository: "https://charts.bitnami.com/bitnami"
    condition: rabbitmq.enabled

  - name: cert-manager
    version: "v1.14.5"
    repository: "https://charts.jetstack.io"
    condition: certManager.enabled

  - name: ingress-nginx
    version: "4.10.1"
    repository: "https://kubernetes.github.io/ingress-nginx"
    condition: ingressNginx.enabled
```

3.3 values.yaml (Default)

```
# infrastructure/helm/helixterm/values.yaml
global:
  imageRegistry: "ghcr.io/helixterm"
  imagePullSecrets:
    - name: ghcr-pull-secret
  imageTag: "latest"
  environment: "development"
  logLevel: "debug"
  logFormat: "text"
```

```yaml
  otel:
    enabled: true
    endpoint: "http://otel-collector:4317"

  kafka:
    brokers: "kafka-headless:9092"

  postgresql:
    host: "postgresql-primary"
    port: 5432

  redis:
    host: "redis-master"
    port: 6379

  rabbitmq:
    host: "rabbitmq"
    port: 5672

# --- Gateway ---
gateway:
  enabled: true
  replicaCount: 2
  image:
    repository: gateway
    tag: ""
    pullPolicy: IfNotPresent
  service:
    type: ClusterIP
    httpPort: 80
    httpsPort: 443
  resources:
    requests:
      memory: "128Mi"
      cpu: "100m"
    limits:
      memory: "256Mi"
      cpu: "500m"
  autoscaling:
    enabled: true
    minReplicas: 2
    maxReplicas: 10
    targetCPUUtilizationPercentage: 70
```

```yaml
  config:
    rateLimitRPS: "100"
    requestTimeoutSeconds: "30"
  ingress:
    enabled: true
    host: "api.localhost"
    tls: false

# --- Auth Service ---
authService:
  enabled: true
  replicaCount: 2
  image:
    repository: auth-service
    tag: ""
    pullPolicy: IfNotPresent
  resources:
    requests:
      memory: "128Mi"
      cpu: "100m"
    limits:
      memory: "256Mi"
      cpu: "500m"
  autoscaling:
    enabled: true
    minReplicas: 2
    maxReplicas: 8
  config:
    jwtAccessTokenTTL: "900"
    jwtRefreshTokenTTL: "86400"
    mfaTotpIssuer: "HelixTerminator"

# --- Vault Service ---
vaultService:
  enabled: true
  replicaCount: 2
  image:
    repository: vault-service
    tag: ""
    pullPolicy: IfNotPresent
  resources:
    requests:
      memory: "256Mi"
      cpu: "250m"
```

```yaml
    limits:
      memory: "512Mi"
      cpu: "1000m"
  autoscaling:
    enabled: true
    minReplicas: 2
    maxReplicas: 6
  persistence:
    enabled: true
    storageClass: "standard"
    size: "5Gi"
  config:
    shamirThreshold: "3"
    shamirShares: "5"

# --- SSH Proxy ---
sshProxy:
  enabled: true
  replicaCount: 2
  image:
    repository: ssh-proxy
    tag: ""
    pullPolicy: IfNotPresent
  service:
    type: NodePort
    sshPort: 2222
    nodePort: 32222
  resources:
    requests:
      memory: "256Mi"
      cpu: "250m"
    limits:
      memory: "1Gi"
      cpu: "2000m"
  autoscaling:
    enabled: true
    minReplicas: 2
    maxReplicas: 20
  config:
    maxConcurrentSessions: "100"
    sessionTimeoutSeconds: "3600"

# --- Session Recorder ---
sessionRecorder:
```

```yaml
  enabled: true
  replicaCount: 2
  image:
    repository: session-recorder
    tag: ""
  resources:
    requests:
      memory: "256Mi"
      cpu: "250m"
    limits:
      memory: "1Gi"
      cpu: "1000m"
  config:
    s3Bucket: "helixterm-session-recordings-dev"
    s3Region: "us-east-1"

# --- All remaining services follow same pattern ---
credentialManager:
  enabled: true
  replicaCount: 1
  resources:
    requests:
      memory: "128Mi"
      cpu: "100m"
    limits:
      memory: "256Mi"
      cpu: "500m"

userService:
  enabled: true
  replicaCount: 2

organizationService:
  enabled: true
  replicaCount: 2

rbacService:
  enabled: true
  replicaCount: 2

auditService:
  enabled: true
  replicaCount: 2
```

```yaml
notificationService:
  enabled: true
  replicaCount: 1

schedulerService:
  enabled: true
  replicaCount: 1

inventoryService:
  enabled: true
  replicaCount: 2

keyRotationService:
  enabled: true
  replicaCount: 1

complianceService:
  enabled: true
  replicaCount: 2

reportingService:
  enabled: true
  replicaCount: 1

billingService:
  enabled: true
  replicaCount: 2

searchService:
  enabled: true
  replicaCount: 2

webhookService:
  enabled: true
  replicaCount: 2

approvalWorkflowService:
  enabled: true
  replicaCount: 2

certificateService:
  enabled: true
  replicaCount: 2
```

```yaml
tunnelService:
  enabled: true
  replicaCount: 2

metricsAggregator:
  enabled: true
  replicaCount: 1

policyEngine:
  enabled: true
  replicaCount: 2

fileTransferService:
  enabled: true
  replicaCount: 2

# --- Infrastructure dependencies ---
kafka:
  enabled: true
  replicaCount: 1
  kraft:
    enabled: true
  persistence:
    enabled: true
    size: "10Gi"

postgresql:
  enabled: true
  auth:
    existingSecret: postgresql-secrets
  primary:
    persistence:
      enabled: true
      size: "20Gi"

redis:
  enabled: true
  auth:
    existingSecret: redis-secrets
  master:
    persistence:
      enabled: true
      size: "5Gi"
```

```yaml
rabbitmq:
  enabled: true
  auth:
    existingPasswordSecret: rabbitmq-secrets
  persistence:
    enabled: true
    size: "5Gi"

certManager:
  enabled: false

ingressNginx:
  enabled: false
```

3.4 values-production.yaml

```yaml
# infrastructure/helm/helixterm/values-production.yaml
global:
  imageTag: ""  # Set at deploy time via --set
      global.imageTag=<sha>
  environment: "production"
  logLevel: "info"
  logFormat: "json"

  postgresql:
    host: "helixterm-prod.cluster-xxxx.us-
      east-1.rds.amazonaws.com"
    port: 5432

  redis:
    host: "helixterm-
      prod.xxxxxx.clustercfg.use1.cache.amazonaws.com"
    port: 6379

  kafka:
    brokers: "b-1.helixterm.kafka.us-
      east-1.amazonaws.com:9092,b-2.helixterm.kafka.us-
      east-1.amazonaws.com:9092,b-3.helixterm.kafka.us-
      east-1.amazonaws.com:9092"

  # Amazon MQ (managed RabbitMQ) — production replacement for the
      in-cluster
  # rabbitmq chart disabled below. See §6.2 "Amazon MQ (Managed
      RabbitMQ)".
  rabbitmq:
    host: "helixterm-prod.mq.us-east-1.amazonaws.com"
    port: 5671
```

```yaml
    tls: true

gateway:
  replicaCount: 3
  resources:
    requests:
      memory: "256Mi"
      cpu: "250m"
    limits:
      memory: "512Mi"
      cpu: "1000m"
  autoscaling:
    minReplicas: 3
    maxReplicas: 20
  config:
    rateLimitRPS: "1000"
  ingress:
    host: "api.helixterm.io"
    tls: true

authService:
  replicaCount: 3
  resources:
    requests:
      memory: "256Mi"
      cpu: "250m"
    limits:
      memory: "512Mi"
      cpu: "1000m"
  autoscaling:
    minReplicas: 3
    maxReplicas: 15

vaultService:
  replicaCount: 3
  resources:
    requests:
      memory: "512Mi"
      cpu: "500m"
    limits:
      memory: "1Gi"
      cpu: "2000m"
  persistence:
    storageClass: "gp3-encrypted"
```

```yaml
    size: "10Gi"

sshProxy:
  replicaCount: 5
  service:
    type: LoadBalancer
  resources:
    requests:
      memory: "512Mi"
      cpu: "500m"
    limits:
      memory: "2Gi"
      cpu: "4000m"
  autoscaling:
    minReplicas: 5
    maxReplicas: 50
  config:
    maxConcurrentSessions: "10000"

sessionRecorder:
  replicaCount: 3
  config:
    s3Bucket: "helixterm-session-recordings-prod"
  resources:
    limits:
      memory: "2Gi"
      cpu: "2000m"

rbacService:
  replicaCount: 3
  resources:
    limits:
      memory: "1Gi"
      cpu: "2000m"
  autoscaling:
    minReplicas: 3
    maxReplicas: 20

auditService:
  replicaCount: 3
  resources:
    limits:
      memory: "1Gi"
      cpu: "2000m"
```

```yaml
# Disable in-cluster databases/brokers — use managed services in
        production.
# kafka -> MSK, postgresql -> RDS Multi-AZ, redis -> ElastiCache,
# rabbitmq -> Amazon MQ (CLUSTER_MULTI_AZ, §6.2). Every one of
        these four has
# a corresponding Terraform-managed resource; none is "disabled
        and left
# unprovisioned" — see the RabbitMQ production-path decision
        comment above
# the `aws_mq_broker.helixterm` resource in §6.2.
kafka:
  enabled: false

postgresql:
  enabled: false

redis:
  enabled: false

rabbitmq:
  enabled: false

certManager:
  enabled: true

ingressNginx:
  enabled: true
  controller:
    replicaCount: 3
    resources:
      requests:
        memory: "256Mi"
        cpu: "250m"
      limits:
        memory: "1Gi"
        cpu: "2000m"
```

3.5 _helpers.tpl

```
{{/*
infrastructure/helm/helixterm/templates/_helpers.tpl
Reusable template helpers for all HelixTerminator Helm charts.
*/}}
```

```
{{/*
Expand the name of the chart.
*/}}
{{- define "helixterm.name" -}}
{{- default .Chart.Name .Values.nameOverride | trunc 63 |
trimSuffix "-" }}
{{- end }}

{{/*
Create a default fully qualified app name.
*/}}
{{- define "helixterm.fullname" -}}
{{- if .Values.fullnameOverride }}
{{- .Values.fullnameOverride | trunc 63 | trimSuffix "-" }}
{{- else }}
{{- $name := default .Chart.Name .Values.nameOverride }}
{{- if contains $name .Release.Name }}
{{- .Release.Name | trunc 63 | trimSuffix "-" }}
{{- else }}
{{- printf "%s-%s" .Release.Name $name | trunc 63 | trimSuffix
"-" }}
{{- end }}
{{- end }}
{{- end }}

{{/*
Create chart label.
*/}}
{{- define "helixterm.chart" -}}
{{- printf "%s-%s" .Chart.Name .Chart.Version | replace "+" "_" |
trunc 63 | trimSuffix "-" }}
{{- end }}

{{/*
Common labels applied to every resource.
*/}}
{{- define "helixterm.labels" -}}
helm.sh/chart: {{ include "helixterm.chart" . }}
{{ include "helixterm.selectorLabels" . }}
{{- if .Chart.AppVersion }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
{{- end }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
environment: {{ .Values.global.environment | default
```

```
"development" }}
{{- end }}

{{/*
Selector labels.
*/}}
{{- define "helixterm.selectorLabels" -}}
app.kubernetes.io/name: {{ include "helixterm.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}

{{/*
Create the name of the service account to use.
*/}}
{{- define "helixterm.serviceAccountName" -}}
{{- if .Values.serviceAccount.create }}
{{- default (include
"helixterm.fullname" .) .Values.serviceAccount.name }}
{{- else }}
{{- default "default" .Values.serviceAccount.name }}
{{- end }}
{{- end }}

{{/*
Image reference helper — resolves registry, repository, and tag.
Usage: {{ include "helixterm.image" (dict "global" .Values.global
"service" .Values.gateway) }}
*/}}
{{- define "helixterm.image" -}}
{{- $registry := .global.imageRegistry | default "ghcr.io/
helixterm" -}}
{{- $repo := .service.image.repository -}}
{{- $tag := .service.image.tag | default .global.imageTag |
default "latest" -}}
{{- printf "%s/%s:%s" $registry $repo $tag -}}
{{- end }}

{{/*
Standard security context for all pods (restricted PSS).
*/}}
{{- define "helixterm.podSecurityContext" -}}
runAsNonRoot: true
runAsUser: 65532
runAsGroup: 65532
```

```
fsGroup: 65532
seccompProfile:
  type: RuntimeDefault
{{- end }}

{{/*
Standard security context for all containers.
*/}}
{{- define "helixterm.containerSecurityContext" -}}
allowPrivilegeEscalation: false
readOnlyRootFilesystem: true
capabilities:
  drop:
  - ALL
{{- end }}

{{/*
Standard OTEL environment variables.
*/}}
{{- define "helixterm.otelEnv" -}}
- name: OTEL_EXPORTER_OTLP_ENDPOINT
  value: {{ .Values.global.otel.endpoint | default "http://otel-
collector:4317" | quote }}
- name: OTEL_TRACES_SAMPLER
  value: "parentbased_traceidratio"
- name: OTEL_TRACES_SAMPLER_ARG
  value: "0.1"
- name: OTEL_PROPAGATORS
  value: "tracecontext,baggage"
{{- end }}

{{/*
Pod identity environment variables.
*/}}
{{- define "helixterm.podEnv" -}}
- name: POD_NAME
  valueFrom:
    fieldRef:
      fieldPath: metadata.name
- name: POD_NAMESPACE
  valueFrom:
    fieldRef:
      fieldPath: metadata.namespace
- name: POD_IP
```

```
  valueFrom:
    fieldRef:
      fieldPath: status.podIP
- name: NODE_NAME
  valueFrom:
    fieldRef:
      fieldPath: spec.nodeName
{{- end }}

{{/*
Standard liveness probe.
Usage: {{ include "helixterm.livenessProbe" (dict "port" 8080
"path" "/healthz/live") }}
*/}}
{{- define "helixterm.livenessProbe" -}}
httpGet:
  path: {{ .path | default "/healthz/live" }}
  port: {{ .port | default 8080 }}
initialDelaySeconds: 10
periodSeconds: 15
failureThreshold: 3
timeoutSeconds: 5
{{- end }}

{{/*
Standard readiness probe.
*/}}
{{- define "helixterm.readinessProbe" -}}
httpGet:
  path: {{ .path | default "/healthz/ready" }}
  port: {{ .port | default 8080 }}
initialDelaySeconds: 5
periodSeconds: 10
failureThreshold: 3
timeoutSeconds: 5
{{- end }}

{{/*
Standard anti-affinity rule (prefer different hosts).
*/}}
{{- define "helixterm.podAntiAffinity" -}}
podAntiAffinity:
  requiredDuringSchedulingIgnoredDuringExecution:
  - labelSelector:
```

```
      matchLabels:
        app: {{ .appName }}
    topologyKey: kubernetes.io/hostname
  preferredDuringSchedulingIgnoredDuringExecution:
  - weight: 100
    podAffinityTerm:
      labelSelector:
        matchLabels:
          app: {{ .appName }}
      topologyKey: topology.kubernetes.io/zone
{{- end }}
```

---

4. Docker & Podman Build

> **NOTE — rootless Podman is the mandated target runtime (Constitution §11.4.161):** HelixConstitution §11.4.161 mandates rootless Podman as the target container build/runtime for HelixTerminator, with Docker rootful mode forbidden except where a target platform genuinely has no rootless option (§11.4.112, documented per §11.4.112). Local development already supports `PodmanRuntime` as an alternative to `DockerRuntime` (see Appendix B). CI is specified below with the same rootless target — no `docker/setup-buildx-action` or Docker daemon anywhere in §5's build jobs.

The GitHub-hosted `ubuntu-24.04` runner ships Podman + Buildah preinstalled and unprivileged, so every image-build job in §5.1/§5.2/§5.3 uses one reusable composite action instead of hand-rolled `docker build` steps:

```yaml
# .github/actions/rootless-podman-build/action.yml
name: 'Rootless Podman Build & Push'
description: 'Build a multi-stage image with rootless Buildah and
        push to Harbor (or GHCR for pre-Harbor bootstrap)'
inputs:
  context:
    description: 'Build context path'
    required: true
  containerfile:
    description: 'Path to the Containerfile/Dockerfile'
    required: true
  image:
    description: 'Fully-qualified image name (registry/org/
        service)'
```

```yaml
    required: true
  tags:
    description: 'Space-separated tags (e.g. "sha-abc123 pr-42")'
    required: true
  build-args:
    description: 'Newline-separated BUILD_ARG=value pairs'
    required: false
    default: ''
  registry:
    description: 'Registry host'
    required: true
  registry-username:
    description: 'Registry username'
    required: true
  registry-password:
    description: 'Registry password/token'
    required: true
outputs:
  digest:
    description: 'Pushed image digest'
    value: ${{ steps.push.outputs.digest }}
runs:
  using: 'composite'
  steps:
    - name: Verify rootless Podman is available
      shell: bash
      run: |
        podman info --format '{{.Host.Security.Rootless}}' | grep
        -qx true \
          || { echo "::error::runner is not rootless — refusing to
        build (Constitution §11.4.161)"; exit 1; }
        buildah --version

    - id: build
      name: Build image (rootless buildah bud)
      uses: redhat-actions/buildah-build@v2
      with:
        context: ${{ inputs.context }}
        containerfiles: ${{ inputs.containerfile }}
        image: ${{ inputs.image }}
        tags: ${{ inputs.tags }}
        build-args: ${{ inputs.build-args }}
        oci: true
        layers: true
        extra-args: |
```

```
          --userns=keep-id
          --isolation=rootless

    - id: push
      name: Push image (rootless skopeo copy, via podman)
      uses: redhat-actions/push-to-registry@v2
      with:
        image: ${{ steps.build.outputs.image }}
        tags: ${{ steps.build.outputs.tags }}
        registry: ${{ inputs.registry }}
        username: ${{ inputs.registry-username }}
        password: ${{ inputs.registry-password }}
```

Every job in §5.1 (`build-images`), §5.2 (`main-build-push`), and §5.3 (`build-and-push-release`) that currently sequences `docker/setup-buildx-action` → `docker/build-push-action` is replaced one-for-one by a single `uses: ./.github/actions/rootless-podman-build` step carrying the same `context` / `file` / `tags` / `build-args` inputs those jobs already pass — no Docker daemon is started on the runner at any point, and the `buildah bud --isolation=rootless --userns=keep-id` flags are the mechanical enforcement of the §11.4.161 mandate (a build that ever elevates to root fails the `podman info --format '{{.Host.Security.Rootless}}'` guard step above before it can produce an image). Harbor (§6.3) is the `registry` input in production; GHCR remains the bootstrap target only until Harbor's first production rollout completes.

4.1 Go Service Dockerfile (Standard Pattern)

```
# infrastructure/docker/images/go-service.Dockerfile
# Multi-stage build for all Go 1.25 microservices.
# Final image is distroless/static:nonroot — no shell, no package
      manager.
# Build args allow per-service customization.

ARG GO_VERSION=1.25
ARG SERVICE_NAME=service

# ──────────────────────────────────────────────────────────────────
# Stage 1: dependency cache
# Separated from build stage so that go mod download is only re-
      run
# when go.mod / go.sum change, not on every source change.
# ──────────────────────────────────────────────────────────────────
FROM golang:${GO_VERSION}-alpine3.21 AS deps
```

```
RUN apk add --no-cache \
    git \
    ca-certificates \
    tzdata

WORKDIR /build

# Copy module files first for layer caching
COPY go.mod go.sum ./

# Download all dependencies — this layer is cached until go.mod/
        go.sum change
RUN go mod download && go mod verify

# ──────────────────────────────────────────────────────────────
# Stage 2: build
# ──────────────────────────────────────────────────────────────
FROM deps AS builder

ARG SERVICE_NAME
ARG BUILD_TIME
ARG GIT_COMMIT
ARG GIT_TAG

WORKDIR /build

# Copy all source code
COPY . .

# Build with optimizations:
# -w: disable DWARF generation (smaller binary)
# -s: disable symbol table (smaller binary)
# CGO_ENABLED=0: static binary, no C stdlib dependency
# GOOS/GOARCH: explicit cross-compilation target
RUN CGO_ENABLED=0 \
    GOOS=linux \
    GOARCH=amd64 \
    go build \
    -ldflags="-w -s \
      -X main.buildTime=${BUILD_TIME} \
      -X main.gitCommit=${GIT_COMMIT} \
      -X main.version=${GIT_TAG}" \
    -trimpath \
```

```
    -o /bin/${SERVICE_NAME} \
    ./cmd/${SERVICE_NAME}

# ──────────────────────────────────────────────────────────────
# Stage 3: test (optional — run unit tests in CI before pushing)
# ──────────────────────────────────────────────────────────────
FROM builder AS test

RUN go test -race -count=1 ./...

# ──────────────────────────────────────────────────────────────
# Stage 4: final (distroless)
# ──────────────────────────────────────────────────────────────
FROM gcr.io/distroless/static:nonroot AS final

ARG SERVICE_NAME

# Copy timezone data and CA certificates from builder
COPY --from=builder /usr/share/zoneinfo /usr/share/zoneinfo
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/
        certs/

# Copy the compiled binary
COPY --from=builder /bin/${SERVICE_NAME} /service

# nonroot user (uid 65532) — defined in distroless base
USER nonroot:nonroot

EXPOSE 8080 9090 9091

ENTRYPOINT ["/service"]
```

4.2 Per-Service Dockerfiles

Each service has its own `Dockerfile` that uses build args to customize the standard template:

```
# services/auth-service/Dockerfile
ARG GO_VERSION=1.25
ARG SERVICE_NAME=auth-service
ARG BUILD_TIME
ARG GIT_COMMIT
ARG GIT_TAG

FROM golang:${GO_VERSION}-alpine3.21 AS deps
```

```
RUN apk add --no-cache git ca-certificates tzdata
WORKDIR /build
COPY go.mod go.sum ./
RUN go mod download && go mod verify

FROM deps AS builder
ARG SERVICE_NAME
ARG BUILD_TIME
ARG GIT_COMMIT
ARG GIT_TAG
WORKDIR /build
COPY . .
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \
    go build \
    -ldflags="-w -s -X main.buildTime=${BUILD_TIME} -X
        main.gitCommit=${GIT_COMMIT} -X main.version=${GIT_TAG}" \
    -trimpath \
    -o /bin/auth-service \
    ./cmd/auth-service

FROM gcr.io/distroless/static:nonroot
COPY --from=builder /usr/share/zoneinfo /usr/share/zoneinfo
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/
        certs/
COPY --from=builder /bin/auth-service /service
USER nonroot:nonroot
EXPOSE 8080 9090 9091
ENTRYPOINT ["/service"]

# services/ssh-proxy/Dockerfile
# SSH Proxy needs additional system libraries for crypto
        operations
ARG GO_VERSION=1.25

FROM golang:${GO_VERSION}-alpine3.21 AS deps
RUN apk add --no-cache git ca-certificates tzdata openssh-keygen
WORKDIR /build
COPY go.mod go.sum ./
RUN go mod download && go mod verify

FROM deps AS builder
ARG BUILD_TIME
ARG GIT_COMMIT
ARG GIT_TAG
WORKDIR /build
COPY . .
```

```
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \
    go build \
    -ldflags="-w -s -X main.buildTime=${BUILD_TIME} -X
        main.gitCommit=${GIT_COMMIT} -X main.version=${GIT_TAG}" \
    -trimpath \
    -o /bin/ssh-proxy \
    ./cmd/ssh-proxy

FROM gcr.io/distroless/static:nonroot
COPY --from=builder /usr/share/zoneinfo /usr/share/zoneinfo
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/
        certs/
COPY --from=builder /bin/ssh-proxy /service
USER nonroot:nonroot
EXPOSE 2222 8080 9090
ENTRYPOINT ["/service"]
```

4.3 Flutter Web Dockerfile

```
# clients/flutter/Dockerfile
# Multi-stage build: Flutter web → Nginx serving

# ──────────────────────────────────────────────────────────────
# Stage 1: Flutter build
# ──────────────────────────────────────────────────────────────
FROM ghcr.io/cirruslabs/flutter:3.24.5 AS builder

WORKDIR /app

# Copy dependency files
COPY pubspec.yaml pubspec.lock ./

# Download Flutter dependencies
RUN flutter pub get

# Copy full source
COPY . .

# Build Flutter web (release mode, with service worker and tree
        shaking)
RUN flutter build web \
    --release \
    --web-renderer canvaskit \
    --dart-define=FLUTTER_WEB_CANVASKIT_URL=https://
        www.gstatic.com/flutter-canvaskit/
```

```dockerfile
# ──────────────────────────────────────────────────────────────
# Stage 2: Nginx production server
# ──────────────────────────────────────────────────────────────
FROM nginx:1.27-alpine AS final

# Remove default nginx config
RUN rm /etc/nginx/conf.d/default.conf

# Copy custom nginx config with security headers and SPA routing
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Copy built web assets
COPY --from=builder /app/build/web /usr/share/nginx/html

# Set proper permissions
RUN chown -R nginx:nginx /usr/share/nginx/html && \
    chmod -R 755 /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

```
# clients/flutter/nginx.conf
server {
    listen 80;
    server_name _;
    root /usr/share/nginx/html;
    index index.html;

    # Security headers
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin"
always;
    add_header Permissions-Policy "camera=(), microphone=(),
geolocation=()" always;
    add_header Content-Security-Policy "default-src 'self';
script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-
inline'; img-src 'self' data: https:; connect-src 'self' https://
api.helixterm.io wss://api.helixterm.io;" always;

    # Gzip compression
```

```
    gzip on;
    gzip_types text/plain text/css application/json application/
javascript text/xml application/xml;
    gzip_min_length 1000;

    # Cache static assets aggressively
    location ~* \.(js|css|wasm|png|jpg|jpeg|gif|ico|svg|woff|
woff2)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Flutter service worker — no cache
    location /flutter_service_worker.js {
        add_header Cache-Control "no-cache";
    }

    # SPA routing — all paths serve index.html
    location / {
        try_files $uri $uri/ /index.html;
        add_header Cache-Control "no-store, no-cache, must-
revalidate";
    }

    # Health check endpoint
    location /health {
        access_log off;
        return 200 "healthy\n";
        add_header Content-Type text/plain;
    }
}
```

4.4 Docker Compose (Local Development — All Services)

```yaml
# infrastructure/docker/compose/docker-compose.yml
# Local development environment — all 25 services + dependencies
# Use: docker compose up -d  OR  podman-compose up -d
version: "3.9"

networks:
  helixterm-net:
    driver: bridge
    ipam:
      config:
```

```
        - subnet: 172.20.0.0/16

volumes:
  pg-data:
  redis-data:
  kafka-data:
  rabbitmq-data:
  vault-data:
  recordings-buffer:

# ─── Infrastructure Dependencies ──────────────────────────────

services:
  postgresql:
    image: postgres:17.2-alpine
    container_name: helixterm-postgresql
    restart: unless-stopped
    environment:
      POSTGRES_USER: helixterm
      POSTGRES_PASSWORD: devpassword
      POSTGRES_MULTIPLE_DATABASES:
        helixterm_auth,helixterm_vault,helixterm_recordings,helixterm_users,helixt
    ports:
      - "5432:5432"
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./scripts/create-multiple-dbs.sh:/docker-entrypoint-
        initdb.d/create-multiple-dbs.sh
    networks:
      - helixterm-net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U helixterm"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:8-alpine
    container_name: helixterm-redis
    restart: unless-stopped
    command: redis-server --appendonly yes --maxmemory 512mb --
        maxmemory-policy allkeys-lru
    ports:
      - "6379:6379"
    volumes:
```

```yaml
    - redis-data:/data
  networks:
    - helixterm-net
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 3s
    retries: 5

kafka:
  image: bitnami/kafka:3.9.0
  container_name: helixterm-kafka
  restart: unless-stopped
  environment:
    KAFKA_CFG_NODE_ID: "1"
    KAFKA_CFG_PROCESS_ROLES: "broker,controller"
    KAFKA_CFG_CONTROLLER_QUORUM_VOTERS: "1@kafka:9094"
    KAFKA_CFG_LISTENERS: "PLAINTEXT://:9092,CONTROLLER://:9094"
    KAFKA_CFG_ADVERTISED_LISTENERS: "PLAINTEXT://localhost:9092"
    KAFKA_CFG_NUM_PARTITIONS: "3"
    KAFKA_CFG_DEFAULT_REPLICATION_FACTOR: "1"
    KAFKA_CFG_OFFSETS_TOPIC_REPLICATION_FACTOR: "1"
    KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE: "true"
    ALLOW_PLAINTEXT_LISTENER: "yes"
  ports:
    - "9092:9092"
  volumes:
    - kafka-data:/bitnami/kafka
  networks:
    - helixterm-net
  healthcheck:
    test: ["CMD-SHELL", "kafka-broker-api-versions.sh --
      bootstrap-server localhost:9092"]
    interval: 30s
    timeout: 10s
    retries: 5

rabbitmq:
  image: rabbitmq:3.13-management-alpine
  container_name: helixterm-rabbitmq
  restart: unless-stopped
  environment:
    RABBITMQ_DEFAULT_USER: helixterm
    RABBITMQ_DEFAULT_PASS: devpassword
```

```yaml
    ports:
      - "5672:5672"
      - "15672:15672"  # Management UI
    volumes:
      - rabbitmq-data:/var/lib/rabbitmq
    networks:
      - helixterm-net
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "check_running"]
      interval: 10s
      timeout: 5s
      retries: 5

  # ─── Application Services ──────────────────────────────────────

  gateway:
    image: ghcr.io/helixdevelopment/gateway:dev
    build:
      context: ../../services/gateway
      dockerfile: Dockerfile.dev
    container_name: helixterm-gateway
    restart: unless-stopped
    ports:
      - "8080:8080"
      - "9090:9090"
    environment:
      ENVIRONMENT: development
      LOG_LEVEL: debug
      LOG_FORMAT: text
      DB_HOST: postgresql
      DB_PORT: "5432"
      DB_USER: helixterm
      DB_PASSWORD: devpassword
      REDIS_HOST: redis
      REDIS_PORT: "6379"
      AUTH_SERVICE_URL: http://auth-service:8080
      VAULT_SERVICE_URL: http://vault-service:8080
      OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-collector:4317
      OTEL_SERVICE_NAME: gateway
    depends_on:
      postgresql:
        condition: service_healthy
      redis:
```

```
      condition: service_healthy
  networks:
    - helixterm-net
  volumes:
    - ../../services/gateway:/app
  healthcheck:
    test: ["CMD", "wget", "-q", "-O", "-", "http://
      localhost:8080/healthz/live"]
    interval: 10s
    timeout: 5s
    retries: 3

auth-service:
  image: ghcr.io/helixdevelopment/auth-service:dev
  build:
    context: ../../services/auth-service
    dockerfile: Dockerfile.dev
  container_name: helixterm-auth-service
  restart: unless-stopped
  ports:
    - "8081:8080"
    - "9091:9090"
  environment:
    ENVIRONMENT: development
    LOG_LEVEL: debug
    DB_HOST: postgresql
    DB_NAME: helixterm_auth
    DB_USER: helixterm
    DB_PASSWORD: devpassword
    REDIS_HOST: redis
    JWT_ISSUER: http://localhost:8081
    JWT_SECRET: dev-secret-do-not-use-in-production
    OTEL_SERVICE_NAME: auth-service
    OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-collector:4317
  depends_on:
    postgresql:
      condition: service_healthy
    redis:
      condition: service_healthy
  networks:
    - helixterm-net

vault-service:
  image: ghcr.io/helixdevelopment/vault-service:dev
```

```yaml
    build:
      context: ../../services/vault-service
      dockerfile: Dockerfile.dev
    container_name: helixterm-vault-service
    restart: unless-stopped
    ports:
      - "8082:8080"
    environment:
      ENVIRONMENT: development
      LOG_LEVEL: debug
      DB_HOST: postgresql
      DB_NAME: helixterm_vault
      DB_USER: helixterm
      DB_PASSWORD: devpassword
      MASTER_KEY: dev-master-key-32-bytes-long-here
      OTEL_SERVICE_NAME: vault-service
      OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-collector:4317
    volumes:
      - vault-data:/data
    depends_on:
      postgresql:
        condition: service_healthy
    networks:
      - helixterm-net

  ssh-proxy:
    image: ghcr.io/helixdevelopment/ssh-proxy:dev
    build:
      context: ../../services/ssh-proxy
      dockerfile: Dockerfile.dev
    container_name: helixterm-ssh-proxy
    restart: unless-stopped
    ports:
      - "2222:2222"
      - "8083:8080"
    environment:
      ENVIRONMENT: development
      SSH_PORT: "2222"
      AUTH_SERVICE_URL: http://auth-service:8080
      VAULT_SERVICE_URL: http://vault-service:8080
      SESSION_RECORDER_URL: http://session-recorder:8080
      KAFKA_BROKERS: kafka:9092
      OTEL_SERVICE_NAME: ssh-proxy
      OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-collector:4317
```

```yaml
  depends_on:
    - auth-service
    - vault-service
    - kafka
  networks:
    - helixterm-net

session-recorder:
  image: ghcr.io/helixdevelopment/session-recorder:dev
  build:
    context: ../../services/session-recorder
    dockerfile: Dockerfile.dev
  container_name: helixterm-session-recorder
  restart: unless-stopped
  ports:
    - "8084:8080"
  environment:
    ENVIRONMENT: development
    DB_HOST: postgresql
    DB_NAME: helixterm_recordings
    DB_USER: helixterm
    DB_PASSWORD: devpassword
    KAFKA_BROKERS: kafka:9092
    S3_ENDPOINT: http://localstack:4566
    S3_BUCKET: helixterm-session-recordings-dev
    AWS_ACCESS_KEY_ID: test
    AWS_SECRET_ACCESS_KEY: test
    OTEL_SERVICE_NAME: session-recorder
    OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-collector:4317
  volumes:
    - recordings-buffer:/recordings
  depends_on:
    - kafka
    - postgresql
  networks:
    - helixterm-net

# (remaining 20 services follow identical pattern — abbreviated
    for space)

# ─── Observability Stack ──────────────────────────────────────

otel-collector:
  image: otel/opentelemetry-collector-contrib:0.101.0
```

```
    container_name: helixterm-otel-collector
    restart: unless-stopped
    volumes:
      - ./otel-collector-config.yaml:/etc/otel-collector-
        config.yaml
    command: ["--config=/etc/otel-collector-config.yaml"]
    ports:
      - "4317:4317"  # OTLP gRPC
      - "4318:4318"  # OTLP HTTP
      - "8888:8888"  # Prometheus metrics
    networks:
      - helixterm-net

  jaeger:
    image: jaegertracing/all-in-one:1.58
    container_name: helixterm-jaeger
    restart: unless-stopped
    ports:
      - "16686:16686"  # Jaeger UI
      - "14250:14250"  # gRPC
    environment:
      COLLECTOR_OTLP_ENABLED: "true"
    networks:
      - helixterm-net

  prometheus:
    image: prom/prometheus:v2.52.0
    container_name: helixterm-prometheus
    restart: unless-stopped
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'
      - '--web.console.libraries=/usr/share/prometheus/
        console_libraries'
      - '--web.console.templates=/usr/share/prometheus/consoles'
    networks:
      - helixterm-net

  grafana:
    image: grafana/grafana:10.4.2
    container_name: helixterm-grafana
```

```
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: devpassword
      GF_AUTH_ANONYMOUS_ENABLED: "false"
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning
    networks:
      - helixterm-net

  localstack:
    image: localstack/localstack:3.4
    container_name: helixterm-localstack
    restart: unless-stopped
    ports:
      - "4566:4566"
    environment:
      SERVICES: s3,ses,sqs
      DEFAULT_REGION: us-east-1
      AWS_ACCESS_KEY_ID: test
      AWS_SECRET_ACCESS_KEY: test
    networks:
      - helixterm-net
```

4.5 Development Dockerfile (with Delve Debugger)

```
# services/auth-service/Dockerfile.dev
# Development image — includes delve debugger and air hot-reload
FROM golang:1.25-alpine3.21

RUN apk add --no-cache git ca-certificates tzdata curl

# Install air for hot reload
RUN go install github.com/air-verse/air@latest

# Install delve debugger
RUN go install github.com/go-delve/delve/cmd/dlv@latest

WORKDIR /app

# Copy and download dependencies
COPY go.mod go.sum ./
```

```
RUN go mod download

# Copy source (will be overridden by volume mount in dev)
COPY . .

EXPOSE 8080 9090 2345

# air watches for changes and recompiles
CMD ["air", "-c", ".air.toml"]
```

4.6 Trivy Image Scanning Configuration

```
# .trivy.yaml — Trivy vulnerability scanner configuration
# Used in CI to block builds with critical/high vulnerabilities

scan:
  security-checks:
    - vuln
    - secret
    - config
  vuln-type:
    - os
    - library

severity:
  - CRITICAL
  - HIGH

exit-code: 1

ignore-unfixed: false

format: sarif
output: trivy-results.sarif

db:
  no-progress: true
  download-java-db-only: false

cache:
  clear: false

# Ignore specific CVEs that have been triaged and accepted
ignorefile: .trivyignore
```

```yaml
misconfig:
  include-non-failures: false
```

---

5. CI/CD Pipelines

5.1 PR Pipeline

```yaml
# .github/workflows/pr.yml
name: Pull Request Validation

on:
  pull_request:
    branches: [main, release/*]
    types: [opened, synchronize, reopened]

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

env:
  GO_VERSION: "1.25"
  FLUTTER_VERSION: "3.24.5"
  GOLANGCI_LINT_VERSION: "v1.59.1"
  COVERAGE_THRESHOLD: "80"

jobs:
  # ──────────────────────────────────────────────────────────────────────
  # 1. Detect changed services for targeted testing
  # ──────────────────────────────────────────────────────────────────────
  detect-changes:
    name: Detect Changed Services
    runs-on: ubuntu-24.04
    outputs:
      services: ${{ steps.filter.outputs.services }}
      infrastructure: ${{ steps.filter.outputs.infrastructure }}
      flutter: ${{ steps.filter.outputs.flutter }}
      matrix: ${{ steps.matrix.outputs.matrix }}
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
```

```yaml
      - uses: dorny/paths-filter@v3
        id: filter
        with:
          filters: |
            services:
              - 'services/**'
            infrastructure:
              - 'infrastructure/**'
              - '.github/workflows/**'
            flutter:
              - 'clients/flutter/**'

      - name: Build service matrix
        id: matrix
        run: |
          CHANGED=$(git diff --name-only origin/main...HEAD | grep
        '^services/' | cut -d/ -f2 | sort -u | jq -R -s -c
        'split("\n") | map(select(length > 0))')
          echo "matrix={\"service\":${CHANGED}}" >> $GITHUB_OUTPUT

  # ──────────────────────────────────────────────────────────────
  # 2. Go Linting
  # ──────────────────────────────────────────────────────────────
  lint-go:
    name: Lint Go (${{ matrix.service }})
    runs-on: ubuntu-24.04
    needs: detect-changes
    if: needs.detect-changes.outputs.services == 'true'
    strategy:
      fail-fast: false
      matrix:
        service:
          - gateway
          - auth-service
          - vault-service
          - ssh-proxy
          - session-recorder
          - credential-manager
          - user-service
          - organization-service
          - rbac-service
          - audit-service
          - notification-service
          - scheduler-service
```

```
          - inventory-service
          - key-rotation-service
          - compliance-service
          - reporting-service
          - billing-service
          - search-service
          - webhook-service
          - approval-workflow-service
          - certificate-service
          - tunnel-service
          - metrics-aggregator
          - policy-engine
          - file-transfer-service
    steps:
      - uses: actions/checkout@v4
        with:
          submodules: recursive

      - uses: actions/setup-go@v5
        with:
          go-version: ${{ env.GO_VERSION }}
          cache-dependency-path: services/${{ matrix.service }}/
        go.sum

      - name: golangci-lint
        uses: golangci/golangci-lint-action@v6
        with:
          version: ${{ env.GOLANGCI_LINT_VERSION }}
          working-directory: services/${{ matrix.service }}
          args: --timeout=10m --config=../../.golangci.yml

  # ──────────────────────────────────────────────────────────────────────
  # 3. Flutter Lint
  # ──────────────────────────────────────────────────────────────────────
  lint-flutter:
    name: Lint Flutter
    runs-on: ubuntu-24.04
    needs: detect-changes
    if: needs.detect-changes.outputs.flutter == 'true'
    steps:
      - uses: actions/checkout@v4

      - uses: subosito/flutter-action@v2
        with:
```

```yaml
        flutter-version: ${{ env.FLUTTER_VERSION }}
        channel: stable

    - name: Get Flutter dependencies
      working-directory: clients/flutter
      run: flutter pub get

    - name: dart analyze
      working-directory: clients/flutter
      run: dart analyze --fatal-infos

    - name: dart format check
      working-directory: clients/flutter
      run: dart format --output=none --set-exit-if-changed .

# ──────────────────────────────────────────────────────────────────────
# 4. Unit Tests with Coverage Gate
# ──────────────────────────────────────────────────────────────────────
test-unit:
  name: Unit Tests (${{ matrix.service }})
  runs-on: ubuntu-24.04
  needs: detect-changes
  strategy:
    fail-fast: false
    matrix:
      service:
        - gateway
        - auth-service
        - vault-service
        - ssh-proxy
        - session-recorder
        - credential-manager
        - user-service
        - organization-service
        - rbac-service
        - audit-service
        - notification-service
        - scheduler-service
        - inventory-service
        - key-rotation-service
        - compliance-service
        - reporting-service
        - billing-service
        - search-service
```

```
        - webhook-service
        - approval-workflow-service
        - certificate-service
        - tunnel-service
        - metrics-aggregator
        - policy-engine
        - file-transfer-service
    steps:
      - uses: actions/checkout@v4
        with:
          submodules: recursive

      - uses: actions/setup-go@v5
        with:
          go-version: ${{ env.GO_VERSION }}
          cache-dependency-path: services/${{ matrix.service }}/
        go.sum

      - name: Run unit tests
        working-directory: services/${{ matrix.service }}
        run: |
          go test -v -race -count=1 \
            -coverprofile=coverage.out \
            -covermode=atomic \
            -tags=unit \
            ./...

      - name: Check coverage threshold
        working-directory: services/${{ matrix.service }}
        run: |
          COVERAGE=$(go tool cover -func=coverage.out | grep total
        | awk '{print $3}' | sed 's/%//')
          echo "Coverage: ${COVERAGE}%"
          if (( $(echo "$COVERAGE < ${{ env.COVERAGE_THRESHOLD }}"
        | bc -l) )); then
            echo "Coverage ${COVERAGE}% is below threshold $
        {{ env.COVERAGE_THRESHOLD }}%"
            exit 1
          fi

      - name: Upload coverage
        uses: codecov/codecov-action@v4
        with:
          file: services/${{ matrix.service }}/coverage.out
          flags: ${{ matrix.service }}
```

```yaml
# ──────────────────────────────────────────────────────────────
# 5. Integration Tests (testcontainers)
# ──────────────────────────────────────────────────────────────
test-integration:
  name: Integration Tests (${{ matrix.service }})
  runs-on: ubuntu-24.04
  needs: test-unit
  strategy:
    fail-fast: false
    matrix:
      service: [auth-service, vault-service, ssh-proxy, rbac-
      service, audit-service]
  steps:
    - uses: actions/checkout@v4
      with:
        submodules: recursive

    - uses: actions/setup-go@v5
      with:
        go-version: ${{ env.GO_VERSION }}
        cache-dependency-path: services/${{ matrix.service }}/
      go.sum

    - name: Run integration tests
      working-directory: services/${{ matrix.service }}
      env:
        TESTCONTAINERS_RYUK_DISABLED: "true"
      run: |
        go test -v -race -count=1 \
          -timeout=10m \
          -tags=integration \
          ./test/integration/...

# ──────────────────────────────────────────────────────────────
# 6. Security Scanning
# ──────────────────────────────────────────────────────────────
security-sast:
  name: SAST Security Scan
  runs-on: ubuntu-24.04
  needs: detect-changes
  if: needs.detect-changes.outputs.services == 'true'
  steps:
    - uses: actions/checkout@v4
      with:
        submodules: recursive
```

```yaml
      - uses: actions/setup-go@v5
        with:
          go-version: ${{ env.GO_VERSION }}

      - name: Install gosec
        run: go install github.com/securego/gosec/v2/cmd/
        gosec@latest

      - name: Run gosec
        run: |
          gosec -fmt=sarif -out=gosec-results.sarif ./...
        continue-on-error: true

      - name: Upload gosec SARIF
        uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: gosec-results.sarif

      - name: Run govulncheck
        run: |
          go install golang.org/x/vuln/cmd/govulncheck@latest
          govulncheck ./...

      - name: Run semgrep
        uses: returntocorp/semgrep-action@v1
        with:
          config: >-
            p/golang
            p/secrets
            p/security-audit

  # ──────────────────────────────────────────────────────────────
  # 7. Docker Build Verification
  # ──────────────────────────────────────────────────────────────
  docker-build-verify:
    name: Docker Build Verify (${{ matrix.service }})
    runs-on: ubuntu-24.04
    needs: [lint-go, test-unit]
    strategy:
      fail-fast: false
      matrix:
        service:
          - gateway
```

```yaml
          - auth-service
          - vault-service
          - ssh-proxy
          - session-recorder
    steps:
      - uses: actions/checkout@v4
        with:
          submodules: recursive

      - uses: docker/setup-buildx-action@v3

      - name: Build Docker image (no push)
        uses: docker/build-push-action@v5
        with:
          context: services/${{ matrix.service }}
          push: false
          tags: ghcr.io/helixdevelopment/${{ matrix.service }}:pr-
        ${{ github.event.number }}
          build-args: |
            GIT_COMMIT=${{ github.sha }}
            GIT_TAG=pr-${{ github.event.number }}
            BUILD_TIME=${{ github.event.head_commit.timestamp }}
          cache-from: type=gha,scope=${{ matrix.service }}
          cache-to: type=gha,scope=${{ matrix.service }},mode=max

  # ──────────────────────────────────────────────────────────────────────
  # 8. Contract Tests (Pact)
  # ──────────────────────────────────────────────────────────────────────
  contract-tests:
    name: Contract Tests
    runs-on: ubuntu-24.04
    needs: test-unit
    steps:
      - uses: actions/checkout@v4
        with:
          submodules: recursive

      - uses: actions/setup-go@v5
        with:
          go-version: ${{ env.GO_VERSION }}

      - name: Run Pact consumer tests
        run: |
          go test -v -tags=contract ./...
```

```yaml
      working-directory: services/gateway

    - name: Publish pacts to broker
      if: github.event_name == 'pull_request'
      run: |
        docker run --rm \
          -v $(pwd)/pacts:/pacts \
          pactfoundation/pact-cli publish \
          --pact-dir /pacts \
          --broker-base-url ${{ secrets.PACT_BROKER_URL }} \
          --broker-token ${{ secrets.PACT_BROKER_TOKEN }} \
          --consumer-app-version ${{ github.sha }} \
          --branch ${{ github.head_ref }}

# ──────────────────────────────────────────────────────────────
# 9. Constitution Compliance Check
# ──────────────────────────────────────────────────────────────
constitution-compliance:
  name: HelixConstitution Compliance
  runs-on: ubuntu-24.04
  needs: detect-changes
  steps:
    - uses: actions/checkout@v4
      with:
        submodules: recursive

    - name: Run constitution compliance
      run: |
        cd constitution
        go run ./cmd/check \
          --target ../services \
          --policies ./policies \
          --schemas ./schemas \
          --output sarif \
          --output-file ../constitution-results.sarif

    - name: Upload compliance results
      uses: github/codeql-action/upload-sarif@v3
      if: always()
      with:
        sarif_file: constitution-results.sarif

# ──────────────────────────────────────────────────────────────
# 10. Final PR Gate
```

```yaml
  # ──────────────────────────────────────────────────────────────
  pr-gate:
    name: PR Gate (All Checks Pass)
    runs-on: ubuntu-24.04
    needs:
      - lint-go
      - lint-flutter
      - test-unit
      - test-integration
      - security-sast
      - docker-build-verify
      - contract-tests
      - constitution-compliance
    if: always()
    steps:
      - name: Check all jobs
        run: |
          if [[ "${{ contains(needs.*.result, 'failure') }}" ==
        "true" ]]; then
            echo "One or more required checks failed."
            exit 1
          fi
          echo "All checks passed."
```

5.2 Main Branch Pipeline

```yaml
# .github/workflows/main.yml
name: Main Branch — Build, Push, Deploy to Staging

on:
  push:
    branches: [main]

env:
  GO_VERSION: "1.25"
  REGISTRY: ghcr.io
  IMAGE_ORG: helixterm
  HELM_VERSION: "3.15.2"
  KUBECTL_VERSION: "1.31.2"

jobs:
  # ─── Build and Push All Service Images ──────────────────────
  build-push:
    name: Build & Push (${{ matrix.service }})
```

```yaml
    runs-on: ubuntu-24.04
    permissions:
      contents: read
      packages: write
      security-events: write
    strategy:
      fail-fast: false
      matrix:
        service:
          - gateway
          - auth-service
          - vault-service
          - ssh-proxy
          - session-recorder
          - credential-manager
          - user-service
          - organization-service
          - rbac-service
          - audit-service
          - notification-service
          - scheduler-service
          - inventory-service
          - key-rotation-service
          - compliance-service
          - reporting-service
          - billing-service
          - search-service
          - webhook-service
          - approval-workflow-service
          - certificate-service
          - tunnel-service
          - metrics-aggregator
          - policy-engine
          - file-transfer-service
          - flutter-web
    steps:
      - uses: actions/checkout@v4
        with:
          submodules: recursive

      - uses: docker/setup-buildx-action@v3

      - name: Login to GitHub Container Registry
        uses: docker/login-action@v3
```

```yaml
    with:
      registry: ${{ env.REGISTRY }}
      username: ${{ github.actor }}
      password: ${{ secrets.GITHUB_TOKEN }}

  - name: Login to Harbor
    uses: docker/login-action@v3
    with:
      registry: ${{ secrets.HARBOR_REGISTRY }}
      username: ${{ secrets.HARBOR_USER }}
      password: ${{ secrets.HARBOR_PASSWORD }}

  - name: Extract metadata
    id: meta
    uses: docker/metadata-action@v5
    with:
      images: |
        ${{ env.REGISTRY }}/${{ env.IMAGE_ORG }}/$
    {{ matrix.service }}
        ${{ secrets.HARBOR_REGISTRY }}/${{ env.IMAGE_ORG }}/$
    {{ matrix.service }}
      tags: |
        type=sha,prefix=sha-
        type=raw,value=latest
        type=raw,value=main-${{ github.run_number }}

  - name: Set service context
    id: ctx
    run: |
      if [[ "${{ matrix.service }}" == "flutter-web" ]]; then
        echo "context=clients/flutter" >> $GITHUB_OUTPUT
      else
        echo "context=services/${{ matrix.service }}" >>
    $GITHUB_OUTPUT
      fi

  - name: Build and push
    uses: docker/build-push-action@v5
    with:
      context: ${{ steps.ctx.outputs.context }}
      push: true
      tags: ${{ steps.meta.outputs.tags }}
      labels: ${{ steps.meta.outputs.labels }}
      build-args: |
        GIT_COMMIT=${{ github.sha }}
```

```
            GIT_TAG=main-${{ github.run_number }}
            BUILD_TIME=${{ github.event.head_commit.timestamp }}
          cache-from: type=gha,scope=${{ matrix.service }}-main
          cache-to: type=gha,scope=${{ matrix.service }}-
        main,mode=max
          provenance: true
          sbom: true

      - name: Generate SBOM
        uses: anchore/sbom-action@v0
        with:
          image: ${{ env.REGISTRY }}/${{ env.IMAGE_ORG }}/$
        {{ matrix.service }}:sha-${{ github.sha }}
          format: spdx-json
          output-file: sbom-${{ matrix.service }}.spdx.json

      - name: Trivy vulnerability scan
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: ${{ env.REGISTRY }}/${{ env.IMAGE_ORG }}/$
        {{ matrix.service }}:sha-${{ github.sha }}
          format: sarif
          output: trivy-${{ matrix.service }}.sarif
          severity: CRITICAL,HIGH
          exit-code: '1'
          ignore-unfixed: true

      - name: Upload Trivy SARIF
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: trivy-${{ matrix.service }}.sarif

  # ── Package Helm Chart ──────────────────────────────────────────

  helm-package:
    name: Package Helm Chart
    runs-on: ubuntu-24.04
    needs: build-push
    steps:
      - uses: actions/checkout@v4
        with:
          submodules: recursive

      - name: Install Helm
        uses: azure/setup-helm@v4
```

```yaml
        with:
          version: ${{ env.HELM_VERSION }}

      - name: Update chart dependencies
        run: |
          helm dependency update infrastructure/helm/helixterm

      - name: Package chart
        run: |
          helm package infrastructure/helm/helixterm \
            --version 1.0.${{ github.run_number }} \
            --app-version sha-${{ github.sha }} \
            --destination ./helm-packages

      - name: Push chart to Harbor OCI
        run: |
          helm push ./helm-packages/helixterm-1.0.$
        {{ github.run_number }}.tgz \
            oci://${{ secrets.HARBOR_REGISTRY }}/helixterm

      - name: Upload chart artifact
        uses: actions/upload-artifact@v4
        with:
          name: helm-chart
          path: ./helm-packages/

  # ─── Deploy to Staging ──────────────────────────────────────────

  deploy-staging:
    name: Deploy to Staging
    runs-on: ubuntu-24.04
    needs: helm-package
    environment: staging
    concurrency:
      group: deploy-staging
      cancel-in-progress: false
    steps:
      - uses: actions/checkout@v4

      - name: Install kubectl
        uses: azure/setup-kubectl@v4
        with:
          version: v${{ env.KUBECTL_VERSION }}

      - name: Install Helm
```

```
      uses: azure/setup-helm@v4
      with:
        version: ${{ env.HELM_VERSION }}

    - name: Configure kubeconfig
      run: |
        echo "${{ secrets.STAGING_KUBECONFIG }}" | base64 -d >
      kubeconfig.yaml
        echo "KUBECONFIG=$(pwd)/kubeconfig.yaml" >> $GITHUB_ENV

    - name: Download helm chart
      uses: actions/download-artifact@v4
      with:
        name: helm-chart
        path: ./helm-packages/

    - name: Deploy to staging
      run: |
        helm upgrade --install helixterm-staging ./helm-
      packages/helixterm-1.0.${{ github.run_number }}.tgz \
          --namespace helixterm-staging \
          --create-namespace \
          --values infrastructure/helm/helixterm/values-
      staging.yaml \
          --set global.imageTag=sha-${{ github.sha }} \
          --timeout 10m \
          --wait \
          --atomic

    - name: Verify deployment
      run: |
        kubectl rollout status deployment --namespace helixterm-
      staging --timeout=5m

# ─── E2E Tests on Staging ──────────────────────────────────────

e2e-staging:
  name: E2E Tests on Staging
  runs-on: ubuntu-24.04
  needs: deploy-staging
  steps:
    - uses: actions/checkout@v4
      with:
        submodules: recursive

    - uses: actions/setup-go@v5
```

```
        with:
          go-version: ${{ env.GO_VERSION }}

      - name: Run E2E tests
        env:
          E2E_BASE_URL: https://api.staging.helixterm.io
          E2E_SSH_HOST: ssh.staging.helixterm.io
          E2E_TEST_USER: ${{ secrets.E2E_TEST_USER }}
          E2E_TEST_PASSWORD: ${{ secrets.E2E_TEST_PASSWORD }}
        run: |
          go test -v -timeout=30m -tags=e2e ./test/e2e/...

  # ─── Performance Tests on Staging ──────────────────────────────
  perf-staging:
    name: Performance Tests on Staging
    runs-on: ubuntu-24.04
    needs: deploy-staging
    steps:
      - uses: actions/checkout@v4

      - name: Run k6 performance tests
        uses: grafana/k6-action@v0.3.1
        with:
          filename: test/performance/k6/load-test.js
          flags: --env BASE_URL=https://api.staging.helixterm.io
        --vus=100 --duration=5m

      - name: Upload k6 results
        uses: actions/upload-artifact@v4
        with:
          name: k6-results
          path: results.json
```

5.3 Release Pipeline

```
# .github/workflows/release.yml
name: Release — Production Canary Deployment

on:
  push:
    tags:
      - 'v[0-9]+.[0-9]+.[0-9]+'

env:
```

```yaml
  HELM_VERSION: "3.15.2"
  KUBECTL_VERSION: "1.31.2"

jobs:
  # ── Validate Release Tag ──────────────────────────────────────
  validate:
    name: Validate Release
    runs-on: ubuntu-24.04
    outputs:
      version: ${{ steps.version.outputs.version }}
      sha: ${{ steps.sha.outputs.sha }}
    steps:
      - uses: actions/checkout@v4

      - name: Extract version
        id: version
        run: echo "version=${GITHUB_REF#refs/tags/}" >>
        $GITHUB_OUTPUT

      - name: Get commit SHA for tag
        id: sha
        run: echo "sha=$(git rev-list -n 1 $
        {{ github.ref_name }})" >> $GITHUB_OUTPUT

      - name: Verify tag matches semantic version
        run: |
          VERSION=${{ steps.version.outputs.version }}
          if ! [[ $VERSION =~ ^v[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
            echo "Invalid version format: $VERSION"
            exit 1
          fi

  # ── Canary Deployment (5%) ────────────────────────────────────
  deploy-canary-5:
    name: Deploy Canary 5%
    runs-on: ubuntu-24.04
    needs: validate
    environment: production-canary
    concurrency:
      group: deploy-production
      cancel-in-progress: false
    steps:
      - uses: actions/checkout@v4
```

```yaml
    - name: Install tools
      uses: azure/setup-helm@v4
      with:
        version: ${{ env.HELM_VERSION }}

    - name: Configure kubeconfig
      run: |
        echo "${{ secrets.PROD_KUBECONFIG }}" | base64 -d >
      kubeconfig.yaml
        echo "KUBECONFIG=$(pwd)/kubeconfig.yaml" >> $GITHUB_ENV

    - name: Deploy canary (5% traffic)
      run: |
        # Deploy canary with 5% traffic weight using Argo
      Rollouts or canary ingress
        helm upgrade --install helixterm-prod-canary \
          oci://${{ secrets.HARBOR_REGISTRY }}/helixterm/
      helixterm \
          --version 1.0.0 \
          --namespace helixterm-prod \
          --values infrastructure/helm/helixterm/values-
      production.yaml \
          --set global.imageTag=sha-$
      {{ needs.validate.outputs.sha }} \
          --set canary.enabled=true \
          --set canary.weight=5 \
          --timeout 10m \
          --wait \
          --atomic

    - name: Smoke tests (canary)
      run: |
        go test -v -timeout=5m -tags=smoke \
          -run TestSmoke \
          ./test/smoke/...
      env:
        SMOKE_BASE_URL: https://api.helixterm.io
        SMOKE_CANARY: "true"

# ── Canary Monitoring (5%) ──────────────────────────────────

monitor-canary-5:
  name: Monitor Canary 5% (10 min)
  runs-on: ubuntu-24.04
  needs: deploy-canary-5
  steps:
```

```yaml
    - uses: actions/checkout@v4

    - name: Monitor error rate for 10 minutes
      run: |
        python3 scripts/monitor-canary.py \
          --prometheus-url ${{ secrets.PROMETHEUS_URL }} \
          --service-version sha-$
      {{ needs.validate.outputs.sha }} \
          --duration-minutes 10 \
          --error-rate-threshold 0.01 \
          --latency-p99-threshold-ms 500

# ── Promote to 25% ──────────────────────────────────────────

deploy-canary-25:
  name: Promote Canary to 25%
  runs-on: ubuntu-24.04
  needs: monitor-canary-5
  environment: production-25pct
  steps:
    - uses: actions/checkout@v4
    - uses: azure/setup-helm@v4
      with:
        version: ${{ env.HELM_VERSION }}
    - name: Configure kubeconfig
      run: |
        echo "${{ secrets.PROD_KUBECONFIG }}" | base64 -d >
      kubeconfig.yaml
        echo "KUBECONFIG=$(pwd)/kubeconfig.yaml" >> $GITHUB_ENV
    - name: Promote to 25%
      run: |
        helm upgrade helixterm-prod-canary \
          oci://${{ secrets.HARBOR_REGISTRY }}/helixterm/
      helixterm \
          --namespace helixterm-prod \
          --reuse-values \
          --set canary.weight=25 \
          --wait

monitor-canary-25:
  name: Monitor Canary 25% (10 min)
  runs-on: ubuntu-24.04
  needs: deploy-canary-25
  steps:
    - uses: actions/checkout@v4
    - name: Monitor error rate
```

```
      run: |
        python3 scripts/monitor-canary.py \
          --prometheus-url ${{ secrets.PROMETHEUS_URL }} \
          --service-version sha-$
      {{ needs.validate.outputs.sha }} \
          --duration-minutes 10 \
          --error-rate-threshold 0.01

# ─── Promote to 50% ──────────────────────────────────────────

deploy-canary-50:
  name: Promote Canary to 50%
  runs-on: ubuntu-24.04
  needs: monitor-canary-25
  environment: production-50pct
  steps:
    - uses: actions/checkout@v4
    - uses: azure/setup-helm@v4
      with:
        version: ${{ env.HELM_VERSION }}
    - name: Configure kubeconfig
      run: |
        echo "${{ secrets.PROD_KUBECONFIG }}" | base64 -d >
      kubeconfig.yaml
        echo "KUBECONFIG=$(pwd)/kubeconfig.yaml" >> $GITHUB_ENV
    - name: Promote to 50%
      run: |
        helm upgrade helixterm-prod-canary \
          oci://${{ secrets.HARBOR_REGISTRY }}/helixterm/
      helixterm \
          --namespace helixterm-prod \
          --reuse-values \
          --set canary.weight=50 \
          --wait

monitor-canary-50:
  name: Monitor Canary 50% (15 min)
  runs-on: ubuntu-24.04
  needs: deploy-canary-50
  steps:
    - uses: actions/checkout@v4
    - name: Monitor error rate
      run: |
        python3 scripts/monitor-canary.py \
          --prometheus-url ${{ secrets.PROMETHEUS_URL }} \
```

```
          --service-version sha-$
        {{ needs.validate.outputs.sha }} \
          --duration-minutes 15 \
          --error-rate-threshold 0.005

  # ─── Full Production Rollout ──────────────────────────────────

  deploy-production-100:
    name: Full Production Rollout (100%)
    runs-on: ubuntu-24.04
    needs: monitor-canary-50
    environment: production
    steps:
      - uses: actions/checkout@v4

      - uses: azure/setup-helm@v4
        with:
          version: ${{ env.HELM_VERSION }}

      - name: Configure kubeconfig
        run: |
          echo "${{ secrets.PROD_KUBECONFIG }}" | base64 -d >
        kubeconfig.yaml
          echo "KUBECONFIG=$(pwd)/kubeconfig.yaml" >> $GITHUB_ENV

      - name: Full production deploy
        run: |
          helm upgrade --install helixterm-prod \
            oci://${{ secrets.HARBOR_REGISTRY }}/helixterm/
        helixterm \
          --namespace helixterm-prod \
          --values infrastructure/helm/helixterm/values-
        production.yaml \
          --set global.imageTag=sha-$
        {{ needs.validate.outputs.sha }} \
          --set global.version=$
        {{ needs.validate.outputs.version }} \
          --timeout 20m \
          --wait \
          --atomic

      - name: Remove canary
        run: |
          helm uninstall helixterm-prod-canary --namespace
        helixterm-prod || true

      - name: Final smoke tests
```

```yaml
      run: |
        go test -v -timeout=5m -tags=smoke ./test/smoke/...
      env:
        SMOKE_BASE_URL: https://api.helixterm.io

# ─── Automatic Rollback on Failure ──────────────────────────

rollback-on-failure:
  name: Automatic Rollback
  runs-on: ubuntu-24.04
  needs:
    - deploy-canary-5
    - monitor-canary-5
    - deploy-canary-25
    - monitor-canary-25
    - deploy-canary-50
    - monitor-canary-50
    - deploy-production-100
  if: failure()
  steps:
    - name: Configure kubeconfig
      run: |
        echo "${{ secrets.PROD_KUBECONFIG }}" | base64 -d > kubeconfig.yaml
        echo "KUBECONFIG=$(pwd)/kubeconfig.yaml" >> $GITHUB_ENV

    - name: Rollback production
      run: |
        helm rollback helixterm-prod --namespace helixterm-prod --wait
        helm uninstall helixterm-prod-canary --namespace helixterm-prod || true

    - name: Notify on-call
      uses: 8398a7/action-slack@v3
      with:
        status: failure
        text: |
          :rotating_light: PRODUCTION ROLLBACK TRIGGERED
          Release: ${{ needs.validate.outputs.version }}
          Reason: Deployment pipeline failure
          Runbook: https://docs.helixterm.io/runbooks/rollback
      env:
        SLACK_WEBHOOK_URL: ${{ secrets.SLACK_ONCALL_WEBHOOK }}
```

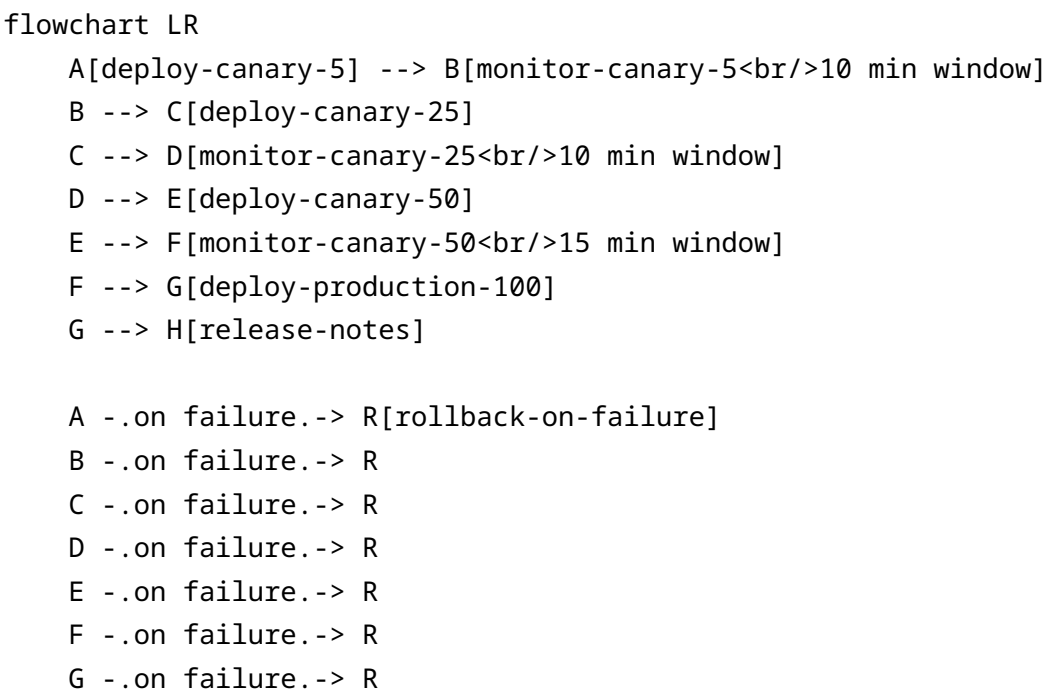
```yaml
  # ─── Release Notes ──────────────────────────────────────────

  release-notes:
    name: Generate Release Notes
    runs-on: ubuntu-24.04
    needs: deploy-production-100
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Generate release notes
        uses: release-drafter/release-drafter@v6
        with:
          version: ${{ needs.validate.outputs.version }}
          publish: true
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

5.4 Canary Promotion Pipeline — Job Graph

The `release.yml` workflow above is a strict linear chain gated by monitoring windows, not a fan-out — every promotion stage `needs:` the previous stage's monitor job, and `rollback-on-failure` (§5.3) fans in from every stage so a budget breach at *any* percentage triggers the same rollback path:



```
    R --> R1[helm rollback helixterm-prod]
    R1 --> R2[helm uninstall helixterm-prod-canary]
    R2 --> R3[Slack page on-call]

    style R fill:#7a1f1f,color:#fff
    style R1 fill:#7a1f1f,color:#fff
    style R2 fill:#7a1f1f,color:#fff
    style R3 fill:#7a1f1f,color:#fff
```

See §8.9 for the sequence-level view of the same promotion (participants and message order rather than job dependencies), and §8.8 for the companion DR cross-region failover sequence diagram.

---

6. Terraform Infrastructure-as-Code

6.1 Directory Structure

```
infrastructure/terraform/
├── modules/
│   ├── eks/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── rds/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── elasticache/
│   ├── msk/
│   ├── amazon_mq/
│   ├── harbor/
│   ├── s3/
│   ├── cloudfront/
│   └── networking/
├── environments/
│   ├── production/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── outputs.tf
│   │   └── terraform.tfvars
│   └── staging/
```

```
│       ├── main.tf
│       ├── variables.tf
│       └── terraform.tfvars
└── shared/
    ├── route53.tf
    ├── acm.tf
    └── iam.tf
```

6.2 Production Main

```
# infrastructure/terraform/environments/production/main.tf

terraform {
  required_version = ">= 1.8.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.50"
    }
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 2.30"
    }
    helm = {
      source  = "hashicorp/helm"
      version = "~> 2.13"
    }
  }

  backend "s3" {
    bucket         = "helixterm-terraform-state-prod"
    key            = "production/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    kms_key_id     = "arn:aws:kms:us-east-1:123456789012:key/
xxxxxxxx"
    dynamodb_table = "helixterm-terraform-locks"
  }
}

provider "aws" {
  region = var.aws_region
```

```
  default_tags {
    tags = {
      Environment = "production"
      Project     = "helixterm"
      ManagedBy   = "terraform"
      Team        = "platform"
    }
  }
}

# ─── VPC Networking ──────────────────────────────────────────────

module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.8"

  name = "helixterm-prod"
  cidr = "10.0.0.0/16"

  azs             = ["us-east-1a", "us-east-1b", "us-east-1c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24",
"10.0.103.0/24"]
  intra_subnets   = ["10.0.201.0/24", "10.0.202.0/24",
"10.0.203.0/24"]

  enable_nat_gateway     = true
  single_nat_gateway     = false
  one_nat_gateway_per_az = true

  enable_vpn_gateway = false
  enable_dns_hostnames = true
  enable_dns_support   = true

  # VPC Flow Logs to S3
  enable_flow_log                      = true
  create_flow_log_cloudwatch_iam_role  = false
  create_flow_log_cloudwatch_log_group = false
  flow_log_destination_type           = "s3"
  flow_log_destination_arn            =
aws_s3_bucket.flow_logs.arn

  # Kubernetes tags required for EKS subnet discovery
  private_subnet_tags = {
```

```
    "kubernetes.io/cluster/helixterm-prod" = "shared"
    "kubernetes.io/role/internal-elb"      = "1"
  }

  public_subnet_tags = {
    "kubernetes.io/cluster/helixterm-prod" = "shared"
    "kubernetes.io/role/elb"               = "1"
  }
}

# ─── EKS Cluster ──────────────────────────────────────────────────────

module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "~> 20.11"

  cluster_name    = "helixterm-prod"
  cluster_version = "1.31"

  vpc_id                   = module.vpc.vpc_id
  subnet_ids               = module.vpc.private_subnets
  control_plane_subnet_ids = module.vpc.intra_subnets

  cluster_endpoint_public_access       = true
  cluster_endpoint_public_access_cidrs = ["0.0.0.0/0"]
  cluster_endpoint_private_access      = true

  # Enable cluster encryption
  cluster_encryption_config = {
    provider_key_arn = aws_kms_key.eks.arn
    resources        = ["secrets"]
  }

  # Enable cluster logging
  cluster_enabled_log_types = ["api", "audit", "authenticator",
"controllerManager", "scheduler"]

  # Managed node groups
  eks_managed_node_groups = {
    # System node group — for cluster add-ons and platform
services
    system = {
      name           = "system"
      instance_types = ["m6i.xlarge"]
```

```
  ami_type       = "AL2_x86_64"

  min_size     = 3
  max_size     = 6
  desired_size = 3

  labels = {
    role = "system"
  }

  taints = [
    {
      key    = "CriticalAddonsOnly"
      value  = "true"
      effect = "NO_SCHEDULE"
    }
  ]

  block_device_mappings = {
    xvda = {
      device_name = "/dev/xvda"
      ebs = {
        volume_size           = 100
        volume_type           = "gp3"
        iops                  = 3000
        throughput            = 125
        encrypted             = true
        kms_key_id            = aws_kms_key.eks_ebs.arn
        delete_on_termination = true
      }
    }
  }
}

# General node group — for application services
general = {
  name           = "general"
  instance_types = ["m6i.2xlarge", "m6a.2xlarge"]
  ami_type       = "AL2_x86_64"

  min_size     = 6
  max_size     = 30
  desired_size = 9
```

```
      labels = {
        role = "general"
      }

      block_device_mappings = {
        xvda = {
          device_name = "/dev/xvda"
          ebs = {
            volume_size           = 200
            volume_type           = "gp3"
            iops                  = 3000
            throughput            = 125
            encrypted             = true
            kms_key_id            = aws_kms_key.eks_ebs.arn
            delete_on_termination = true
          }
        }
      }
    }

    # GPU node group — for ML-based anomaly detection (session
analysis)
    gpu = {
      name           = "gpu"
      instance_types = ["g4dn.xlarge"]
      ami_type       = "AL2_x86_64_GPU"

      min_size     = 0
      max_size     = 5
      desired_size = 0

      labels = {
        role = "gpu"
        "nvidia.com/gpu" = "true"
      }

      taints = [
        {
          key    = "nvidia.com/gpu"
          value  = "true"
          effect = "NO_SCHEDULE"
        }
      ]
    }
```

```
  }

  # Cluster add-ons
  cluster_addons = {
    coredns = {
      most_recent = true
    }
    kube-proxy = {
      most_recent = true
    }
    vpc-cni = {
      most_recent              = true
      service_account_role_arn = module.vpc_cni_irsa.iam_role_arn
    }
    aws-ebs-csi-driver = {
      most_recent              = true
      service_account_role_arn = module.ebs_csi_irsa.iam_role_arn
    }
    aws-efs-csi-driver = {
      most_recent = true
    }
    aws-guardduty-agent = {
      most_recent = true
    }
  }

  # Node security group rules
  node_security_group_additional_rules = {
    ingress_self_all = {
      description = "Node to node all ports/protocols"
      protocol    = "-1"
      from_port   = 0
      to_port     = 0
      type        = "ingress"
      self        = true
    }
  }

  # Enable IRSA
  enable_irsa = true
}

# ─── KMS Keys
─────────────────────────────────────────────────────────────
```

```
resource "aws_kms_key" "eks" {
  description             = "EKS Secrets Encryption"
  deletion_window_in_days = 7
  enable_key_rotation     = true
}

resource "aws_kms_key" "eks_ebs" {
  description             = "EKS EBS Volumes Encryption"
  deletion_window_in_days = 7
  enable_key_rotation     = true
}

resource "aws_kms_key" "rds" {
  description             = "RDS Encryption"
  deletion_window_in_days = 7
  enable_key_rotation     = true
}

resource "aws_kms_key" "s3" {
  description             = "S3 Buckets Encryption"
  deletion_window_in_days = 7
  enable_key_rotation     = true
}

# ─── RDS PostgreSQL Multi-AZ
──────────────────────────────────────────────────

resource "aws_db_subnet_group" "helixterm" {
  name       = "helixterm-prod"
  subnet_ids = module.vpc.private_subnets
}

resource "aws_security_group" "rds" {
  name        = "helixterm-rds-prod"
  description = "Security group for RDS PostgreSQL"
  vpc_id      = module.vpc.vpc_id

  ingress {
    from_port   = 5432
    to_port     = 5432
    protocol    = "tcp"
    cidr_blocks = module.vpc.private_subnets_cidr_blocks
  }

  egress {
```

```
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_db_instance" "helixterm_prod" {
  identifier = "helixterm-prod"

  engine               = "postgres"
  engine_version       = "17.2"
  instance_class       = "db.r6g.2xlarge"
  allocated_storage    = 500
  max_allocated_storage = 5000
  storage_type         = "gp3"
  storage_encrypted    = true
  kms_key_id           = aws_kms_key.rds.arn

  db_name  = "helixterm"
  username = "helixterm_admin"
  password = var.rds_password

  multi_az               = true
  db_subnet_group_name   = aws_db_subnet_group.helixterm.name
  vpc_security_group_ids = [aws_security_group.rds.id]

  backup_retention_period = 35
  backup_window           = "02:00-03:00"
  maintenance_window      = "sun:04:00-sun:05:00"

  # Enable enhanced monitoring
  monitoring_interval = 30
  monitoring_role_arn = aws_iam_role.rds_enhanced_monitoring.arn

  # Enable Performance Insights
  performance_insights_enabled          = true
  performance_insights_kms_key_id       = aws_kms_key.rds.arn
  performance_insights_retention_period = 731  # 2 years

  # Enable automated minor version upgrades
  auto_minor_version_upgrade = true

  # Deletion protection
```

```
  deletion_protection = true
  skip_final_snapshot = false
  final_snapshot_identifier = "helixterm-prod-final-snapshot"

  # CloudWatch log exports
  enabled_cloudwatch_logs_exports = ["postgresql", "upgrade"]

  parameter_group_name = aws_db_parameter_group.helixterm.name

  tags = {
    Name = "helixterm-prod"
  }
}

resource "aws_db_parameter_group" "helixterm" {
  name   = "helixterm-prod-pg17"
  family = "postgres17"

  parameter {
    name  = "shared_preload_libraries"
    value = "pg_stat_statements,pgaudit,pg_cron"
  }

  parameter {
    name  = "log_min_duration_statement"
    value = "1000"  # Log queries taking >1 second
  }

  parameter {
    name  = "pgaudit.log"
    value = "ddl,role"
  }

  parameter {
    name  = "max_connections"
    value = "1000"
  }

  parameter {
    name  = "work_mem"
    value = "16384"  # 16MB
  }

  parameter {
```

```
    name  = "maintenance_work_mem"
    value = "524288"  # 512MB
  }

  parameter {
    name  = "effective_cache_size"
    value = "24576000"  # 24GB (for r6g.2xlarge with 64GB RAM)
  }
}

# Read replica for reporting
resource "aws_db_instance" "helixterm_replica" {
  identifier = "helixterm-prod-replica"

  replicate_source_db = aws_db_instance.helixterm_prod.identifier

  instance_class    = "db.r6g.xlarge"
  storage_encrypted = true
  kms_key_id        = aws_kms_key.rds.arn

  multi_az               = false
  vpc_security_group_ids = [aws_security_group.rds.id]

  performance_insights_enabled = true

  tags = {
    Name = "helixterm-prod-replica"
    Role = "read-replica"
  }
}

# ─── ElastiCache Redis ──────────────────────────────────────────

resource "aws_elasticache_subnet_group" "helixterm" {
  name       = "helixterm-prod"
  subnet_ids = module.vpc.private_subnets
}

resource "aws_security_group" "elasticache" {
  name        = "helixterm-elasticache-prod"
  description = "Security group for ElastiCache Redis"
  vpc_id      = module.vpc.vpc_id

  ingress {
```

```
    from_port   = 6379
    to_port     = 6379
    protocol    = "tcp"
    cidr_blocks = module.vpc.private_subnets_cidr_blocks
  }
}

resource "aws_elasticache_replication_group" "helixterm" {
  replication_group_id = "helixterm-prod"
  description          = "HelixTerminator production Redis
cluster"

  node_type            = "cache.r7g.xlarge"
  num_cache_clusters   = 3
  parameter_group_name =
aws_elasticache_parameter_group.helixterm.name
  port                 = 6379

  subnet_group_name    =
aws_elasticache_subnet_group.helixterm.name
  security_group_ids   = [aws_security_group.elasticache.id]

  at_rest_encryption_enabled = true
  transit_encryption_enabled = true
  kms_key_id                 = aws_kms_key.s3.arn

  automatic_failover_enabled  = true
  multi_az_enabled            = true

  # Snapshot configuration
  snapshot_retention_limit = 7
  snapshot_window          = "03:00-04:00"
  maintenance_window       = "sun:05:00-sun:06:00"

  auto_minor_version_upgrade = true

  log_delivery_configuration {
    destination      =
aws_cloudwatch_log_group.redis_slow_log.name
    destination_type = "cloudwatch-logs"
    log_format       = "json"
    log_type         = "slow-log"
  }
}
```

```
resource "aws_elasticache_parameter_group" "helixterm" {
  family = "redis8"
  name   = "helixterm-prod-redis8"

  parameter {
    name  = "maxmemory-policy"
    value = "allkeys-lru"
  }

  parameter {
    name  = "notify-keyspace-events"
    value = "Ex"  # Expired key events
  }
}

# ─── MSK (Managed Kafka) ──────────────────────────────────────

resource "aws_msk_cluster" "helixterm" {
  cluster_name           = "helixterm-prod"
  kafka_version          = "3.9.0"
  number_of_broker_nodes = 3

  broker_node_group_info {
    instance_type   = "kafka.m5.2xlarge"
    client_subnets  = module.vpc.private_subnets
    security_groups = [aws_security_group.msk.id]

    storage_info {
      ebs_storage_info {
        provisioned_throughput {
          enabled           = true
          volume_throughput = 250
        }
        volume_size = 1000  # 1TB per broker
      }
    }
  }

  encryption_info {
    encryption_in_transit {
      client_broker = "TLS"
      in_cluster    = true
    }
```

```
    encryption_at_rest_kms_key_arn = aws_kms_key.s3.arn
  }

  configuration_info {
    arn      = aws_msk_configuration.helixterm.arn
    revision = aws_msk_configuration.helixterm.latest_revision
  }

  open_monitoring {
    prometheus {
      jmx_exporter {
        enabled_in_broker = true
      }
      node_exporter {
        enabled_in_broker = true
      }
    }
  }

  logging_info {
    broker_logs {
      cloudwatch_logs {
        enabled   = true
        log_group = aws_cloudwatch_log_group.msk.name
      }
      s3 {
        enabled = true
        bucket  = aws_s3_bucket.logs.id
        prefix  = "msk/"
      }
    }
  }
}

resource "aws_msk_configuration" "helixterm" {
  kafka_versions = ["3.9.0"]
  name           = "helixterm-prod"

  server_properties = <<PROPERTIES
auto.create.topics.enable=false
default.replication.factor=3
min.insync.replicas=2
num.partitions=12
num.replica.fetchers=4
```

```
replica.lag.time.max.ms=30000
socket.send.buffer.bytes=102400
socket.receive.buffer.bytes=102400
socket.request.max.bytes=104857600
log.retention.hours=168
log.segment.bytes=1073741824
log.retention.check.interval.ms=300000
PROPERTIES
}

# ─── Amazon MQ (Managed RabbitMQ) ──────────────────────────────

# Decision (Remediation Register §4 "RabbitMQ production path", 2026-07):
# RabbitMQ is a real, in-use dependency (notification-service + webhook-service
# consume from it, §2.8; docker-compose ships it for local dev, §4.4/§9.4; the
# testing strategy doc's RabbitMQ contract/integration suite exercises it
# against a live broker — see doc 03 §RabbitMQ Messaging Tests). Rather than
# remove it and migrate two consumers onto Kafka (which would turn a
# point-to-point work-queue pattern into topic/partition bookkeeping those
# services do not need), the decision is to PROVISION it: Amazon MQ for
# RabbitMQ, a multi-AZ managed broker cluster, is the canonical production
# path. `helm upgrade --values values-production.yaml` already sets
# `rabbitmq.enabled: false` (§3.4) — this resource is what that disabled
# in-cluster chart is replaced by, mirroring the Kafka→MSK / PostgreSQL→RDS /
# Redis→ElastiCache pattern in the same file.
resource "aws_security_group" "amazon_mq" {
  name        = "helixterm-amazonmq-prod"
  description = "Security group for Amazon MQ (RabbitMQ) — AMQPS + management UI"
  vpc_id      = module.vpc.vpc_id

  ingress {
```

```
    from_port   = 5671
    to_port     = 5671
    protocol    = "tcp"
    cidr_blocks = module.vpc.private_subnets_cidr_blocks
    description = "AMQPS (TLS-only, plaintext 5672 is not opened
in prod)"
  }

  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = module.vpc.private_subnets_cidr_blocks
    description = "RabbitMQ management console (internal only,
behind VPN/bastion)"
  }
}

resource "aws_mq_broker" "helixterm" {
  broker_name = "helixterm-prod"

  engine_type   = "RabbitMQ"
  # Track the platform RabbitMQ pin in Appendix A; confirm against
the
  # Amazon MQ-supported engine-version list at apply time (AWS
publishes a
  # closed set of supported minor releases that lags the upstream
project —
  # do not assume the pin below is deployable without that check,
§11.4.6).
  engine_version = "3.13"
  host_instance_type = "mq.m5.xlarge"

  deployment_mode = "CLUSTER_MULTI_AZ"

  publicly_accessible = false
  subnet_ids          = [module.vpc.private_subnets[0],
module.vpc.private_subnets[1]]
  security_groups     = [aws_security_group.amazon_mq.id]

  encryption_options {
    kms_key_id        = aws_kms_key.s3.arn
    use_aws_owned_key = false
  }
```

```
  logs {
    general = true
    audit   = true
  }

  maintenance_window_start_time {
    day_of_week = "SUNDAY"
    time_of_day = "05:00"
    time_zone   = "UTC"
  }

  auto_minor_version_upgrade = true

  user {
    username = "helixterm"
    password = var.rabbitmq_password
  }

  tags = {
    Name    = "helixterm-prod"
    Service = "notification-service+webhook-service"
  }
}

# Cross-region DR mirror: a second, smaller Amazon MQ cluster in
eu-west-1.
# RabbitMQ has no built-in cross-region replication (unlike RDS/
MSK), so
# durability for the DR region is achieved by the Shovel plugin
republishing
# from the primary queues into the DR broker's mirror queues — a
live
# forwarding mirror, not a byte-identical replica. RPO for
RabbitMQ DR is
# therefore best-effort (seconds-to-low-minutes, shovel-lag
dependent) and is
# explicitly weaker than the PostgreSQL RPO in §8.3.4 — documented
here so it
# is never silently assumed to match (§11.4.6 no-guessing).
resource "aws_mq_broker" "helixterm_dr" {
  provider = aws.eu-west-1

  broker_name         = "helixterm-dr"
```

```
  engine_type         = "RabbitMQ"
  engine_version      = "3.13"
  host_instance_type  = "mq.m5.large"
  deployment_mode     = "SINGLE_INSTANCE"
  publicly_accessible = false
  subnet_ids          = [module.vpc_dr.private_subnets[0]]
  security_groups     = [aws_security_group.amazon_mq_dr.id]

  encryption_options {
    kms_key_id        = aws_kms_key.s3_dr.arn
    use_aws_owned_key = false
  }

  user {
    username = "helixterm"
    password = var.rabbitmq_password
  }

  tags = {
    Name = "helixterm-dr"
    Role = "dr-shovel-target"
  }
}

# ─── S3 Buckets ──────────────────────────────────────────────────────

resource "aws_s3_bucket" "session_recordings" {
  bucket = "helixterm-session-recordings-prod"
  force_destroy = false
}

resource "aws_s3_bucket_versioning" "session_recordings" {
  bucket = aws_s3_bucket.session_recordings.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration"
"session_recordings" {
  bucket = aws_s3_bucket.session_recordings.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
```

```
      kms_master_key_id = aws_kms_key.s3.arn
    }
    bucket_key_enabled = true
  }
}

resource "aws_s3_bucket_lifecycle_configuration"
"session_recordings" {
  bucket = aws_s3_bucket.session_recordings.id

  rule {
    id     = "transition-to-ia"
    status = "Enabled"

    transition {
      days          = 30
      storage_class = "STANDARD_IA"
    }

    transition {
      days          = 90
      storage_class = "GLACIER"
    }

    transition {
      days          = 365
      storage_class = "DEEP_ARCHIVE"
    }

    expiration {
      days = 2555  # 7 years for compliance
    }
  }
}

resource "aws_s3_bucket_public_access_block" "session_recordings"
{
  bucket = aws_s3_bucket.session_recordings.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

```
# Backup bucket
resource "aws_s3_bucket" "backups" {
  bucket = "helixterm-backups-prod"
}

resource "aws_s3_bucket_versioning" "backups" {
  bucket = aws_s3_bucket.backups.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_replication_configuration" "backups" {
  depends_on = [aws_s3_bucket_versioning.backups]

  role   = aws_iam_role.s3_replication.arn
  bucket = aws_s3_bucket.backups.id

  rule {
    id     = "replicate-to-dr-region"
    status = "Enabled"

    destination {
      bucket        = aws_s3_bucket.backups_dr.arn
      storage_class = "STANDARD_IA"
    }
  }
}

# DR bucket in secondary region (CD-6: eu-west-1 is the canonical
DR region —
# a stray us-west-2 provider here previously reversed/contradicted
the DR
# region used everywhere else in this document, e.g. §8)
resource "aws_s3_bucket" "backups_dr" {
  provider = aws.eu-west-1
  bucket   = "helixterm-backups-prod-dr-euw1"
}

# ─── CloudFront CDN
─────────────────────────────────────────────────────

resource "aws_cloudfront_distribution" "helixterm_app" {
  enabled             = true
  is_ipv6_enabled     = true
```

```
  default_root_object = "index.html"
  aliases             = ["app.helixterm.io"]
  price_class         = "PriceClass_100"  # North America + Europe

  origin {
    domain_name =
aws_s3_bucket.flutter_web.bucket_regional_domain_name
    origin_id   = "S3-flutter-web"

    s3_origin_config {
      origin_access_identity =
aws_cloudfront_origin_access_identity.flutter.cloudfront_access_ide
    }
  }

  default_cache_behavior {
    allowed_methods  = ["GET", "HEAD", "OPTIONS"]
    cached_methods   = ["GET", "HEAD"]
    target_origin_id = "S3-flutter-web"

    forwarded_values {
      query_string = false
      cookies {
        forward = "none"
      }
    }

    viewer_protocol_policy = "redirect-to-https"
    min_ttl                = 0
    default_ttl            = 3600
    max_ttl                = 86400

    function_association {
      event_type   = "viewer-response"
      function_arn = aws_cloudfront_function.security_headers.arn
    }
  }

  # Cache JS/CSS/WASM assets aggressively
  ordered_cache_behavior {
    path_pattern     = "/assets/*"
    allowed_methods  = ["GET", "HEAD"]
    cached_methods   = ["GET", "HEAD"]
    target_origin_id = "S3-flutter-web"
```

```
    forwarded_values {
      query_string = false
      cookies {
        forward = "none"
      }
    }

    viewer_protocol_policy = "redirect-to-https"
    min_ttl                = 31536000
    default_ttl            = 31536000
    max_ttl                = 31536000
  }

  restrictions {
    geo_restriction {
      restriction_type = "none"
    }
  }

  viewer_certificate {
    acm_certificate_arn      =
aws_acm_certificate.helixterm_app.arn
    ssl_support_method       = "sni-only"
    minimum_protocol_version = "TLSv1.2_2021"
  }

  custom_error_response {
    error_code         = 404
    response_code      = 200
    response_page_path = "/index.html"  # SPA routing
  }

  web_acl_id = aws_wafv2_web_acl.cloudfront.arn
}

# ─── Route53 ──────────────────────────────────────────────────────────

resource "aws_route53_zone" "helixterm" {
  name = "helixterm.io"
}

resource "aws_route53_record" "api" {
  zone_id = aws_route53_zone.helixterm.zone_id
```

```
  name    = "api.helixterm.io"
  type    = "A"

  alias {
    name                   = module.eks.cluster_endpoint
    zone_id                = data.aws_lb.eks_lb.zone_id
    evaluate_target_health = true
  }
}

resource "aws_route53_record" "app" {
  zone_id = aws_route53_zone.helixterm.zone_id
  name    = "app.helixterm.io"
  type    = "A"

  alias {
    name                   =
aws_cloudfront_distribution.helixterm_app.domain_name
    zone_id                =
aws_cloudfront_distribution.helixterm_app.hosted_zone_id
    evaluate_target_health = false
  }
}

resource "aws_route53_health_check" "api" {
  fqdn              = "api.helixterm.io"
  port              = 443
  type              = "HTTPS"
  resource_path     = "/healthz/live"
  failure_threshold = 3
  request_interval  = 30

  tags = {
    Name = "helixterm-api-health"
  }
}

# ─── ACM Certificates
──────────────────────────────────────────────────────

resource "aws_acm_certificate" "helixterm" {
  domain_name               = "helixterm.io"
  subject_alternative_names = ["*.helixterm.io"]
  validation_method         = "DNS"
```

```
  lifecycle {
    create_before_destroy = true
  }
}

resource "aws_acm_certificate_validation" "helixterm" {
  certificate_arn         = aws_acm_certificate.helixterm.arn
  validation_record_fqdns = [for record in
aws_route53_record.cert_validation : record.fqdn]
}

# CloudFront requires cert in us-east-1
resource "aws_acm_certificate" "helixterm_app" {
  provider = aws.us-east-1-cf

  domain_name               = "app.helixterm.io"
  subject_alternative_names = ["helixterm.io"]
  validation_method         = "DNS"

  lifecycle {
    create_before_destroy = true
  }
}

# ─── IAM Roles
─────────────────────────────────────────────────────────────

# Session Recorder — S3 access for recording uploads
resource "aws_iam_role" "session_recorder" {
  name = "helixterm-session-recorder-prod"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Principal = {
          Federated = module.eks.oidc_provider_arn
        }
        Action = "sts:AssumeRoleWithWebIdentity"
        Condition = {
          StringEquals = {
            "${module.eks.oidc_provider}:sub" =
"system:serviceaccount:helixterm-prod:session-recorder"
            "${module.eks.oidc_provider}:aud" =
```

```
"sts.amazonaws.com"
          }
        }
      }
    ]
  })
}

resource "aws_iam_role_policy" "session_recorder_s3" {
  name = "session-recorder-s3"
  role = aws_iam_role.session_recorder.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "s3:PutObject",
          "s3:GetObject",
          "s3:DeleteObject",
          "s3:ListBucket"
        ]
        Resource = [
          aws_s3_bucket.session_recordings.arn,
          "${aws_s3_bucket.session_recordings.arn}/*"
        ]
      },
      {
        Effect = "Allow"
        Action = [
          "kms:GenerateDataKey",
          "kms:Decrypt"
        ]
        Resource = aws_kms_key.s3.arn
      }
    ]
  })
}
```

6.3 Harbor Container Registry (Self-Hosted, Rootless)

`ghcr.io` (used by CI in §5) is the build-time push target; **Harbor** is the platform's own, self-hosted, rootless-Podman-compatible registry (§11.4.161 / §11.4.76 — the project's containers submodule is the sole orchestration layer for it) sitting in front of every production `pull`. It gives HelixTerminator three things GHCR alone does not: (1) a pull-through cache/replication mirror so no production node ever depends on an external registry's availability during an incident, (2) Trivy/Clair vulnerability scanning + a signed, project-controlled promotion gate between "built" and "production-approved" images, and (3) the OCI chart repository the release pipeline already pushes Helm charts to (`oci://${{ secrets.HARBOR_REGISTRY }}/helixterm`, §5.3) — which had no backing infrastructure until this section.

Harbor runs in-cluster on the `system` EKS node group (same tier as other platform add-ons), backed by S3 for blob storage (so registry storage scales independently of node-local/EBS capacity) and the shared production PostgreSQL/Redis tier for its own metadata:

```
# infrastructure/terraform/modules/harbor/main.tf
resource "aws_s3_bucket" "harbor_registry" {
  bucket        = "helixterm-harbor-registry-prod"
  force_destroy = false
}

resource "aws_s3_bucket_versioning" "harbor_registry" {
  bucket = aws_s3_bucket.harbor_registry.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration"
"harbor_registry" {
  bucket = aws_s3_bucket.harbor_registry.id
  rule {
    apply_server_side_encryption_by_default {
      kms_master_key_id = aws_kms_key.s3.arn
      sse_algorithm     = "aws:kms"
    }
  }
}

resource "aws_s3_bucket_lifecycle_configuration" "harbor_registry"
```

```
{
  bucket = aws_s3_bucket.harbor_registry.id
  rule {
    id     = "expire-untagged-manifests"
    status = "Enabled"
    filter {}
    expiration {
      days = 90  # matches Harbor's own tag-retention GC policy,
see values below
    }
  }
}

resource "aws_iam_role" "harbor_registry" {
  name = "helixterm-harbor-registry-prod"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = { Federated = module.eks.oidc_provider_arn }
      Action = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "${module.eks.oidc_provider}:sub" =
"system:serviceaccount:helixterm-platform:harbor-registry"
          "${module.eks.oidc_provider}:aud" = "sts.amazonaws.com"
        }
      }
    }]
  })
}

resource "aws_iam_role_policy" "harbor_registry_s3" {
  name = "harbor-registry-s3"
  role = aws_iam_role.harbor_registry.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect   = "Allow"
      Action   = ["s3:PutObject", "s3:GetObject",
"s3:DeleteObject", "s3:ListBucket"]
      Resource = [aws_s3_bucket.harbor_registry.arn, "$
```

```
{aws_s3_bucket.harbor_registry.arn}/*"]
    }]
  })
}
```

Helm values for the upstream `harbor/harbor` chart, deployed to a dedicated `helixterm-platform` namespace (not `helixterm-prod`, since Harbor is platform infrastructure, not a product microservice):

```yaml
# infrastructure/helm/harbor/values-production.yaml
expose:
  type: ingress
  tls:
    enabled: true
    certSource: secret
    secret:
      secretName: harbor-tls
  ingress:
    hosts:
      core: harbor.helixterminator.io
    className: nginx

externalURL: https://harbor.helixterminator.io

persistence:
  imageChartStorage:
    type: s3
    s3:
      bucket: helixterm-harbor-registry-prod
      region: us-east-1
      encrypt: true
      keyid: "${HARBOR_KMS_KEY_ID}"

database:
  type: external
  external:
    host: helixterm-prod.cluster-xxxx.us-east-1.rds.amazonaws.com
    port: "5432"
    username: harbor_registry
    coreDatabase: harbor_registry   # its own logical database,
        §8.3.5

redis:
  type: external
  external:
```

```
    addr: "helixterm-
        prod.xxxxxx.clustercfg.use1.cache.amazonaws.com:6379"

trivy:
  enabled: true
  vulnType: "os,library"
  severity: "CRITICAL,HIGH,MEDIUM"

notary:
  enabled: false   # image signing is Cosign-based (§10.9), not
        Notary

# Harbor Robot Account used by CI (§5) to push; distinct from the
# human-admin account, scoped to project push+pull only.
proxy:
  htpasswd: ""  # managed via Harbor API (robot accounts), not
        chart values

garbageCollection:
  schedule: "0 3 * * *"   # nightly GC of untagged manifests

metrics:
  enabled: true
  serviceMonitor:
    enabled: true   # scraped by the existing Prometheus in §7.1
```

Replication policy (configured post-install via the Harbor API, idempotent script `scripts/harbor/configure-replication.sh`): a **pull-through cache** rule mirrors `docker.io`, `ghcr.io`, and `bitnami` base images referenced anywhere in §4's Dockerfiles into Harbor's local project namespaces on first pull, so a Docker Hub or GHCR outage never blocks a production rollout or a `docker pull` inside an air-gapped incident-response session. A second **push-based replication** rule mirrors the `helixterm/*` project (the platform's own built images) from `harbor.helixterminator.io` to the `eu-west-1` DR region's Harbor instance every 15 minutes, so a regional failover (§8.6) never blocks on registry availability either.

---

7. Observability Stack

7.1 Prometheus Configuration

```yaml
# infrastructure/kubernetes/base/monitoring/prometheus-config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
  namespace: helixterm-monitoring
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s
      evaluation_interval: 15s
      scrape_timeout: 10s
      external_labels:
        cluster: helixterm-prod
        region: us-east-1

    rule_files:
      - /etc/prometheus/rules/*.yaml

    alerting:
      alertmanagers:
        - static_configs:
            - targets: ['alertmanager:9093']
          timeout: 5s

    scrape_configs:
      # Kubernetes API server
      - job_name: 'kubernetes-apiservers'
        kubernetes_sd_configs:
          - role: endpoints
        scheme: https
        tls_config:
          ca_file: /var/run/secrets/kubernetes.io/serviceaccount/
        ca.crt
        bearer_token_file: /var/run/secrets/kubernetes.io/
        serviceaccount/token
        relabel_configs:
          - source_labels: [__meta_kubernetes_namespace,
        __meta_kubernetes_service_name,
        __meta_kubernetes_endpoint_port_name]
            action: keep
```

```
      regex: default;kubernetes;https

# Node exporter
- job_name: 'kubernetes-nodes'
  scheme: https
  tls_config:
    ca_file: /var/run/secrets/kubernetes.io/serviceaccount/
  ca.crt
  bearer_token_file: /var/run/secrets/kubernetes.io/
  serviceaccount/token
  kubernetes_sd_configs:
    - role: node
  relabel_configs:
    - action: labelmap
      regex: __meta_kubernetes_node_label_(.+)

# Pod metrics via prometheus.io annotations
- job_name: 'helixterm-pods'
  kubernetes_sd_configs:
    - role: pod
      namespaces:
        names: ['helixterm-prod', 'helixterm-staging']
  relabel_configs:
    - source_labels:
  [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
      action: keep
      regex: "true"
    - source_labels:
  [__meta_kubernetes_pod_annotation_prometheus_io_path]
      action: replace
      target_label: __metrics_path__
      regex: (.+)
    - source_labels: [__address__,
  __meta_kubernetes_pod_annotation_prometheus_io_port]
      action: replace
      regex: ([^:]+)(?::\d+)?;(\d+)
      replacement: $1:$2
      target_label: __address__
    - action: labelmap
      regex: __meta_kubernetes_pod_label_(.+)
    - source_labels: [__meta_kubernetes_namespace]
      action: replace
      target_label: kubernetes_namespace
    - source_labels: [__meta_kubernetes_pod_name]
      action: replace
      target_label: kubernetes_pod_name
```

```
      - source_labels: [__meta_kubernetes_pod_label_app]
        action: replace
        target_label: service

  # Kafka via JMX exporter
  - job_name: 'kafka'
    static_configs:
      - targets: ['kafka-0.kafka-headless:9308',
    'kafka-1.kafka-headless:9308', 'kafka-2.kafka-
    headless:9308']
    relabel_configs:
      - target_label: cluster
        replacement: helixterm-prod

  # PostgreSQL via postgres_exporter
  - job_name: 'postgresql'
    static_configs:
      - targets: ['postgres-exporter:9187']

  # Redis via redis_exporter
  - job_name: 'redis'
    static_configs:
      - targets: ['redis-exporter:9121']

  # Nginx Ingress Controller
  - job_name: 'nginx-ingress'
    kubernetes_sd_configs:
      - role: pod
        namespaces:
          names: ['ingress-nginx']
    relabel_configs:
      - source_labels:
    [__meta_kubernetes_pod_label_app_kubernetes_io_name]
        action: keep
        regex: ingress-nginx
```

7.2 Prometheus Alert Rules

```
# infrastructure/kubernetes/base/monitoring/alert-rules.yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: helixterm-alerts
  namespace: helixterm-monitoring
  labels:
```

```yaml
    role: alert-rules
spec:
  groups:
    # ── SSH Service Alerts ──────────────────────────────────────
    - name: ssh.alerts
      interval: 30s
      rules:
        - alert: SSHConnectionFailureRateHigh
          expr: |
            sum(rate(ssh_connection_total{status="failed"}[5m])) /
            sum(rate(ssh_connection_total[5m])) > 0.05
          for: 2m
          labels:
            severity: critical
            team: ssh-team
            runbook: https://docs.helixterm.io/runbooks/ssh-high-failure-rate
          annotations:
            summary: "SSH connection failure rate is above 5%"
            description: "SSH failure rate is {{ $value | humanizePercentage }} over the last 5 minutes. Threshold: 5%."

        - alert: SSHConnectionFailureRateWarning
          expr: |
            sum(rate(ssh_connection_total{status="failed"}[5m])) /
            sum(rate(ssh_connection_total[5m])) > 0.01
          for: 5m
          labels:
            severity: warning
            team: ssh-team
          annotations:
            summary: "SSH connection failure rate elevated"
            description: "SSH failure rate is {{ $value | humanizePercentage }} over the last 5 minutes."

        - alert: SSHSessionDurationAnomaly
          expr: |
            histogram_quantile(0.99, rate(ssh_session_duration_seconds_bucket[10m])) > 7200
          for: 5m
          labels:
            severity: warning
            team: ssh-team
          annotations:
```

```
        summary: "SSH session p99 duration is abnormally high"
        description: "p99 SSH session duration is {{ $value |
      humanizeDuration }}. Sessions exceeding 2 hours may
      indicate runaway processes."

    - alert: SSHProxyHighConcurrentSessions
      expr: ssh_concurrent_sessions > 9000
      for: 1m
      labels:
        severity: warning
        team: ssh-team
      annotations:
        summary: "SSH proxy approaching max concurrent
      sessions"
        description: "Concurrent sessions: {{ $value }}. Max:
      10000. Consider scaling."

# ─── Vault Service Alerts ──────────────────────────────────

- name: vault.alerts
  rules:
    - alert: VaultEncryptLatencyHigh
      expr: |
        histogram_quantile(0.99,
      rate(vault_encrypt_duration_seconds_bucket[5m])) > 0.1
      for: 5m
      labels:
        severity: critical
        team: security-team
        runbook: https://docs.helixterm.io/runbooks/vault-
      latency
      annotations:
        summary: "Vault encrypt p99 latency > 100ms"
        description: "Vault encrypt p99: {{ $value |
      humanizeDuration }}. Threshold: 100ms."

    - alert: VaultDecryptLatencyHigh
      expr: |
        histogram_quantile(0.95,
      rate(vault_decrypt_duration_seconds_bucket[5m])) > 0.05
      for: 5m
      labels:
        severity: warning
        team: security-team
      annotations:
        summary: "Vault decrypt p95 latency > 50ms"
        description: "Vault decrypt p95: {{ $value |
      humanizeDuration }}."
```

```
    - alert: VaultUnsealRequired
      expr: vault_sealed == 1
      for: 0m
      labels:
        severity: critical
        team: security-team
        pager: "true"
      annotations:
        summary: "Vault service is sealed and requires unseal"
        description:
    "Vault service instance {{ $labels.pod }} is sealed.
    Immediate action required."

# ─── API Gateway Alerts
    ──────────────────────────────────────────────────

- name: gateway.alerts
  rules:
    - alert: GatewayHighErrorRate
      expr: |

    sum(rate(http_requests_total{service="gateway",status=~"5.."}
    [5m])) /
        sum(rate(http_requests_total{service="gateway"}[5m]))
    > 0.01
      for: 2m
      labels:
        severity: critical
        team: platform
        runbook: https://docs.helixterm.io/runbooks/gateway-
    high-error-rate
      annotations:
        summary: "Gateway HTTP 5xx error rate > 1%"
        description: "Gateway error rate: {{ $value |
    humanizePercentage }}."

    - alert: GatewayHighLatencyP99
      expr: |
        histogram_quantile(0.99,
    sum(rate(http_request_duration_seconds_bucket{service="gateway"}
    [5m])) by (le)) > 2
      for: 5m
      labels:
        severity: critical
        team: platform
      annotations:
        summary: "Gateway p99 latency > 2 seconds"
```

```
      description: "Gateway p99: {{ $value |
    humanizeDuration }}."

  - alert: GatewayHighLatencyP95
    expr: |
      histogram_quantile(0.95,
    sum(rate(http_request_duration_seconds_bucket{service="gateway"}
    [5m])) by (le)) > 0.5
    for: 5m
    labels:
      severity: warning
      team: platform
    annotations:
      summary: "Gateway p95 latency > 500ms"
      description: "Gateway p95: {{ $value |
    humanizeDuration }}."

  - alert: GatewayRequestRateDrop
    expr: |
      sum(rate(http_requests_total{service="gateway"}[5m]))
    <
      sum(rate(http_requests_total{service="gateway"}[5m]
    offset 1h)) * 0.5
    for: 5m
    labels:
      severity: warning
      team: platform
    annotations:
      summary: "Gateway request rate dropped >50% compared
    to 1 hour ago"
      description: "Possible traffic issue or upstream DNS
    problem."

# ── Kafka Alerts ──────────────
  - alert: KafkaConsumerLagHigh
    expr: |
      kafka_consumer_group_lag{topic!~"__.*"} > 10000
    for: 5m
    labels:
      severity: critical
      team: platform
      runbook: https://docs.helixterm.io/runbooks/kafka-
    consumer-lag
    annotations:
      summary: "Kafka consumer group
    {{ $labels.consumergroup }} lag > 10000 on
    {{ $labels.topic }}"
```

```
    description: "Lag: {{ $value }} messages. Possible
slow consumer or processing bottleneck."

- alert: KafkaConsumerLagWarning
  expr: |
    kafka_consumer_group_lag{topic!~"__.*"} > 1000
  for: 10m
  labels:
    severity: warning
    team: platform
  annotations:
    summary: "Kafka consumer lag elevated for
{{ $labels.consumergroup }}"
    description: "Lag: {{ $value }} messages on topic
{{ $labels.topic }}."

- alert: KafkaBrokerDown
  expr: |
    count(kafka_brokers) < 3
  for: 1m
  labels:
    severity: critical
    team: platform
    runbook: https://docs.helixterm.io/runbooks/kafka-
broker-down
  annotations:
    summary: "Kafka cluster has fewer than 3 brokers"
    description: "Active brokers: {{ $value }}. Cluster
may be degraded."

- alert: KafkaUnderReplicatedPartitions
  expr: |
    kafka_underreplicated_partitions > 0
  for: 2m
  labels:
    severity: critical
    team: platform
  annotations:
    summary: "Kafka has {{ $value }} under-replicated
partitions"
    description: "Under-replicated partitions indicate
broker or network issues."
    #
NOTE: the ssh.alerts and vault.alerts groups defined
earlier in
    # this file (§7.2 top) are the single canonical
definitions for
```

```
    # those alert groups. A second, conflicting
  ssh.alerts/vault.alerts
    # pair (different metric names/thresholds) previously
  existed here
    # and has been removed as a duplicate — Prometheus
  rule files must
    # not declare two groups with the same `name`.

- name: database.alerts
  rules:
    - alert: PostgresConnectionPoolExhausted
      expr: |
        pg_stat_activity_count / pg_settings_max_connections >
  0.85
      for: 5m
      labels:
        severity: critical
        team: data
        runbook: https://docs.helixterm.io/runbooks/postgres-
  pool
      annotations:
        summary: "PostgreSQL connection pool > 85% utilized"
        description: "{{ $value | humanizePercentage }}
  connections used. Risk of connection refusal."

    - alert: PostgresReplicationLag
      expr: |
        pg_replication_lag_seconds > 30
      for: 5m
      labels:
        severity: critical
        team: data
        runbook: https://docs.helixterm.io/runbooks/postgres-
  replication
      annotations:
        summary: "PostgreSQL replication lag > 30 seconds"
        description: "Replication lag: {{ $value |
  humanizeDuration }}. RPO at risk."

    - alert: PostgresSlowQueries
      expr: |
        pg_stat_statements_mean_exec_time_seconds > 1
      for: 5m
      labels:
        severity: warning
        team: data
      annotations:
```

```
    summary: "PostgreSQL slow queries detected (mean >
1s)"
    description: "Query: {{ $labels.query }} - mean time:
{{ $value | humanizeDuration }}."

- alert: PostgresDiskSpaceWarning
  expr: |
    (pg_database_size_bytes /
node_filesystem_size_bytes{mountpoint="/var/lib/
postgresql"}) > 0.75
  for: 10m
  labels:
    severity: warning
    team: data
  annotations:
    summary: "PostgreSQL disk usage > 75%"
    description: "Disk at {{ $value |
humanizePercentage }}. Plan for expansion."

- alert: RedisCacheHitRateLow
  expr: |
    redis_keyspace_hits_total / (redis_keyspace_hits_total
+ redis_keyspace_misses_total) < 0.80
  for: 10m
  labels:
    severity: warning
    team: platform
  annotations:
    summary: "Redis cache hit rate < 80%"
    description: "Hit rate: {{ $value |
humanizePercentage }}. Possible cache thrashing."

- alert: RedisMemoryUsageHigh
  expr: |
    redis_memory_used_bytes / redis_memory_max_bytes >
0.90
  for: 5m
  labels:
    severity: critical
    team: platform
    runbook: https://docs.helixterm.io/runbooks/redis-
memory
  annotations:
    summary: "Redis memory usage > 90%"
    description: "Redis memory: {{ $value |
humanizePercentage }}. Risk of eviction."

- alert: RedisDown
```

```
      expr: |
        redis_up == 0
      for: 1m
      labels:
        severity: critical
        team: platform
      annotations:
        summary: "Redis instance is down"
        description: "Redis at {{ $labels.instance }} is
    unreachable."

- name: infra.alerts
  rules:
    - alert: NodeMemoryPressure
      expr: |
        (node_memory_MemTotal_bytes -
    node_memory_MemAvailable_bytes) /
    node_memory_MemTotal_bytes > 0.90
      for: 5m
      labels:
        severity: critical
        team: infra
        runbook: https://docs.helixterm.io/runbooks/node-
    memory
      annotations:
        summary: "Node {{ $labels.instance }} memory > 90%"
        description: "Memory usage: {{ $value |
    humanizePercentage }}."

    - alert: NodeDiskSpaceCritical
      expr: |
        (node_filesystem_size_bytes -
    node_filesystem_avail_bytes) / node_filesystem_size_bytes
    > 0.90
      for: 5m
      labels:
        severity: critical
        team: infra
      annotations:
        summary: "Node {{ $labels.instance }} disk
    {{ $labels.mountpoint }} > 90% full"
        description: "Disk usage: {{ $value |
    humanizePercentage }}."

    - alert: PodCrashLooping
      expr: |
        rate(kube_pod_container_status_restarts_total[15m]) *
    60 * 15 > 3
```

```
    for: 5m
    labels:
      severity: critical
      team: platform
    annotations:
      summary: "Pod {{ $labels.namespace }}/
{{ $labels.pod }} is crash-looping"
      description: "Restarts in last 15 min: {{ $value }}."

  - alert: PodNotReady
    expr: |
      kube_pod_status_ready{condition="true"} == 0
    for: 5m
    labels:
      severity: warning
      team: platform
    annotations:
      summary: "Pod {{ $labels.namespace }}/
{{ $labels.pod }} is not ready"
      description: "Pod has been not ready for > 5 minutes."

  - alert: HorizontalPodAutoscalerMaxReached
    expr: |
      kube_horizontalpodautoscaler_status_current_replicas
== kube_horizontalpodautoscaler_spec_max_replicas
    for: 10m
    labels:
      severity: warning
      team: platform
    annotations:
      summary: "HPA {{ $labels.namespace }}/
{{ $labels.horizontalpodautoscaler }} at max replicas"
      description: "HPA has been at maximum capacity for >
10 minutes."

  - alert: CertificateExpiryWarning
    expr: |
      certmanager_certificate_expiration_timestamp_seconds -
time() < 86400 * 14
    for: 1h
    labels:
      severity: warning
      team: platform
      runbook: https://docs.helixterm.io/runbooks/cert-
renewal
    annotations:
```

```
    summary: "TLS certificate {{ $labels.name }} expires
in < 14 days"
    description: "Certificate expires: {{ $value |
humanizeDuration }} from now."

- alert: CertificateExpiryCritical
  expr: |
    certmanager_certificate_expiration_timestamp_seconds -
time() < 86400 * 3
  for: 1h
  labels:
    severity: critical
    team: platform
  annotations:
    summary: "TLS certificate {{ $labels.name }} expires
in < 3 days"
    description: "URGENT: Renew certificate immediately."
```

7.5 Grafana Dashboard Definitions

Key dashboard panels and their PromQL queries, organized by board:

Board: HelixTerminator Platform Overview

Panel	Type	Query
Total RPS	Stat	`sum(rate(http_requests_total{namespace="helixterm-prod"}[2m]`
P99 Latency (Gateway)	Gauge	`histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket{service="gatew [5m])) by (le))`
Error Rate	Stat	`sum(rate(http_requests_total{status=~"5.."}[2m])) / sum(rate(http_requests_total[2m]))`
Active SSH Sessions	Stat	`ssh_active_sessions_total`
Kafka Consumer Lag	Time series	`kafka_consumer_group_lag`
Pod Restarts (24h)	Table	`sum by (pod) (increase(kube_pod_container_status_restarts_total[24h]))`
Redis Hit Rate	Gauge	`redis_keyspace_hits_total / (redis_keyspace_hits_total + redis_keyspace_misses_total)`
		`pg_stat_activity_count`

Panel	Type	Query
PostgreSQL Connections	Time series	

Board: SSH Service Deep-Dive

Panel	Type	Query
Connections/min (success)	Time series	`sum(rate(ssh_connections_total{status="success"}[1m])) * 60`
Connections/min (failure)	Time series	`sum(rate(ssh_connections_total{status="failure"}[1m])) * 60`
Session Duration p50/p95/p99	Time series	`histogram_quantile(0.50|0.95|0.99, rate(ssh_session_duration_seconds_bucket[5m]))`
Auth Method Breakdown	Pie	`sum by (method)(ssh_auth_attempts_total)`
Top Target Hosts	Table	`topk(10, sum by (target_host)(ssh_connections_total))`

Board: Vault Service Latency

Panel	Type	Query
Encrypt p50/p95/p99	Time series	`histogram_quantile(0.50|0.95|0.99, rate(vault_encrypt_duration_seconds_bucket[5m]))`
Decrypt p50/p95/p99	Time series	`histogram_quantile(0.50|0.95|0.99, rate(vault_decrypt_duration_seconds_bucket[5m]))`
Operations/sec	Time series	`sum(rate(vault_operations_total[1m]))`
Error Rate	Stat	`sum(rate(vault_operations_total{status="error"}[5m]))/sum(rate(vault_operations_total[5m]))`

7.6 OpenTelemetry Collector Configuration

```yaml
# infrastructure/observability/otel-collector.yaml
apiVersion: v1
kind: ConfigMap
metadata:
```

```
  name: otel-collector-config
  namespace: helixterm-monitoring
data:
  config.yaml: |
    receivers:
      otlp:
        protocols:
          grpc:
            endpoint: 0.0.0.0:4317
          http:
            endpoint: 0.0.0.0:4318

      prometheus:
        config:
          scrape_configs:
            - job_name: otel-collector
              scrape_interval: 10s
              static_configs:
                - targets: ['0.0.0.0:8888']

      jaeger:
        protocols:
          grpc:
            endpoint: 0.0.0.0:14250
          thrift_http:
            endpoint: 0.0.0.0:14268

      zipkin:
        endpoint: 0.0.0.0:9411

    processors:
      batch:
        timeout: 1s
        send_batch_size: 1024
        send_batch_max_size: 2048

      memory_limiter:
        check_interval: 1s
        limit_mib: 512
        spike_limit_mib: 128

      resource:
        attributes:
          - key: deployment.environment
```

```
        value: production
        action: upsert
      - key: service.namespace
        value: helixterm
        action: upsert

  attributes:
    actions:
      - key: http.user_agent
        action: delete
      - key: db.statement
        action: hash

  filter:
    error_mode: ignore
    traces:
      span:
        - 'attributes["http.route"] == "/healthz"'
        - 'attributes["http.route"] == "/readyz"'
        - 'attributes["http.route"] == "/metrics"'

  tail_sampling:
    decision_wait: 10s
    num_traces: 100000
    expected_new_traces_per_sec: 1000
    policies:
      - name: errors-policy
        type: status_code
        status_code:
          status_codes: [ERROR]
      - name: slow-traces-policy
        type: latency
        latency:
          threshold_ms: 1000
      - name: probabilistic-policy
        type: probabilistic
        probabilistic:
          sampling_percentage: 5

exporters:
  otlp/jaeger:
    endpoint: jaeger-collector.helixterm-
     monitoring.svc.cluster.local:4317
    tls:
```

```
      insecure: false
      ca_file: /etc/ssl/certs/ca-certificates.crt

  prometheus:
    endpoint: 0.0.0.0:8889
    namespace: otelcol
    send_timestamps: true
    metric_expiration: 180m

  loki:
    endpoint: http://loki.helixterm-
    monitoring.svc.cluster.local:3100/loki/api/v1/push
    labels:
      resource:
        service.name: "service_name"
        service.namespace: "namespace"
        deployment.environment: "environment"

extensions:
  health_check:
    endpoint: 0.0.0.0:13133
  pprof:
    endpoint: 0.0.0.0:1777
  zpages:
    endpoint: 0.0.0.0:55679

service:
  extensions: [health_check, pprof, zpages]
  pipelines:
    traces:
      receivers: [otlp, jaeger, zipkin]
      processors: [memory_limiter, filter, tail_sampling,
    resource, batch]
      exporters: [otlp/jaeger]
    metrics:
      receivers: [otlp, prometheus]
      processors: [memory_limiter, resource, batch]
      exporters: [prometheus]
    logs:
      receivers: [otlp]
      processors: [memory_limiter, resource, attributes,
    batch]
      exporters: [loki]
```

7.7 Loki Configuration

```yaml
# infrastructure/observability/loki-config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: loki-config
  namespace: helixterm-monitoring
data:
  loki.yaml: |
    auth_enabled: false

    server:
      http_listen_port: 3100
      grpc_listen_port: 9096

    common:
      instance_addr: 127.0.0.1
      path_prefix: /tmp/loki
      storage:
        s3:
          endpoint: s3.amazonaws.com
          region: us-east-1
          bucketnames: helixterm-loki-chunks
          access_key_id: ${AWS_ACCESS_KEY_ID}
          secret_access_key: ${AWS_SECRET_ACCESS_KEY}
      replication_factor: 1
      ring:
        kvstore:
          store: inmemory

    query_range:
      results_cache:
        cache:
          embedded_cache:
            enabled: true
            max_size_mb: 100

    schema_config:
      configs:
        - from: 2024-01-01
          store: tsdb
          object_store: s3
          schema: v13
```

```
      index:
        prefix: loki_index_
        period: 24h

ruler:
  alertmanager_url: http://alertmanager.helixterm-
    monitoring.svc.cluster.local:9093

limits_config:
  reject_old_samples: true
  reject_old_samples_max_age: 168h
  ingestion_rate_mb: 32
  ingestion_burst_size_mb: 64
  max_query_length: 721h
  max_streams_per_user: 0
  retention_period: 90d
```

---

8. Disaster Recovery

8.1 Overview and Objectives

The HelixTerminator platform operates under the following recovery objectives:

Objective	Target	Rationale
Recovery Time Objective (RTO)	30 minutes	Maximum acceptable downtime for any production service
Recovery Point Objective (RPO)	5 minutes	Maximum acceptable data loss window
Mean Time to Detect (MTTD)	2 minutes	Alert firing latency + on-call notification
Mean Time to Respond (MTTR)	5 minutes	On-call engineer acknowledgment SLA

These objectives are enforced through continuous WAL archiving, multi-AZ deployment, automated failover, and regular chaos engineering exercises.

8.2 Availability Zones and Regional Architecture

```
AWS Region: us-east-1 (Primary)
├── AZ: us-east-1a
│   ├── EKS Node Group: system (2 nodes, m5.large)
│   ├── EKS Node Group: general (min 3, max 20, c5.xlarge)
│   ├── RDS Primary Instance
│   ├── ElastiCache Primary Node
│   └── MSK Broker 1
├── AZ: us-east-1b
│   ├── EKS Node Group: general (min 3, max 20, c5.xlarge)
│   ├── RDS Standby Instance (synchronous replication)
│   ├── ElastiCache Replica Node 1
│   └── MSK Broker 2
└── AZ: us-east-1c
    ├── EKS Node Group: general (min 3, max 20, c5.xlarge)
    ├── RDS Read Replica
    ├── ElastiCache Replica Node 2
    └── MSK Broker 3

AWS Region: eu-west-1 (DR / Read Region)
├── EKS Cluster (standby, scaled down)
├── RDS Read Replica (async, cross-region)
├── S3 Cross-Region Replication
└── Route53 Health Checks + Failover Records
```

8.3 PostgreSQL Backup Strategy

8.3.1 Continuous WAL Archiving

WAL archiving is configured via `postgresql.conf` with WAL-E/WAL-G pushing segments to S3 every 60 seconds, achieving an RPO of under 5 minutes.

```
# /etc/postgresql/postgresql.conf additions
wal_level = replica
archive_mode = on
archive_command = 'wal-g wal-push %p'
archive_timeout = 60
max_wal_senders = 5
wal_keep_size = 1GB
hot_standby = on
```

```
# /etc/wal-g/walg.json
{
  "WALG_S3_PREFIX": "s3://helixterm-postgres-backup/prod/wal",
  "AWS_REGION": "us-east-1",
  "WALG_COMPRESSION_METHOD": "brotli",
  "WALG_DELTA_MAX_STEPS": 6,
  "WALG_UPLOAD_CONCURRENCY": 16,
  "WALG_DOWNLOAD_CONCURRENCY": 10,
  "PGDATA": "/var/lib/postgresql/data"
}
```

8.3.2 Daily Full Backup CronJob

```
# infrastructure/kubernetes/base/backup/postgres-backup.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: postgres-full-backup
  namespace: helixterm-prod
spec:
  schedule: "0 2 * * *"   # 02:00 UTC daily
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 7
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName: postgres-backup
          restartPolicy: OnFailure
          containers:
            - name: walg-backup
              image: ghcr.io/helixdevelopment/walg-runner:1.0.0
              command:
                - /bin/sh
                - -c
                - |
                  set -euo pipefail
                  echo "Starting full backup at $(date -u +%Y-%m-
        %dT%H:%M:%SZ)"
                  wal-g backup-push $PGDATA
                  echo "Backup complete"
                  # Retain 30 daily, 4 weekly, 12 monthly
                  wal-g delete retain FIND_FULL 30 --confirm
```

```
            env:
              - name: PGHOST
                valueFrom:
                  secretKeyRef:
                    name: postgres-creds
                    key: host
              - name: PGUSER
                valueFrom:
                  secretKeyRef:
                    name: postgres-creds
                    key: username
              - name: PGPASSWORD
                valueFrom:
                  secretKeyRef:
                    name: postgres-creds
                    key: password
              - name: PGDATA
                value: /var/lib/postgresql/data
            envFrom:
              - secretRef:
                  name: walg-s3-config
            resources:
              requests:
                cpu: "500m"
                memory: "512Mi"
              limits:
                cpu: "2000m"
                memory: "2Gi"
```

8.3.3 Point-in-Time Recovery Procedure

```
#!/bin/bash
# scripts/dr/postgres-pitr-restore.sh
# Usage: ./postgres-pitr-restore.sh <TARGET_TIME_ISO8601>
# Example: ./postgres-pitr-restore.sh "2026-06-28T12:00:00Z"

set -euo pipefail

TARGET_TIME="${1:?Target time required (ISO 8601)}"
RESTORE_PATH="/var/lib/postgresql/data-restore"

echo "[DR] Starting PITR restore to: ${TARGET_TIME}"

# 1. Find the most recent full backup before target time
```

```
BACKUP_NAME=$(wal-g backup-list --json | \
  jq -r --arg t "${TARGET_TIME}" \

        '[.[] | select(.start_time <= $t)] | sort_by(.start_time)
        | last | .backup_name')

echo "[DR] Using base backup: ${BACKUP_NAME}"

# 2. Restore base backup
wal-g backup-fetch "${RESTORE_PATH}" "${BACKUP_NAME}"

# 3. Write recovery configuration
cat > "${RESTORE_PATH}/postgresql.auto.conf" << EOF
restore_command = 'wal-g wal-fetch "%f" "%p"'
recovery_target_time = '${TARGET_TIME}'
recovery_target_action = promote
recovery_target_inclusive = true
EOF

# Create recovery signal file (PostgreSQL 12+)
touch "${RESTORE_PATH}/recovery.signal"

echo "[DR] Restore files prepared. Start PostgreSQL to begin WAL
        replay."
echo "[DR] Monitor recovery progress in postgresql.log"
```

8.3.4 PostgreSQL High Availability and Automated Write-Path Failover

This is the canonical PostgreSQL HA/DR specification for HelixTerminator — doc 01 (core architecture) references this section rather than re-describing it (Remediation Register §4 “PostgreSQL DR + HA”). It closes the gap between “PostgreSQL is on AWS RDS Multi-AZ” (a one-line claim elsewhere in this document, §2.10 / §6.2) and an actual, testable HA/DR contract with named RPO/RTO numbers and a failover mechanism per deployment tier.

Two HA tiers, one failover contract. HelixTerminator runs PostgreSQL in two places: (1) AWS RDS Multi-AZ in production (§6.2, `aws_db_instance.helixterm_prod`) and (2) a self-hosted PostgreSQL StatefulSet (§2.10) for staging, integration tests, DR fire-drills, and any on-prem/non-AWS deployment. Both tiers are governed by the **same** RPO/RTO contract; only the failover *mechanism* differs, because RDS’s synchronous Multi-AZ failover is an AWS-managed control plane the

platform does not operate, while the self-hosted tier needs an explicit, self-hosted failover controller — **Patroni**, backed by an `etcd` distributed-consensus store.

Failure mode	RPO target	RTO target	Mechanism
Intra-region AZ loss (primary AZ down)	**0** (synchronous replication)	≤ **2 minutes**	RDS Multi-AZ automatic failover (prod) / Patroni automatic leader election via `etcd` lease expiry (self-hosted)
Primary instance crash (process/host, AZ healthy)	**0** (synchronous replication)	≤ **60 seconds**	RDS Multi-AZ automatic failover (prod) / Patroni `etcd`-driven leader re-election (self-hosted)
Full regional loss (`us-east-1` down)	≤ **5 minutes** (async cross-region streaming lag budget)	≤ **30 minutes** (matches the platform-wide §8.1 RTO)	Manual-triggered promotion of the `eu-west-1` standby — `aws rds promote-read-replica` (prod) / `patronictl failover --candidate <eu-west-1-leader>` against the Patroni standby cluster (self-hosted)

These numbers are the PostgreSQL-specific refinement of the platform-wide table in §8.1 — the platform RTO/RPO (30 min / 5 min) is the *outer bound* that must hold across every dependency; PostgreSQL's *intra-*

region numbers (0 RPO, ≤ 2 min RTO) are materially tighter because synchronous Multi-AZ replication is already zero-data-loss and near-immediate.

Patroni configuration (self-hosted tier). Patroni wraps each PostgreSQL process, uses `etcd` as the distributed consensus store for leader election, and drives promote/demote/reconfigure through PostgreSQL's own `pg_ctl` + `recovery.conf`/`standby.signal` mechanics — this is the same role AWS's internal Multi-AZ agent plays for RDS, made explicit and operable for the self-hosted tier:

```
# infrastructure/kubernetes/base/postgresql/patroni-config.yaml
# Mounted into every postgresql StatefulSet pod (§2.10) at
# /etc/patroni/patroni.yml — supersedes ad-hoc postgresql.conf
       edits for
# the self-hosted tier; RDS is unaffected (AWS manages its own
       control plane).
scope: helixterm-postgresql
namespace: /helixterm/
name: "{{ POD_NAME }}"

restapi:
  listen: 0.0.0.0:8008
  connect_address: "{{ POD_IP }}:8008"

etcd3:
  hosts:
    - etcd-0.etcd.helixterm-prod.svc.cluster.local:2379
    - etcd-1.etcd.helixterm-prod.svc.cluster.local:2379
    - etcd-2.etcd.helixterm-prod.svc.cluster.local:2379

bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover:
        1048576   # 1MB — refuse to promote a replica more than
        1MB behind
    synchronous_mode: true             # zero-data-loss intra-
        region: matches RDS Multi-AZ's RPO=0
    synchronous_mode_strict: false     # do not block writes
        entirely if the sole sync replica is down
    postgresql:
      use_pg_rewind: true
      parameters:
        wal_level: replica
```

```
        hot_standby: "on"
        max_wal_senders: 10
        max_replication_slots: 10
        wal_keep_size: 1GB
        synchronous_commit: "on"
        archive_mode: "on"
        archive_command: "wal-g wal-push %p"   # same WAL-G
        pipeline as §8.3.1

  # Cross-region DR: eu-west-1 runs its own Patroni cluster in
  # "standby cluster" mode — a full HA cluster (its own etcd +
        local
  # synchronous replicas for intra-DR-region resilience) that is
        itself an
  # ASYNC standby of the us-east-1 primary. This is what makes the
  # cross-region link resilient to a single DR-region node loss
        without
  # ever promoting across regions unintentionally.
  standby_cluster:
    host: helixterm-postgresql-leader.helixterm-
        prod.svc.cluster.local
    port: 5432
    create_replica_methods:
      - basebackup

postgresql:
  listen: 0.0.0.0:5432
  connect_address: "{{ POD_IP }}:5432"
  data_dir: /var/lib/postgresql/data
  authentication:
    replication:
      username: replicator
      password: "{{ REPLICATOR_PASSWORD }}"
    superuser:
      username: postgres
      password: "{{ POSTGRES_SUPERUSER_PASSWORD }}"

tags:
  nofailover: false
  noloadbalance: false
  clonefrom: false
  nosync: false
```

`on_role_change` / `on_start` callback that keeps the Kubernetes Service pointed at whichever pod Patroni just promoted (the piece that makes failover invisible to every service's `PGHOST`, which always resolves to the same in-cluster Service name regardless of which pod is currently primary):

```
#!/bin/bash
# infrastructure/kubernetes/base/postgresql/patroni-callback.sh
# Registered in patroni.yml as
        `postgresql.callbacks.on_role_change`.
set -euo pipefail
ROLE="$1"   # "master" or "replica" (Patroni's own vocabulary)
NAME="$2"

if [ "${ROLE}" = "master" ]; then
  kubectl label pod "${NAME}" role=primary --overwrite -n
        helixterm-prod
  kubectl label pod --all -l app=postgresql,role=primary --
        overwrite -n helixterm-prod \
    --selector="!(metadata.name==${NAME})" role=replica 2>/dev/
        null || true
  echo "[patroni-callback] ${NAME} promoted to primary — Service
        selector now routes here"
fi
```

Failover runbook — write-path only (self-hosted / Patroni tier). Distinct from and complementary to the full application-tier failover runbook in §8.6 (which is written for the RDS-managed prod tier and covers DNS/Helm/scale-up as well as the database). This runbook is scoped purely to the write path when Patroni is the controller (staging, DR fire-drills per §8.7, or an on-prem deployment):

1. **Detect** — `patronictl -c /etc/patroni/patroni.yml list` reports the current leader's health, or the `etcd` lease TTL (30s, `bootstrap.dcs.ttl` above) expires without renewal, which is Patroni's own automatic detection signal — no external monitor is required for the automatic case.
2. **Automatic leader election (intra-region)** — the remaining synchronous replica with the smallest replication lag acquires the `etcd` leader key and self-promotes; `maximum_lag_on_failover` (1MB) refuses to promote a replica that is materially behind, preferring to stay down over a silent-data-loss promotion.
3. **Callback fires** — `patroni-callback.sh` relabels the new leader pod; the Kubernetes Service selector re-resolves within one kube-proxy sync interval (typically < 5s), so `PGHOST=postgresql.helixterm-prod.svc.cluster.local` never needs to change in any service's configuration.

4. **Verify** — `patronictl list` shows exactly one `Leader` and N `Sync Standby`/`Replica` rows in `streaming` state; cross-check `SELECT * FROM pg_stat_replication;` on the new leader for `replay_lag` near zero.
5. **Cross-region promotion (manual, DR-only)** — operator-triggered: `patronictl failover --candidate <eu-west-1-pod> --force`. This is the ONLY step in this runbook that is not automatic — promoting across regions is a deliberate, high-blast-radius operator decision (mirrors §8.6 Step 3's `aws rds promote-read-replica` for the RDS tier), never triggered by lag/lease-expiry alone.
6. **Post-failover** — re-point the DR region's own `standby_cluster` config at the new leader (or, if the DR region itself was promoted, reconfigure `us-east-1` as the new standby once it recovers), then follow §8.6 Steps 4–7 for the application-tier half of the recovery.

Restore-drill procedure is specified in §8.3.5 immediately below, alongside the per-service-database PITR/backup cadence — the two are the same continuous WAL-archiving + nightly full-backup pipeline (§8.3.1/§8.3.2) applied per logical database rather than per instance.

8.3.5 Per-Service Database PITR, Retention, and Restore-Drill

Topology. Every microservice owns a logically isolated PostgreSQL database (database-per-service, not shared-schema) inside the same Multi-AZ instance/cluster described in §6.2 — doc 01's service registry is the authoritative enumeration of the full service→database mapping (per CD-3, this document does not re-enumerate it divergently). A representative slice, sufficient to illustrate the retention model without asserting an unverified total count:

Service	Database	Notes
auth-service	`auth`	credentials, sessions, MFA state
vault-service	`vault`	encrypted secret containers (client-side zero-knowledge payloads only, CD-10)
rbac-service	`rbac`	roles, org/team membership
audit-service	`audit`	append-only audit event log (hash-chained)
session-recorder	`session_recorder`	recording index (blobs live in S3, §6.2

Service	Database	Notes
		`session_recordings` bucket)
harbor-registry (platform tier, §6.3)	`harbor_registry`	Harbor's own metadata — not a HelixTerminator microservice, but co-located on the same instance
(remaining ~20 per-service databases)	*(see doc 01 §3 service registry)*	same backup/ retention contract applies uniformly

Retention is instance-wide, applied per database. Continuous WAL archiving (§8.3.1, `archive_timeout = 60`) and the nightly full backup (§8.3.2, `wal-g backup-push`) operate on the whole PostgreSQL instance — one WAL stream, one base backup — which means every per-service database is captured atomically in the same backup artifact; there is no database-level backup gap. The retention window is:

Tier	Cadence	Retention	Applies to
WAL (continuous archive)	every 60s (`archive_timeout`)	5 minutes of recovery granularity (RPO)	every database, instance-wide
Full backup	daily, 02:00 UTC (§8.3.2 CronJob)	30 daily + 4 weekly + 12 monthly (`wal-g delete retain FIND_FULL 30`)	every database, instance-wide
Cross-region replica	continuous (async streaming)	≤ 5 min replication lag budget	every database, replicated to `eu-west-1` as a unit

Surgical single-database restore (recovering one service's data without a full-instance PITR cutover) restores the whole instance into a disposable scratch target, then extracts only the target database:

```
#!/bin/bash
# scripts/dr/postgres-restore-single-db.sh
```

```
# Usage: ./postgres-restore-single-db.sh <SERVICE_DB_NAME>
        <TARGET_TIME_ISO8601>
set -euo pipefail

SERVICE_DB="${1:?Service database name required, e.g. vault}"
TARGET_TIME="${2:?Target time required (ISO 8601)}"
SCRATCH_HOST="postgres-restore-scratch.helixterm-
        prod.svc.cluster.local"

echo "[DR] Restoring full instance to scratch target at $
        {TARGET_TIME}"
./scripts/dr/postgres-pitr-restore.sh "${TARGET_TIME}"
# (operator starts the scratch instance from the prepared restore
        path,
#  waits for `recovery_target_action = promote` to complete)

echo "[DR] Extracting single database '$
        {SERVICE_DB}' from scratch instance"
pg_dump --host="${SCRATCH_HOST}" --username=postgres --
        format=custom \
  --dbname="${SERVICE_DB}" --file="/tmp/${SERVICE_DB}-$
        {TARGET_TIME}.dump"

echo "[DR] Restoring '${SERVICE_DB}' into target instance"
pg_restore --host="${PGHOST}" --username=postgres --dbname="$
        {SERVICE_DB}" \
  --clean --if-exists "/tmp/${SERVICE_DB}-${TARGET_TIME}.dump"

echo "[DR] Single-database restore complete: ${SERVICE_DB} @ $
        {TARGET_TIME}"
```

Restore-drill procedure — the automated evidence behind §8.7's "Backup restore verification | Weekly | Automated" row. This is not a manual checklist; it is a script that actually performs the restore and verifies content, producing a pass/fail artifact rather than a "the backup job didn't error" absence-of-failure claim:

```
#!/bin/bash
# scripts/dr/postgres-restore-drill.sh
# Runs weekly via CronJob (infrastructure/kubernetes/base/backup/
        restore-drill-cronjob.yaml).
# Restores the most recent full backup into an isolated shadow
        instance and
# verifies every per-service database is present with a sane row
        count —
# never marks the drill green on "the restore command exited 0"
        alone.
set -euo pipefail

DRILL_ID="drill-$(date -u +%Y%m%dT%H%M%SZ)"
```

```
SHADOW_HOST="postgres-restore-drill.helixterm-
        staging.svc.cluster.local"
REPORT_PATH="/var/log/dr-drills/${DRILL_ID}.json"

echo "[DRILL $
        {DRILL_ID}] Restoring latest full backup into shadow
        instance"
LATEST_BACKUP=$(wal-g backup-list --json | jq -r
        'sort_by(.start_time) | last | .backup_name')
wal-g backup-fetch "/var/lib/postgresql/shadow-data" "$
        {LATEST_BACKUP}"

echo "[DRILL ${DRILL_ID}] Verifying per-service databases on
        shadow instance"
FAILURES=0
DBS=$(psql --host="${SHADOW_HOST}" --username=postgres --tuples-
        only --command="SELECT datname FROM pg_database WHERE
        datistemplate = false;")
for db in ${DBS}; do
  ROWCOUNT=$(psql --host="${SHADOW_HOST}" --username=postgres --
        dbname="${db}" --tuples-only \
    --command="SELECT COALESCE(SUM(n_live_tup), 0) FROM
        pg_stat_user_tables;" | tr -d '[:space:]')
  if [ "${ROWCOUNT}" -eq 0 ]; then
    echo "[DRILL ${DRILL_ID}] FAIL: database '$
        {db}' restored with zero live rows"
    FAILURES=$((FAILURES + 1))
  else
    echo "[DRILL ${DRILL_ID}] OK: database '${db}' — ${ROWCOUNT}
        live rows"
  fi
done

cat > "${REPORT_PATH}" <<EOF
{"drill_id": "${DRILL_ID}", "backup": "${LATEST_BACKUP}",
        "failures": ${FAILURES}, "verdict": "$([ ${FAILURES} -eq
        0 ] && echo PASS || echo FAIL)"}
EOF

echo "[DRILL ${DRILL_ID}] Report: ${REPORT_PATH}"
[ "${FAILURES}" -eq 0 ] || exit 1
```

8.4 Redis Backup Strategy

Redis is configured with both RDB snapshots and AOF persistence for combined durability:

```
# Redis configuration additions (redis.conf)
# RDB: snapshot every 60s if >= 1000 keys changed, every 300s if
```

```
>= 100 changed
save 60 1000
save 300 100
save 900 1

# AOF: append-only file with fsync every second
appendonly yes
appendfsync everysec
no-appendfsync-on-rewrite no
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 128mb

# Backup file naming
dbfilename dump-<timestamp>.rdb
dir /data/redis

# Replication
replicaof <primary-host> 6379
replica-serve-stale-data yes
replica-read-only yes
```

Redis backups are pushed to S3 using a sidecar container every 5 minutes:

```
# Redis backup sidecar container snippet (added to Redis
        StatefulSet)
- name: redis-backup
  image: ghcr.io/helixdevelopment/redis-backup:1.0.0
  command:
    - /bin/sh
    - -c
    - |
      while true; do
        TIMESTAMP=$(date -u +%Y%m%dT%H%M%SZ)
        redis-cli --no-auth-warning -a "${REDIS_PASSWORD}" BGSAVE
        sleep 5
        aws s3 cp /data/redis/dump.rdb \
          s3://helixterm-redis-backup/prod/${TIMESTAMP}/dump.rdb \
          --storage-class STANDARD_IA
        sleep 295
      done
  volumeMounts:
    - name: redis-data
      mountPath: /data/redis
```

8.5 Kafka Data Retention and Replication

Kafka is configured for durability with a replication factor of 3 and min-ISR of 2:

```
# Kafka broker configuration
default.replication.factor=3
min.insync.replicas=2
unclean.leader.election.enable=false
log.retention.hours=168          # 7 days default
log.retention.bytes=107374182400 # 100GB per partition cap
log.segment.bytes=1073741824     # 1GB segment size
log.cleanup.policy=delete

# Topic-level overrides for critical topics
# ssh.sessions → retain 30 days
# audit.events → retain 90 days
# vault.operations → retain 30 days
```

Kafka topic retention overrides:

```
# scripts/kafka/configure-retention.sh
kafka-configs.sh --bootstrap-server kafka:9092 --entity-type
        topics \
  --entity-name ssh.sessions \
  --alter --add-config retention.ms=2592000000  # 30 days

kafka-configs.sh --bootstrap-server kafka:9092 --entity-type
        topics \
  --entity-name audit.events \
  --alter --add-config retention.ms=7776000000  # 90 days

kafka-configs.sh --bootstrap-server kafka:9092 --entity-type
        topics \
  --entity-name vault.operations \
  --alter --add-config retention.ms=2592000000  # 30 days
```

Kafka MirrorMaker 2 replicates critical topics to the DR region:

```
# infrastructure/kubernetes/base/kafka/mirrormaker2.yaml
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaMirrorMaker2
metadata:
  name: helixterm-mm2
  namespace: helixterm-prod
spec:
  version: 3.9.0
```

```yaml
replicas: 2
connectCluster: us-east-1
clusters:
  - alias: us-east-1
    bootstrapServers: kafka-bootstrap.helixterm-
      prod.svc.cluster.local:9093
    tls:
      trustedCertificates:
        - secretName: kafka-cluster-ca-cert
          certificate: ca.crt
    authentication:
      type: tls
      certificateAndKey:
        secretName: mm2-tls
        certificate: user.crt
        key: user.key
  - alias: eu-west-1
    bootstrapServers: kafka.helixterm-dr.eu-
      west-1.example.com:9093
    tls:
      trustedCertificates:
        - secretName: kafka-dr-ca-cert
          certificate: ca.crt
    authentication:
      type: tls
      certificateAndKey:
        secretName: mm2-dr-tls
        certificate: user.crt
        key: user.key
mirrors:
  - sourceCluster: us-east-1
    targetCluster: eu-west-1
    sourceConnector:
      config:
        replication.factor: 3
        offset-syncs.topic.replication.factor: 3
        sync.topic.acls.enabled: "false"
        replication.policy.separator: "."
        replication.policy.class:
       "org.apache.kafka.connect.mirror.IdentityReplicationPolicy"
    heartbeatConnector:
      config:
        heartbeats.topic.replication.factor: 3
    checkpointConnector:
      config:
```

```
      checkpoints.topic.replication.factor: 3
      sync.group.offsets.enabled: "true"
    topicsPattern: "ssh\\..*|audit\\..*|vault\\..*|sessions\
      \..*"
    groupsPattern: "helix-.*"
```

8.6 Cross-Region Failover Runbook

This runbook covers the full procedure for failing over from `us-east-1` (primary) to `eu-west-1` (DR).

Step 1: Declare Incident

```
# Trigger PagerDuty incident
curl -X POST https://events.pagerduty.com/v2/enqueue \
  -H "Content-Type: application/json" \
  -d '{
    "routing_key": "'${PAGERDUTY_ROUTING_KEY}'",
    "event_action": "trigger",
    "payload": {
      "summary": "REGIONAL FAILOVER INITIATED: us-east-1 → eu-
        west-1",
      "severity": "critical",
      "source": "dr-runbook"
    }
  }'
```

Step 2: Verify DR Region Status

```
# Switch kubectl context to DR cluster
kubectl config use-context helix-dr-eu-west-1

# Verify cluster health
kubectl get nodes
kubectl get pods -n helixterm-prod

# Check replication lag
aws rds describe-db-instances \
  --db-instance-identifier helixterm-postgres-dr \
  --query 'DBInstances[0].StatusInfos'
```

Step 3: Promote PostgreSQL DR Replica

```
# Promote the read replica to primary (AWS RDS)
aws rds promote-read-replica \
  --db-instance-identifier helixterm-postgres-dr \
```

```
  --region eu-west-1

# Wait for promotion to complete (typically 3-5 minutes)
aws rds wait db-instance-available \
  --db-instance-identifier helixterm-postgres-dr \
  --region eu-west-1

echo "PostgreSQL DR promoted to primary"
```

Step 4: Scale Up DR Kubernetes Workloads

```
# Scale up all deployments from 0 to production scale
kubectl scale deployment --all -n helixterm-prod --replicas=2

# Wait for rollout
kubectl rollout status deployment -n helixterm-prod --timeout=600s

# Apply production values (Helm)
helm upgrade helix-terminator ./infrastructure/helm/helix-
        terminator \
  --namespace helixterm-prod \
  --values ./infrastructure/helm/helix-terminator/values-
        production.yaml \
  --set global.region=eu-west-1 \
  --set global.postgresHost=$(get-dr-postgres-endpoint)
```

Step 5: Update Route53 DNS

```
# Flip primary DNS record to DR endpoint
aws route53 change-resource-record-sets \
  --hosted-zone-id ${HOSTED_ZONE_ID} \
  --change-batch '{
    "Changes": [{
      "Action": "UPSERT",
      "ResourceRecordSet": {
        "Name": "api.helixterm.io",
        "Type": "A",
        "AliasTarget": {
          "HostedZoneId": "'${DR_ALB_HOSTED_ZONE_ID}'",
          "DNSName": "'${DR_ALB_DNS}'",
          "EvaluateTargetHealth": true
        }
      }
    }]
  }'
```

Step 6: Validate Failover

```
# scripts/dr/validate-failover.sh
set -euo pipefail

BASE_URL="https://api.helixterm.io"

# Health check
HTTP_STATUS=$(curl -so /dev/null -w "%{http_code}" "${BASE_URL}/
        healthz")
[ "$HTTP_STATUS" = "200" ] || { echo "FAIL: health check returned
        $HTTP_STATUS"; exit 1; }

# Auth smoke test
TOKEN=$(curl -sX POST "${BASE_URL}/v1/auth/token" \
  -H "Content-Type: application/json" \
  -d '{"username":"smoke-test","password":"'$
        {SMOKE_TEST_PASSWORD}'"}' | jq -r .token)
[ -n "$TOKEN" ] && [ "$TOKEN" != "null" ] || { echo "FAIL: auth
        returned no token"; exit 1; }

echo "PASS: Failover validated. API responding from DR region."
```

Step 7: Post-Failover Actions

- Update status page (StatusPage.io / Atlassian)
- Notify engineering leadership
- Begin incident retrospective
- Schedule primary region remediation
- Re-establish replication back to primary once restored
- Conduct failback test within 48 hours

8.7 Regular DR Exercises

Exercise	Frequency	Type	Scope
Backup restore verification	Weekly	Automated	PostgreSQL PITR to shadow DB
Redis snapshot restore	Weekly	Automated	Restore dump to isolated Redis
Kafka consumer replay	Monthly	Manual	Replay 1h of events on staging

Exercise	Frequency	Type	Scope
Full regional failover drill	Quarterly	Manual	Full runbook execution in staging
Chaos engineering (pod kill)	Weekly	Automated	Chaos Mesh random pod deletion
Chaos engineering (network partition)	Monthly	Manual	Simulate AZ failure in staging

8.8 DR Runbook Sequence Diagram

The following sequence diagram is the visual form of the §8.6 cross-region failover runbook — the on-call engineer, PagerDuty, the DR-region Kubernetes API, RDS, and Route53 as the participants, in the same 7-step order §8.6 describes in prose:

```
sequenceDiagram
    autonumber
    actor OC as On-Call Engineer
    participant PD as PagerDuty
    participant K8sDR as EKS (eu-west-1, DR)
    participant RDS as RDS PostgreSQL
    participant R53 as Route53
    participant Users as End Users

    Note over OC,Users: us-east-1 (primary) declared down

    OC->>PD: Declare incident (§8.6 Step 1)
    PD-->>OC: Incident channel opened, on-call paged

    OC->>K8sDR: kubectl config use-context helix-dr-eu-west-1
    OC->>K8sDR: Verify cluster health + replication lag (§8.6 Step
2)
    K8sDR-->>OC: Nodes Ready, pods Pending (scaled to 0)

    OC->>RDS: aws rds promote-read-replica (§8.6 Step 3)
    RDS-->>OC: Promotion in progress (~3-5 min)
    RDS-->>OC: DB instance available, now primary
```

```
    OC->>K8sDR: kubectl scale deployment --all --replicas=2 (§8.6
Step 4)
    K8sDR-->>OC: Rollout complete
    OC->>K8sDR: helm upgrade --set global.region=eu-west-1

    OC->>R53: UPSERT api.helixterm.io -> DR ALB (§8.6 Step 5)
    R53-->>Users: DNS resolves to eu-west-1 (propagation ~60s, low
TTL)

    OC->>K8sDR: scripts/dr/validate-failover.sh (§8.6 Step 6)
    K8sDR-->>OC: /healthz 200, auth smoke test token issued
    OC->>PD: Mark incident mitigated

    Note over OC,Users: Post-failover (§8.6 Step 7): status page,
retro,<br/>re-establish replication back to us-east-1, failback
test in 48h
```

8.9 Canary Promotion Sequence Diagram

Companion diagram for the §5.3 release pipeline's five-stage canary promotion (5% → 25% → 50% → 100%), each stage gated by an automated monitoring window before the next promotion — see §5.4 for the CI/CD job graph this diagram is drawn from.

```
sequenceDiagram
    autonumber
    participant GH as GitHub Actions (release.yml)
    participant Harbor as Harbor / OCI Chart Repo
    participant K8s as EKS (us-east-1, prod)
    participant Mon as monitor-canary.py (Prometheus)
    actor OC as On-Call / Release Owner

    GH->>Harbor: helm upgrade helixterm-prod-canary --set
canary.weight=5
    Harbor-->>K8s: Chart pulled, canary pods rolled out (5%
traffic)
    GH->>Mon: Monitor canary 5% (10 min)
    Mon-->>GH: error-rate/latency within budget -> proceed

    GH->>Harbor: helm upgrade --set canary.weight=25
    Harbor-->>K8s: Canary scaled to 25% traffic
    GH->>Mon: Monitor canary 25% (10 min)
    Mon-->>GH: within budget -> proceed
```

```
    GH->>Harbor: helm upgrade --set canary.weight=50
    Harbor-->>K8s: Canary scaled to 50% traffic
    GH->>Mon: Monitor canary 50% (15 min)
    Mon-->>GH: within budget -> proceed

    GH->>K8s: helm upgrade helixterm-prod (100%, full promotion)
    K8s-->>GH: Rollout complete
    GH->>K8s: helm uninstall helixterm-prod-canary

    alt Any monitoring window breaches budget
        Mon-->>GH: error-rate/latency budget breached
        GH->>K8s: helm rollback helixterm-prod --wait
        GH->>K8s: helm uninstall helixterm-prod-canary
        GH->>OC: Slack page :rotating_light: PRODUCTION ROLLBACK
TRIGGERED
    end
```

---

9. Local Development

9.1 Prerequisites

Before setting up the HelixTerminator development environment, install the following tools:

Tool	Minimum Version	Install
Go	1.25	`brew install go` or go.dev/dl
Flutter	3.24+	flutter.dev/docs/get-started
Docker	27.x	docs.docker.com
Podman	5.x	`brew install podman`
kubectl	1.31+	`brew install kubectl`
helm	3.16+	`brew install helm`
k3d / kind	latest	`brew install k3d`
golangci-lint	1.61+	`brew install golangci-lint`
protoc	28+	

Tool	Minimum Version	Install
		`brew install protobuf`
buf	1.40+	`brew install bufbuild/buf/buf`
jq	1.7+	`brew install jq`
yq	4.x	`brew install yq`
mkcert	latest	`brew install mkcert`

9.2 Initial Repository Setup

```
#!/bin/bash
# scripts/dev/init-repo.sh
# Run once after cloning the repository

set -euo pipefail

REPO_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/../.." && pwd)"

echo "==> Initializing HelixTerminator development environment"
echo "    Repository root: ${REPO_ROOT}"

# 1. Initialize all Git submodules
echo "==> Fetching submodules..."
git submodule update --init --recursive --depth=1

# 2. Verify Go version
REQUIRED_GO="1.25"
CURRENT_GO=$(go version | awk '{print $3}' | sed 's/go//')
if [[ "$(printf '%s\n' "$REQUIRED_GO" "$CURRENT_GO" | sort -V |
         head -n1)" != "$REQUIRED_GO" ]]; then
  echo "ERROR: Go ${REQUIRED_GO}+ required. Found: ${CURRENT_GO}"
  exit 1
fi

# 3. Install Go tools
echo "==> Installing Go development tools..."
go install github.com/golangci/golangci-lint/cmd/golangci-
        lint@latest
go install github.com/securego/gosec/v2/cmd/gosec@latest
go install golang.org/x/vuln/cmd/govulncheck@latest
```

```
go install github.com/grpc-ecosystem/grpc-gateway/v2/protoc-gen-
        grpc-gateway@latest
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest

# 4. Install Node tools (for contract testing)
echo "==> Installing Node.js tools..."
npm install -g @pact-foundation/pact-node

# 5. Set up local TLS certificates
echo "==> Configuring local TLS (mkcert)..."
mkcert -install
mkcert "*.helixterm.local" helixterm.local localhost 127.0.0.1 ::1
mv _wildcard.helixterm.local+3.pem certs/local-tls.pem
mv _wildcard.helixterm.local+3-key.pem certs/local-tls-key.pem

# 6. Copy environment template
if [ ! -f "${REPO_ROOT}/.env" ]; then
  cp "${REPO_ROOT}/.env.example" "${REPO_ROOT}/.env"
  echo "==> Copied .env.example → .env. Edit as needed."
fi

# 7. Pull container images for local dev
echo "==> Pre-pulling dependency images..."
docker compose -f docker-compose.deps.yml pull

# 8. Generate Go protobufs
echo "==> Generating protobuf files..."
buf generate

# 9. Download Go module dependencies
echo "==> Downloading Go modules..."
go work sync

echo ""
echo "==> Setup complete! Run 'make dev-up' to start the
        development environment."
```

9.3 Go Workspace Configuration

The monorepo uses Go workspaces (`go.work`) to manage all 25 services and shared modules:

```
# go.work
go 1.25
```

```
use (
    ./services/gateway
    ./services/auth-service
    ./services/vault-service
    ./services/ssh-proxy
    ./services/session-service
    ./services/tunnel-service
    ./services/key-manager
    ./services/policy-engine
    ./services/audit-service
    ./services/notification-service
    ./services/user-service
    ./services/org-service
    ./services/billing-service
    ./services/metrics-collector
    ./services/health-check
    ./services/config-service
    ./services/secret-rotator
    ./services/certificate-service
    ./services/bastion-manager
    ./services/recording-service
    ./services/replay-service
    ./services/search-service
    ./services/webhook-service
    ./services/scheduler-service
    ./services/ai-analyzer
    ./submodules/containers
    ./submodules/docs_chain
    ./submodules/security
    ./submodules/auth
)
```

9.4 Docker Compose for Local Development

```yaml
# docker-compose.yml (full dependency stack for local development)
version: "3.9"

networks:
  helix-net:
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.0.0/16
```

```yaml
volumes:
  postgres-data:
  redis-data:
  kafka-data:
  zookeeper-data:
  rabbitmq-data:
  vault-data:
  minio-data:
  jaeger-data:

services:
  # ── PostgreSQL ──────────────────────────────────────────────────────
  postgres:
    image: postgres:17-alpine
    container_name: helix-postgres
    restart: unless-stopped
    networks:
      - helix-net
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: helixterm
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-localdev}
      POSTGRES_DB: helixterm
      POSTGRES_INITDB_ARGS: "--encoding=UTF8 --locale=C"
    volumes:
      - postgres-data:/var/lib/postgresql/data
      - ./scripts/db/init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U helixterm -d helixterm"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
    command: >
      postgres
      -c max_connections=200
      -c shared_buffers=256MB
      -c effective_cache_size=1GB
      -c wal_level=logical
      -c max_wal_senders=10
```

```yaml
# ── Redis ──────────────────────────────────────────────────────────
redis:
  image: redis:8-alpine
  container_name: helix-redis
  restart: unless-stopped
  networks:
    - helix-net
  ports:
    - "6379:6379"
  command: >
    redis-server
    --requirepass ${REDIS_PASSWORD:-localdev}
    --maxmemory 512mb
    --maxmemory-policy allkeys-lru
    --appendonly yes
    --appendfsync everysec
  volumes:
    - redis-data:/data
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD:-
      localdev}", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5

# ── Zookeeper (for local Kafka) ────────────────────────────────────
zookeeper:
  image: confluentinc/cp-zookeeper:7.9.0
  container_name: helix-zookeeper
  restart: unless-stopped
  networks:
    - helix-net
  ports:
    - "2181:2181"
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181
    ZOOKEEPER_TICK_TIME: 2000
    ZOOKEEPER_SYNC_LIMIT: 2
  volumes:
    - zookeeper-data:/var/lib/zookeeper/data

# ── Kafka ──────────────────────────────────────────────────────────
kafka:
```

```yaml
  image: confluentinc/cp-kafka:7.9.0
  container_name: helix-kafka
  restart: unless-stopped
  depends_on:
    zookeeper:
      condition: service_started
  networks:
    - helix-net
  ports:
    - "9092:9092"
    - "9101:9101"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
      PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://
      kafka:29092,PLAINTEXT_HOST://localhost:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
    KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
    KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
    KAFKA_NUM_PARTITIONS: 3
    KAFKA_DEFAULT_REPLICATION_FACTOR: 1
    KAFKA_JMX_PORT: 9101
    KAFKA_JMX_HOSTNAME: localhost
  volumes:
    - kafka-data:/var/lib/kafka/data
  healthcheck:
    test: ["CMD-SHELL", "kafka-broker-api-versions --bootstrap-
      server localhost:9092"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 60s

# ─── RabbitMQ ──────────────────────────────────────────────────

rabbitmq:
  image: rabbitmq:4-management-alpine
  container_name: helix-rabbitmq
  restart: unless-stopped
  networks:
    - helix-net
  ports:
```

```
    - "5672:5672"
    - "15672:15672"
  environment:
    RABBITMQ_DEFAULT_USER: helixterm
    RABBITMQ_DEFAULT_PASS: ${RABBITMQ_PASSWORD:-localdev}
    RABBITMQ_DEFAULT_VHOST: helix
  volumes:
    - rabbitmq-data:/var/lib/rabbitmq
  healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "ping"]
    interval: 30s
    timeout: 10s
    retries: 5

# ── HashiCorp Vault (dev mode) ──────────────────────────────
vault:
  image: hashicorp/vault:1.19
  container_name: helix-vault
  restart: unless-stopped
  networks:
    - helix-net
  ports:
    - "8200:8200"
  cap_add:
    - IPC_LOCK
  environment:
    VAULT_DEV_ROOT_TOKEN_ID: ${VAULT_DEV_TOKEN:-dev-root-token}
    VAULT_DEV_LISTEN_ADDRESS: 0.0.0.0:8200
    VAULT_LOG_LEVEL: warn
  volumes:
    - vault-data:/vault/file
    - ./scripts/vault/init:/vault/init:ro
  healthcheck:
    test: ["CMD", "vault", "status"]
    interval: 10s
    timeout: 5s
    retries: 5

# ── MinIO (S3 mock for local dev) ──────────────────────────
minio:
  image: minio/minio:RELEASE.2025-04-08T15-41-24Z
  container_name: helix-minio
  restart: unless-stopped
```

```yaml
    networks:
      - helix-net
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: ${MINIO_USER:-minioadmin}
      MINIO_ROOT_PASSWORD: ${MINIO_PASSWORD:-minioadmin}
    volumes:
      - minio-data:/data
    command: server /data --console-address ":9001"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:9000/minio/
        health/live"]
      interval: 30s
      timeout: 20s
      retries: 3

  # ── Jaeger (local tracing) ──────────────────────────────────
  jaeger:
    image: jaegertracing/all-in-one:1.62
    container_name: helix-jaeger
    restart: unless-stopped
    networks:
      - helix-net
    ports:
      - "16686:16686"    # Jaeger UI
      - "4317:4317"      # OTLP gRPC
      - "4318:4318"      # OTLP HTTP
      - "14268:14268"    # Jaeger HTTP
    environment:
      COLLECTOR_OTLP_ENABLED: "true"
      SPAN_STORAGE_TYPE: badger
      BADGER_EPHEMERAL: "false"
      BADGER_DIRECTORY_VALUE: /badger/data
      BADGER_DIRECTORY_KEY: /badger/key
    volumes:
      - jaeger-data:/badger

  # ── Prometheus ──────────────────────────────────────────────
  prometheus:
    image: prom/prometheus:v3.0.0
    container_name: helix-prometheus
```

```
    restart: unless-stopped
    networks:
      - helix-net
    ports:
      - "9090:9090"
    volumes:
      - ./infrastructure/observability/prometheus-local.yaml:/etc/
        prometheus/prometheus.yml:ro

  # ── Grafana ──────────────────────────────────────────────────────

  grafana:
    image: grafana/grafana:11.3.0
    container_name: helix-grafana
    restart: unless-stopped
    networks:
      - helix-net
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD:-localdev}
      GF_AUTH_DISABLE_LOGIN_FORM: "false"
    volumes:
      - ./infrastructure/observability/grafana/provisioning:/etc/
        grafana/provisioning:ro
      - ./infrastructure/observability/grafana/dashboards:/var/
        lib/grafana/dashboards:ro

  # ── SMTP Mock (Mailhog) ──────────────────────────────────────────

  mailhog:
    image: mailhog/mailhog:v1.0.1
    container_name: helix-mailhog
    restart: unless-stopped
    networks:
      - helix-net
    ports:
      - "1025:1025"
      - "8025:8025"
```

9.5 Makefile Targets for Local Development

```
# Makefile
```

```
.PHONY: dev-up dev-down dev-reset dev-build dev-test lint test-
        unit test-integration proto-gen db-seed

COMPOSE_FILE := docker-compose.yml
SERVICES := $(shell ls services/)

## ─── Environment ──────────────────────────────────────────────

dev-up:  ## Start all infrastructure dependencies
    docker compose -f $(COMPOSE_FILE) up -d
    @echo "==> Waiting for services to be healthy..."
    @./scripts/dev/wait-for-deps.sh
    @echo "==> Dev environment ready."
    @echo "    PostgreSQL: localhost:5432"
    @echo "    Redis:      localhost:6379"
    @echo "    Kafka:      localhost:9092"
    @echo "    Jaeger UI:  http://localhost:16686"
    @echo "    Grafana:    http://localhost:3000"
    @echo "    MinIO:      http://localhost:9001"
    @echo "    Vault:      http://localhost:8200"

dev-down:  ## Stop all infrastructure dependencies
    docker compose -f $(COMPOSE_FILE) down

dev-reset:  ## Destroy and recreate all volumes (full reset)
    docker compose -f $(COMPOSE_FILE) down -v
    docker compose -f $(COMPOSE_FILE) up -d

## ─── Build ────────────────────────────────────────────────────

build:  ## Build all Go services
    @for svc in $(SERVICES); do \
      echo "Building $$svc..."; \
      CGO_ENABLED=0 GOOS=linux go build -o bin/$$svc ./services/$
        $svc/cmd/...; \
    done

build/%:  ## Build a specific service (e.g. make build/gateway)
    CGO_ENABLED=0 GOOS=linux go build -o bin/$* ./services/$*/
        cmd/...

## ─── Code Quality ─────────────────────────────────────────────
```

```
lint:  ## Run golangci-lint across all services
    golangci-lint run ./...

lint-fix:  ## Run golangci-lint with auto-fix
    golangci-lint run --fix ./...

fmt:  ## Run gofmt + goimports
    gofmt -w ./services ./submodules
    goimports -w ./services ./submodules

vet:  ## Run go vet
    go vet ./...

## ─── Testing ──────────────────────────────────────────────────────

test-unit:  ## Run unit tests across all services
    go test -v -race -timeout 120s
        -coverprofile=coverage.out ./...
    go tool cover -html=coverage.out -o coverage.html

test-unit/%:  ## Run unit tests for a specific service
    go test -v -race -timeout 120s -coverprofile=coverage-
        $*.out ./services/$*/...

test-integration:  ## Run integration tests (requires running
        deps)
    go test -v -race -timeout 300s -tags=integration ./...

test-e2e:  ## Run E2E tests against local environment
    go test -v -timeout 600s -tags=e2e ./tests/e2e/...

## ─── Database ─────────────────────────────────────────────────────

db-migrate:  ## Run database migrations
    ./scripts/db/migrate.sh up

db-rollback:  ## Rollback last database migration
    ./scripts/db/migrate.sh down 1

db-seed:  ## Seed database with development test data
    go run ./scripts/db/seed/main.go

db-reset:  ## Drop and recreate database, run migrations, seed
    ./scripts/db/reset.sh
```

```
## ─── Proto generation ──────────────────────────────────────────

proto-gen:  ## Generate protobuf code
	buf generate

proto-lint:  ## Lint protobuf definitions
	buf lint

proto-breaking:  ## Check for breaking changes
	buf breaking --against '.git#branch=main'

## ─── Kubernetes (local k3d) ────────────────────────────────────

k8s-create-local:  ## Create local k3d cluster
	k3d cluster create helix-local \
	  --config infrastructure/k3d/local-cluster.yaml

k8s-delete-local:  ## Delete local k3d cluster
	k3d cluster delete helix-local

k8s-deploy-local:  ## Deploy services to local k3d cluster
	helm upgrade --install helix-terminator ./infrastructure/helm/
	    helix-terminator \
	  --namespace helixterm-dev --create-namespace \
	  --values ./infrastructure/helm/helix-terminator/values-
	    development.yaml

## ─── Hot Reload ────────────────────────────────────────────────

watch/%:  ## Watch and auto-rebuild/restart a service (requires
	    air)
	cd services/$* && air -c .air.toml

## ─── Security ──────────────────────────────────────────────────

sec-scan:  ## Run gosec security scan
	gosec -fmt=sarif -out=gosec-results.sarif ./...

vuln-check:  ## Run govulncheck
	govulncheck ./...

trivy-scan/%:  ## Scan a built image with Trivy
```

```
    trivy image ghcr.io/helixdevelopment/$*:local

help:  ## Display this help
    @grep -E '^[a-zA-Z_/-]+:.*?## .*$$' $(MAKEFILE_LIST) | \
      awk
        'BEGIN {FS = ":.*?## "}; {printf "  \033[36m%-30s\033[0m
        %s\n", $$1, $$2}'
```

9.6 Hot Reload Configuration

Each Go service uses Air for hot reload during development:

```
# services/gateway/.air.toml
root = "."
testdata_dir = "testdata"
tmp_dir = "tmp"

[build]
args_bin = []
bin = "./tmp/main"
cmd = "go build -o ./tmp/main ./cmd/gateway"
delay = 500
exclude_dir = ["assets", "tmp", "vendor", "testdata"]
exclude_file = []
exclude_regex = ["_test.go"]
exclude_unchanged = false
follow_symlink = false
full_bin = ""
include_dir = []
include_ext = ["go", "tpl", "tmpl", "html"]
include_file = []
kill_delay = "0s"
log = "build-errors.log"
poll = false
poll_interval = 0
post_cmd = []
pre_cmd = []
rerun = false
rerun_delay = 500
send_interrupt = false
stop_on_error = false

[color]
app = ""
build = "yellow"
```

```
main = "magenta"
runner = "green"
watcher = "cyan"

[log]
main_only = false
time = false

[misc]
clean_on_exit = false

[proxy]
app_port = 0
enabled = false
proxy_port = 0

[screen]
clear_on_rebuild = false
keep_scroll = true
```

9.7 Database Seeding

```
// scripts/db/seed/main.go
// +build tools

package main

import (
    "context"
    "database/sql"
    "fmt"
    "log"
    "os"
    "time"

    _ "github.com/lib/pq"
    "github.com/brianvoe/gofakeit/v7"
)

func main() {
    dsn := os.Getenv("DATABASE_URL")
    if dsn == "" {
        dsn = "postgres://helixterm:localdev@localhost:5432/
        helixterm?sslmode=disable"
```

```
}

db, err := sql.Open("postgres", dsn)
if err != nil {
    log.Fatalf("Failed to connect: %v", err)
}
defer db.Close()

ctx := context.Background()

log.Println("Seeding organizations...")
for i := 0; i < 5; i++ {
    orgID := gofakeit.UUID()
    _, err := db.ExecContext(ctx, `
        INSERT INTO organizations (id, name, slug, plan,
    created_at)
        VALUES ($1, $2, $3, 'enterprise', $4)
        ON CONFLICT DO NOTHING`,
        orgID,
        gofakeit.Company(),
        fmt.Sprintf("org-%d", i),
        time.Now().UTC(),
    )
    if err != nil {
        log.Printf("Warning: org seed: %v", err)
    }
}

log.Println("Seeding users...")
_, err = db.ExecContext(ctx, `
    INSERT INTO users (id, email, hashed_password, role,
    org_id, created_at)
    VALUES
      ('00000000-0000-0000-0000-000000000001',
    'admin@helixterm.local',
       '$2a$12$VeryHashedPasswordForDevOnly', 'admin', NULL,
    NOW()),
      ('00000000-0000-0000-0000-000000000002',
    'dev@helixterm.local',
       '$2a$12$VeryHashedPasswordForDevOnly', 'user', NULL,
    NOW())
    ON CONFLICT (email) DO NOTHING`)
if err != nil {
    log.Printf("Warning: user seed: %v", err)
}
```

```
    log.Println("Seeding SSH targets...")
    for i := 0; i < 10; i++ {
        _, err := db.ExecContext(ctx, `
            INSERT INTO ssh_targets (id, hostname, port, os_type,
        created_at)
            VALUES ($1, $2, 22, 'linux', NOW())
            ON CONFLICT DO NOTHING`,
            gofakeit.UUID(),
            fmt.Sprintf("target-%d.helixterm.local", i),
        )
        if err != nil {
            log.Printf("Warning: ssh target seed: %v", err)
        }
    }

    log.Println("Database seed complete.")
}
```

9.8 Environment Variables Reference

```
# .env.example — copy to .env and customize

# ── Database ──────────────────────────────────────────────
DATABASE_URL=postgres://helixterm:localdev@localhost:5432/
        helixterm?sslmode=disable
POSTGRES_PASSWORD=localdev

# ── Redis ─────────────────────────────────────────────────
REDIS_URL=redis://:localdev@localhost:6379/0
REDIS_PASSWORD=localdev

# ── Kafka ─────────────────────────────────────────────────
KAFKA_BROKERS=localhost:9092
KAFKA_CONSUMER_GROUP=helix-dev

# ── RabbitMQ ──────────────────────────────────────────────
RABBITMQ_URL=amqp://helixterm:localdev@localhost:5672/helix
RABBITMQ_PASSWORD=localdev

# ── Vault ─────────────────────────────────────────────────
VAULT_ADDR=http://localhost:8200
VAULT_TOKEN=dev-root-token
```

```
# ── Auth ──────────────────────────────────────────────────────────
JWT_SECRET=dev-jwt-secret-min-32-chars-long-please
JWT_ISSUER=helixterm-dev
JWT_EXPIRY=3600s

# ── Observability ─────────────────────────────────────────────────
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
OTEL_SERVICE_NAME=gateway
OTEL_TRACES_SAMPLER=traceidratio
OTEL_TRACES_SAMPLER_ARG=0.1

# ── Storage ───────────────────────────────────────────────────────
S3_ENDPOINT=http://localhost:9000
S3_ACCESS_KEY=minioadmin
S3_SECRET_KEY=minioadmin
S3_BUCKET_RECORDINGS=helix-recordings-dev
S3_REGION=us-east-1

# ── Service Ports ─────────────────────────────────────────────────
GATEWAY_PORT=8080
AUTH_SERVICE_PORT=8081
VAULT_SERVICE_PORT=8082
SSH_PROXY_PORT=2222
METRICS_PORT=9090

# ── Feature Flags ─────────────────────────────────────────────────
FEATURE_AI_ANALYZER=true
FEATURE_SESSION_RECORDING=true
FEATURE_AUDIT_LOG=true

# ── Log Level ─────────────────────────────────────────────────────
LOG_LEVEL=debug
LOG_FORMAT=text
```

9.9 Mock Services for External Dependencies

When working on services that call external APIs (Stripe, SendGrid, AWS services), use local mocks:

```go
// internal/testutil/mocks/stripe_mock.go
package mocks

import (
    "encoding/json"
    "net/http"
    "net/http/httptest"
)

// NewStripeMockServer returns a test server that mimics Stripe's
    API
func NewStripeMockServer() *httptest.Server {
    mux := http.NewServeMux()

    mux.HandleFunc("/v1/customers", func(w http.ResponseWriter, r
        *http.Request) {
        if r.Method == http.MethodPost {
            json.NewEncoder(w).Encode(map[string]interface{}{
                "id":     "cus_mock_" + randString(10),
                "object": "customer",
                "email":  r.FormValue("email"),
            })
        }
    })

    mux.HandleFunc("/v1/subscriptions", func(w
        http.ResponseWriter, r *http.Request) {
        json.NewEncoder(w).Encode(map[string]interface{}{
            "id":     "sub_mock_" + randString(10),
            "object": "subscription",
            "status": "active",
        })
    })

    return httptest.NewServer(mux)
}
```

9.10 Cloud Development Environment (Coder, Rootless)

§9.1–§9.9 cover a laptop-local dev loop; this subsection specifies the **cloud development environment** — a self-hosted, browser-accessible workspace for contributors who cannot (or should not) run the full 25-service dependency stack (§9.4) on a laptop, and for onboarding a new engineer to a fully-provisioned environment in minutes rather than a multi-hour local setup. This was previously unprovisioned (no infra, no

Terraform, no workspace template existed anywhere in this document) — it is specified here as a self-hosted **Coder** deployment, chosen over a third-party SaaS (GitHub Codespaces, Gitpod) so workspace containers can run **rootless Podman** (§11.4.161) end to end instead of a vendor-managed Docker-in-Docker sidecar the platform does not control.

Placement. Coder's control plane runs on the `system` EKS node group (same tier as Harbor, §6.3) in a dedicated `helixterm-dev` namespace; per-developer workspaces are Kubernetes Pods on the `general` node group, so workspace capacity autoscales with the same cluster autoscaler already managing production load.

```
# infrastructure/terraform/modules/coder/main.tf
resource "kubernetes_namespace" "helixterm_dev" {
  metadata {
    name = "helixterm-dev"
    labels = {
      "pod-security.kubernetes.io/enforce" = "baseline"  #
workspaces run rootless Podman, not privileged Docker
    }
  }
}

resource "helm_release" "coder" {
  name       = "coder"
  namespace  = kubernetes_namespace.helixterm_dev.metadata[0].name
  repository = "https://helm.coder.com/v2"
  chart      = "coder"
  version    = "2.18.0"

  values = [yamlencode({
    coder = {
      env = [
        { name = "CODER_ACCESS_URL", value = "https://
dev.helixterminator.io" }
        { name = "CODER_OIDC_ISSUER_URL", value = "https://
auth.helixterminator.io" }  # SSO via the platform's own auth-
service (§7 IdP)
        { name = "CODER_OIDC_CLIENT_ID", valueFrom =
{ secretKeyRef = { name = "coder-oidc", key = "client-id" } } }
        { name = "CODER_OIDC_CLIENT_SECRET", valueFrom =
{ secretKeyRef = { name = "coder-oidc", key = "client-
secret" } } }
      ]
      ingress = {
```

```
        enable = true
        host   = "dev.helixterminator.io"
        tls    = { enable = true, secretNames = ["coder-tls"] }
      }
    }
  })]
}
```

Workspace template — every workspace is a rootless-Podman-capable Pod (the same `containers.ContainerRuntime` abstraction from Appendix B, using `PodmanRuntime`, not `DockerRuntime`), with the repo cloned and `go.work` (§9.3) pre-resolved, and the local dependency stack (§9.4) available via `podman-compose` inside the workspace itself — never a shared multi-tenant Postgres/Kafka/Redis, so one developer's workload cannot corrupt another's:

```
# infrastructure/terraform/modules/coder/templates/
helixterminator-workspace/main.tf
resource "coder_agent" "main" {
  os             = "linux"
  arch           = "amd64"
  startup_script = <<-EOT
    set -euo pipefail
    git clone --recurse-submodules
git@github.com:HelixDevelopment/helix_terminator.git ~/
helix_terminator
    cd ~/helix_terminator
    bash scripts/dev/init-repo.sh   # §9.2 — same bootstrap
laptop-local dev uses
    code-server --auth none --port 13337 &
  EOT
}

resource "kubernetes_pod" "workspace" {
  metadata {
    name      = "coder-${data.coder_workspace.me.owner}-$
{data.coder_workspace.me.name}"
    namespace = "helixterm-dev"
  }
  spec {
    security_context {
      run_as_non_root = true
      run_as_user      = 65532   # matches the distroless nonroot
UID used by every service image, §4.1
      seccomp_profile { type = "RuntimeDefault" }
```

```
    }
    container {
      name    = "workspace"
      image   = "harbor.helixterminator.io/helixterm/dev-
workspace:1.0.0"  # built via §5, includes go1.25 + podman + code-
server
      command = ["sh", "-c", coder_agent.main.init_script]

      resources {
        requests = { cpu = "2", memory = "4Gi" }
        limits   = { cpu = "4", memory = "8Gi" }
      }

      port { container_port = 8080 }  # code-server / dev preview
(matches the platform's internal gateway port, §2.3)
      port { container_port = 443 }   # local TLS-terminated
preview of the gateway under test
    }
  }
}
```

Cost + lifecycle controls (tie-in to §11 FinOps): workspaces auto-stop after 2 hours of idle CPU (`coder templates edit --ttl 2h`, `autostop_requirement`), so an abandoned workspace never runs at full `general` node group cost indefinitely — the single biggest historical source of "forgotten dev box" spend in cloud-dev-environment deployments.

---

10. Security Hardening

10.1 Kubernetes Pod Security Standards

All namespaces enforce the `restricted` Pod Security Standard, which is the most secure built-in profile:

```yaml
# infrastructure/kubernetes/base/namespaces/helixterm-prod.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: helixterm-prod
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: v1.31
```

```
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/audit-version: v1.31
    pod-security.kubernetes.io/warn: restricted
    pod-security.kubernetes.io/warn-version: v1.31
    environment: production
    team: platform
```

The `restricted` profile requires every Pod to:

- Set `runAsNonRoot: true`
- Set `seccompProfile.type: RuntimeDefault` or `Localhost`
- Drop all Linux capabilities (`drop: ["ALL"]`)
- Disallow privilege escalation (`allowPrivilegeEscalation: false`)
- Use a non-root UID (`runAsUser: ≥ 1000`)
- Mount volumes only from allowed types (no hostPath)

Every service Deployment in HelixTerminator applies these settings:

```
# Baseline security context applied to ALL containers
securityContext:
  runAsNonRoot: true
  runAsUser:
        65532          # distroless/static:nonroot UID (65534
        "nobody" is undefined in distroless images)
  runAsGroup: 65532
  fsGroup: 65532
  seccompProfile:
    type: RuntimeDefault

containers:
  - name: service
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop:
          - ALL
    volumeMounts:
      - name: tmp-dir
        mountPath: /tmp
      - name: var-run
        mountPath: /var/run

volumes:
  - name: tmp-dir
    emptyDir: {}
```

```
  - name: var-run
    emptyDir: {}
```

10.2 Network Segmentation

The default-deny NetworkPolicy is applied at namespace creation, with explicit allow rules per service:

```
# infrastructure/kubernetes/base/network/default-deny.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: helixterm-prod
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress

---
# Allow DNS for all pods (required for service discovery)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
  namespace: helixterm-prod
spec:
  podSelector: {}
  policyTypes:
    - Egress
  egress:
    - ports:
        - port: 53
          protocol: UDP
        - port: 53
          protocol: TCP

---
# Allow metrics scraping from Prometheus
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-prometheus-scrape
```

```yaml
  namespace: helixterm-prod
spec:
  podSelector: {}
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: helixterm-monitoring
          podSelector:
            matchLabels:
              app: prometheus
      ports:
        - port: 9090
          protocol: TCP

---
# Example: Gateway may receive traffic from the ingress controller
        only
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-ingress-to-gateway
  namespace: helixterm-prod
spec:
  podSelector:
    matchLabels:
      app: gateway
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ingress-nginx
          podSelector:
            matchLabels:
              app.kubernetes.io/name: ingress-nginx
      ports:
        - port: 8080
          protocol: TCP

---
```

```yaml
# Example: auth-service may only be called from gateway and vault-
        service
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-auth-service-ingress
  namespace: helixterm-prod
spec:
  podSelector:
    matchLabels:
      app: auth-service
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: gateway
        - podSelector:
            matchLabels:
              app: vault-service
      ports:
        - port: 8080
          protocol: TCP
        - port: 9090
          protocol: TCP
```

10.3 Secret Management with Sealed Secrets

Kubernetes Secrets are encrypted at rest using Bitnami Sealed Secrets. All plaintext secrets are **never committed to Git**; only sealed manifests are stored:

```yaml
# infrastructure/kubernetes/base/secrets/sealed/database-
        creds.sealed.yaml
# Generated with: kubeseal --format yaml < database-creds-
        plain.yaml
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: database-creds
  namespace: helixterm-prod
  annotations:
    sealedsecrets.bitnami.com/cluster-wide: "false"
spec:
```

```
  encryptedData:
    host: AgBy3...truncated
    username: AgBy4...truncated
    password: AgBy5...truncated
    port: AgBy6...truncated
  template:
    metadata:
      name: database-creds
      namespace: helixterm-prod
    type: Opaque
```

Sealing workflow:

```
#!/bin/bash
# scripts/security/seal-secret.sh
# Usage: ./seal-secret.sh <namespace> <secret-name> <key>=<value>
        [<key>=<value> ...]

NAMESPACE="$1"
SECRET_NAME="$2"
shift 2

# Create a temporary plaintext secret manifest
kubectl create secret generic "${SECRET_NAME}" \
  --namespace "${NAMESPACE}" \
  --dry-run=client \
  -o yaml \
  $(for kv in "$@"; do echo "--from-literal=${kv}"; done) | \
kubeseal \
  --controller-name=sealed-secrets-controller \
  --controller-namespace=kube-system \
  --format yaml \
  > "infrastructure/kubernetes/base/secrets/sealed/$
        {SECRET_NAME}.sealed.yaml"

echo "Sealed secret written to infrastructure/kubernetes/base/
        secrets/sealed/${SECRET_NAME}.sealed.yaml"
echo "IMPORTANT: Never commit the plaintext secret."
```

For highly sensitive secrets (vault master keys, signing keys), HashiCorp Vault is used with the Vault Agent Injector sidecar pattern:

```
# Vault annotation example on a Pod
annotations:
  vault.hashicorp.com/agent-inject: "true"
  vault.hashicorp.com/role: "auth-service"
```

```
  vault.hashicorp.com/agent-inject-secret-config: "secret/data/
      helixterm/auth-service"
  vault.hashicorp.com/agent-inject-template-config: |
    {{- with secret "secret/data/helixterm/auth-service" -}}
    export JWT_SECRET="{{ .Data.data.jwt_secret }}"
    export SIGNING_KEY="{{ .Data.data.signing_key }}"
    {{- end }}
```

10.4 Container Image Vulnerability Scanning Gates

Trivy is integrated as a blocking gate in CI/CD pipelines:

```
# .github/workflows/pr.yml (security scanning step)
- name: Scan image for vulnerabilities
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ghcr.io/helixdevelopment/${{ matrix.service }}:$
        {{ github.sha }}
    format: sarif
    output: trivy-results.sarif
    severity: CRITICAL,HIGH
    exit-code: 1           # Fail the pipeline on CRITICAL/HIGH
        CVEs
    ignore-unfixed: false
    vuln-type: os,library

- name: Upload Trivy SARIF to GitHub Security
  uses: github/codeql-action/upload-sarif@v3
  if: always()
  with:
    sarif_file: trivy-results.sarif
    category: trivy-${{ matrix.service }}
```

Trivy `.trivyignore` is managed by the security team and reviewed quarterly:

```
# .trivyignore
# Vulnerability IDs that have been reviewed and accepted as low-
risk
# Each entry requires: CVE-ID, review date, reviewer,
justification

# CVE-2023-XXXXX: false positive in static binary, no network
exposure
```

```
# Reviewed: 2026-01-15, reviewer: security@helixterm.io
# CVE-2023-XXXXX
```

SBOM (Software Bill of Materials) generation is automated post-build:

```
# Generate SBOM in CycloneDX format
syft ghcr.io/helixdevelopment/${SERVICE}:${TAG} -o cyclonedx-json
        > sbom-${SERVICE}-${TAG}.json

# Attest SBOM with cosign
cosign attest \
  --predicate sbom-${SERVICE}-${TAG}.json \
  --type cyclonedx \
  --key cosign.key \
  ghcr.io/helixdevelopment/${SERVICE}:${TAG}
```

10.5 SAST Integration (gosec + Semgrep)

```
# .github/workflows/sast.yml
name: SAST

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  gosec:
    name: gosec security scan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run gosec
        uses: securego/gosec@master
        with:
          args: >
            -exclude=G104,G304
            -fmt=sarif
            -out=gosec-results.sarif
            ./...

      - name: Upload gosec SARIF
        uses: github/codeql-action/upload-sarif@v3
```

```yaml
      if: always()
      with:
        sarif_file: gosec-results.sarif
        category: gosec

  semgrep:
    name: Semgrep SAST
    runs-on: ubuntu-latest
    container:
      image: semgrep/semgrep:1.99.0
    steps:
      - uses: actions/checkout@v4

      - name: Run Semgrep
        run: |
          semgrep scan \
            --config p/golang \
            --config p/docker \
            --config p/kubernetes \
            --config p/secrets \
            --sarif \
            --output semgrep-results.sarif \
            --error \
            .

      - name: Upload Semgrep SARIF
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: semgrep-results.sarif
          category: semgrep

  govulncheck:
    name: govulncheck dependency scan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-go@v5
        with:
          go-version: '1.25'

      - name: Install govulncheck
        run: go install golang.org/x/vuln/cmd/govulncheck@latest
```

```
      - name: Run govulncheck
        run: govulncheck ./...
```

10.6 Falco Runtime Security

Falco monitors container runtime behavior and alerts on suspicious activity:

```
# infrastructure/kubernetes/base/security/falco-rules.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: falco-custom-rules
  namespace: falco
data:
  helix-rules.yaml: |
    # ─── HelixTerminator Custom Falco Rules
        ──────────────────────────────────────


    - rule: Unexpected Outbound Connection from Auth Service
      desc: The auth-service should only connect to PostgreSQL,
        Redis, and Vault
      condition: >
        outbound and
        container.image.repository = "ghcr.io/helixdevelopment/
        auth-service" and
        not fd.rip in (postgres_ips, redis_ips, vault_ips) and
        not fd.rport in (5432, 6379, 8200)
      output: >
        Unexpected outbound connection from auth-service
        (user=%user.name container=%container.name dst=%fd.rip:
        %fd.rport)
      priority: WARNING
      tags: [network, helixterm, auth]


    - rule: Shell Spawned in Service Container
      desc: No service should spawn a shell during normal
        operation
      condition: >
        spawned_process and
        container and
        container.image.repository startswith "ghcr.io/
        helixdevelopment/" and
        proc.name in (shell_binaries) and
        not proc.pname in (known_parent_binaries)
```

```
  output: >
    Shell spawned in HelixTerminator container
    (user=%user.name container=%container.name
    image=%container.image.repository
     shell=%proc.name parent=%proc.pname
    cmdline=%proc.cmdline)
  priority: CRITICAL
  tags: [shell, helixterm, anomaly]

- rule: Sensitive File Read in Vault Service
  desc: Vault service reading files outside expected paths
  condition: >
    open_read and
    container.image.repository = "ghcr.io/helixdevelopment/
    vault-service" and
    not fd.name startswith "/service" and
    not fd.name startswith "/tmp" and
    not fd.name startswith "/etc/ssl" and
    not fd.name startswith "/proc" and
    not fd.name startswith "/dev/null"
  output: >
    Vault service reading unexpected file
    (user=%user.name container=%container.name file=%fd.name)
  priority: WARNING
  tags: [file, helixterm, vault]

- rule: Crypto Mining Detected
  desc: Detected potential crypto mining process
  condition: >
    spawned_process and
    container and
    container.image.repository startswith "ghcr.io/
    helixdevelopment/" and
    (proc.name in (crypto_miners) or
     proc.cmdline contains "stratum+tcp" or
     proc.cmdline contains "nicehash")
  output: >
    Crypto mining detected in HelixTerminator container
    (user=%user.name container=%container.name proc=%proc.name
    cmdline=%proc.cmdline)
  priority: CRITICAL
  tags: [crypto, helixterm, malware]

- macro: shell_binaries
  condition: >
```

```
      proc.name in (bash, sh, ash, zsh, ksh, fish, dash, tcsh,
      csh)

  - list: postgres_ips
    items: []

  - list: redis_ips
    items: []

  - list: vault_ips
    items: []
```

10.7 Kubernetes API Audit Logging

EKS audit logging is enabled for all API calls:

```
// infrastructure/aws/eks-audit-policy.json
{
  "apiVersion": "audit.k8s.io/v1",
  "kind": "Policy",
  "rules": [
    {
      "level": "None",
      "resources": [{"group": "", "resources": ["events"]}]
    },
    {
      "level": "None",
      "users": ["system:kube-proxy"],
      "verbs": ["watch"],
      "resources": [{"group": "", "resources": ["endpoints",
        "services"]}]
    },
    {
      "level": "None",
      "users": ["system:unsecured"],
      "namespaces": ["kube-system"],
      "verbs": ["get"],
      "resources": [{"group": "", "resources": ["configmaps"]}]
    },
    {
      "level": "None",
      "userGroups": ["system:authenticated"],
      "nonResourceURLs": ["/api*", "/version", "/healthz"]
    },
    {
```

```
      "level": "Request",
      "verbs": ["get", "list", "watch"],
      "resources": [
        {"group": "", "resources": ["nodes", "pods", "secrets",
        "configmaps"]},
        {"group": "apps", "resources": ["deployments",
        "daemonsets", "statefulsets"]}
      ]
    },
    {
      "level": "RequestResponse",
      "verbs": ["create", "update", "patch", "delete",
        "deletecollection"],
      "resources": [
        {"group": "", "resources": ["secrets", "configmaps",
        "serviceaccounts"]},
        {"group": "rbac.authorization.k8s.io", "resources":
        ["*"]},
        {"group": "apps", "resources": ["deployments",
        "statefulsets", "daemonsets"]}
      ]
    },
    {
      "level": "RequestResponse",
      "resources": [{"group": "", "resources": ["pods/exec",
        "pods/attach"]}]
    },
    {
      "level": "Metadata"
    }
  ]
}
```

Audit logs are streamed to CloudWatch Logs and then forwarded to the SIEM:

```
# infrastructure/terraform/modules/eks/main.tf (audit log
configuration)
resource "aws_eks_cluster" "main" {
  # ... (other config) ...

  enabled_cluster_log_types = [
    "api",
    "audit",
    "authenticator",
    "controllerManager",
    "scheduler"
```

```
  ]
}

resource "aws_cloudwatch_log_subscription_filter" "audit_logs" {
  name            = "helix-eks-audit-to-siem"
  log_group_name  = "/aws/eks/${var.cluster_name}/cluster"
  filter_pattern  = ""
  destination_arn = var.siem_firehose_arn
  distribution    = "ByLogStream"
}
```

10.8 RBAC Configuration

Principle of least privilege is enforced through granular RBAC roles:

```
# infrastructure/kubernetes/base/rbac/service-roles.yaml

# Developers can view resources but not modify secrets
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: helix-developer
rules:
  - apiGroups: [""]
    resources: ["pods", "services", "configmaps", "endpoints"]
    verbs: ["get", "list", "watch"]
  - apiGroups: [""]
    resources: ["pods/log"]
    verbs: ["get", "list"]
  - apiGroups: ["apps"]
    resources: ["deployments", "statefulsets", "daemonsets",
        "replicasets"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["autoscaling"]
    resources: ["horizontalpodautoscalers"]
    verbs: ["get", "list", "watch"]

---
# On-call engineers can restart pods and view logs
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: helix-oncall
rules:
  - apiGroups: [""]
```

```
    resources: ["pods"]
    verbs: ["get", "list", "watch", "delete"]
  - apiGroups: [""]
    resources: ["pods/log", "pods/exec"]
    verbs: ["get", "list", "create"]
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "patch", "update"]

---
# CI/CD service account: limited to image updates
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: helix-cicd
rules:
  - apiGroups: ["apps"]
    resources: ["deployments", "statefulsets"]
    verbs: ["get", "list", "watch", "patch", "update"]
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["batch"]
    resources: ["jobs"]
    verbs: ["get", "list", "create", "delete"]

---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: helix-cicd-binding
subjects:
  - kind: ServiceAccount
    name: github-actions
    namespace: helixterm-prod
roleRef:
  kind: ClusterRole
  name: helix-cicd
  apiGroup: rbac.authorization.k8s.io
```

10.9 Image Signing with Cosign

All production images are signed with cosign to prevent supply chain attacks:

```
# scripts/security/sign-image.sh
#!/bin/bash
set -euo pipefail

IMAGE="${1:?Image reference required}"

echo "Signing image: ${IMAGE}"

# Sign the image (keyless via OIDC in CI, key-based locally)
if [ -n "${COSIGN_KEY:-}" ]; then
  cosign sign --key "${COSIGN_KEY}" "${IMAGE}"
else
  # Keyless signing using GitHub OIDC in CI
  cosign sign "${IMAGE}"
fi

echo "Image signed: ${IMAGE}"
```

Admission webhook enforces that only signed images can run in production:

```
# Connaisseur or Policy Controller configuration
apiVersion: cosigned.sigstore.dev/v1beta1
kind: ClusterImagePolicy
metadata:
  name: helix-image-policy
spec:
  images:
    - glob: "ghcr.io/helixdevelopment/**"
  authorities:
    - keyless:
        url: https://fulcio.sigstore.dev
        identities:
          - issuer: https://token.actions.githubusercontent.com
            subject: "https://github.com/HelixTerminator/helix-
terminator/.github/workflows/release.yml@refs/heads/main"
```

10.10 Dependency Vulnerability Scanning

```yaml
# .github/workflows/dependency-scan.yml
name: Dependency Vulnerability Scan

on:
  schedule:
    - cron: '0 6 * * *'   # Daily at 06:00 UTC
  push:
    branches: [main]

jobs:
  go-vulnerabilities:
    name: Go module vulnerabilities
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-go@v5
        with:
          go-version: '1.25'

      - name: Install govulncheck
        run: go install golang.org/x/vuln/cmd/govulncheck@latest

      - name: Run govulncheck
        run: |
          govulncheck -json ./... > govulncheck-results.json || true
          # Fail if any HIGH or CRITICAL vulnerabilities found
          jq -e '.vulnerability | select(.modules[].found_in != null) |
              select(.osv.database_specific.severity == "HIGH" or
                     .osv.database_specific.severity == "CRITICAL")' \
              govulncheck-results.json && exit 1 || true

  flutter-dependencies:
    name: Flutter/Dart dependency audit
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: subosito/flutter-action@v2
        with:
```

```
          flutter-version: '3.24.x'

      - name: Check Flutter dependencies
        working-directory: clients/flutter
        run: |
          flutter pub outdated --json > pub-outdated.json
          dart pub audit

  docker-base-images:
    name: Scan base images for vulnerabilities
    runs-on: ubuntu-latest
    strategy:
      matrix:
        image:
          - golang:1.25-alpine
          - gcr.io/distroless/static:nonroot
          - nginx:alpine
    steps:
      - name: Scan base image
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: ${{ matrix.image }}
          format: table
          exit-code: 0
          severity: CRITICAL
```

10.11 Constitution Compliance Check

The HelixConstitution submodule contains enforceable rules about service behavior, API design, and security practices. The compliance check runs in every PR pipeline:

```
#!/bin/bash
# scripts/constitution/check-compliance.sh
# Validates that all services comply with HelixConstitution rules

set -euo pipefail

CONSTITUTION_DIR="constitution"
SERVICES_DIR="services"
VIOLATIONS=0

echo "==> Running HelixConstitution compliance check"
```

```
# Rule: Every service must have /healthz/live and /healthz/ready
        endpoints
# (the actual route pair every service registers — see §2.3
        probes; a bare
# "/healthz" literal never appears in service code, so grepping
        for it alone
# false-fails every service).
for svc in "${SERVICES_DIR}"/*/; do
  svc_name=$(basename "${svc}")
  if ! grep -rqE '"/healthz/(live|ready)"' "${svc}"; then
    echo "VIOLATION: ${svc_name} missing /healthz/live or /
        healthz/ready endpoint"
    VIOLATIONS=$((VIOLATIONS + 1))
  fi
done

# Rule: Every service must export Prometheus metrics on :9090/
        metrics
for svc in "${SERVICES_DIR}"/*/; do
  svc_name=$(basename "${svc}")
  if ! grep -rq 'prometheus' "${svc}"; then
    echo "VIOLATION: ${svc_name} missing Prometheus metrics"
    VIOLATIONS=$((VIOLATIONS + 1))
  fi
done

# Rule: No service may use fmt.Println for logging (must use
        structured logger)
PRINTLN_COUNT=$(grep -r 'fmt\.Println' "${SERVICES_DIR}" --
        include="*.go" | grep -v "_test.go" | wc -l || true)
if [ "${PRINTLN_COUNT}" -gt 0 ]; then
  echo "VIOLATION: $
        {PRINTLN_COUNT} instances of fmt.Println found (use
        structured logger)"
  grep -rn 'fmt\.Println' "${SERVICES_DIR}" --include="*.go" |
        grep -v "_test.go"
  VIOLATIONS=$((VIOLATIONS + PRINTLN_COUNT))
fi

# Rule: No hardcoded secrets
HARDCODED_SECRETS=$(grep -rE '(password|secret|key|token)\s*[:=]
        \s*"[^"]{8,}"' \
  "${SERVICES_DIR}" --include="*.go" | grep -v "_test.go" | grep
        -v "example" | wc -l || true)
if [ "${HARDCODED_SECRETS}" -gt 0 ]; then
  echo "VIOLATION: Potential hardcoded secrets detected"
  VIOLATIONS=$((VIOLATIONS + HARDCODED_SECRETS))
fi
```

```
# Rule: All services must have a Dockerfile
for svc in "${SERVICES_DIR}"/*/; do
  svc_name=$(basename "${svc}")
  if [ ! -f "${svc}/Dockerfile" ]; then
    echo "VIOLATION: ${svc_name} missing Dockerfile"
    VIOLATIONS=$((VIOLATIONS + 1))
  fi
done

echo ""
if [ "${VIOLATIONS}" -gt 0 ]; then
  echo "FAILED: ${VIOLATIONS} constitution violation(s) found."
  exit 1
else
  echo "PASSED: All services comply with HelixConstitution."
fi
```

10.12 Security Review Checklist

Before any service reaches production, it must pass this security review:

Category	Check	Owner
Authentication	All API endpoints require authentication (except /healthz, /readyz)	Backend team
Authorization	RBAC enforced at service level, not just gateway	Backend team
Input validation	All external input validated and sanitized	Backend team
SQL injection	Parameterized queries used everywhere (no raw string concatenation)	Backend team
Secrets	No secrets in environment variables visible	Platform team

Category	Check	Owner
	to all containers (use Vault or Sealed Secrets)	
TLS	All inter-service communication uses mTLS (via Istio or manual cert provisioning)	Platform team
Logging	No PII or secrets logged	Backend team
Rate limiting	All public endpoints have rate limiting configured at gateway	Backend team
Dependency audit	`govulncheck` passes with no HIGH/CRITICAL findings	All teams
Image scan	Trivy scan passes with no HIGH/CRITICAL CVEs	Platform team
SAST	gosec and semgrep scans pass	All teams
Container	Runs as non-root, read-only filesystem, no privilege escalation	Platform team
Network	Outbound connections declared in NetworkPolicy	Platform team
Audit trail	All sensitive operations emit audit events to Kafka	Backend team

11. Cost and FinOps Governance

Every managed AWS resource this document provisions — EKS (§6.2), RDS Multi-AZ + cross-region replica (§6.2, §8.3.4), ElastiCache Redis (§6.2), MSK Kafka (§6.2), Amazon MQ (§6.2), the `eu-west-1` DR footprint (§8.2), Harbor's S3 backend (§6.3), and the Coder cloud dev environment (§9.10) — carries a real, ongoing cost. This section was previously absent from an 8,000+ line infrastructure spec; it exists so cost is a first-class, reviewed input to every infrastructure decision rather than an after-the-fact surprise.

> **All figures below are planning-order-of-magnitude ESTIMATES**, not a vendor quote — rounded to avoid false precision, using On-Demand US `us-east-1` list pricing as the baseline before any of the discounting strategies in §11.2 are applied. Re-price before committing budget; do not treat these numbers as a committed spend ceiling.

11.1 Per-Environment Monthly Cost Estimate

Component	Dev/Sandbox	Staging	Production (`us-east-1`)	DR (`eu-west-1`)
EKS control plane	$73	$73	$73	$73
EKS nodes (system + general + gpu, §6.2)	~$400 (small, scaled to 0 gpu)	~$1,800 (min-size node groups)	~$9,000 (desired-size: 3 system m6i.xlarge + 9 general m6i/m6a.2xlarge)	~$1,200 (scaled-down standby, §8.2
RDS PostgreSQL (§6.2)	db.t4g.medium, single-AZ, ~$100	db.r6g.large Multi-AZ, ~$700	db.r6g.2xlarge Multi-AZ + reporting read replica, ~$4,200	Cross-region async read replica, db.r6g.xlarge ~$1,100
ElastiCache Redis (§6.2)	cache.t4g.micro, ~$25	cache.r7g.large x2, ~$600	cache.r7g.xlarge x3 Multi-AZ, ~$2,700	Not replicate (Redis is cache-tier, rebuilt on failover)
MSK Kafka (§6.2, §8.5)	Not provisioned (docker-compose, §9.4)	kafka.m5.large x3, ~$1,100	kafka.m5.2xlarge x3 + 3TB storage, ~$3,600	MirrorMaker target,

Component	Dev/Sandbox	Staging	Production (`us-east-1`)	DR (`eu-west-1`)
				kafka.m5.lar x3, ~$1,100
Amazon MQ / RabbitMQ (§6.2)	Not provisioned (docker-compose, §9.4)	mq.m5.large single-instance, ~$220	mq.m5.xlarge CLUSTER_MULTI_AZ, ~$900	mq.m5.large single-instance (shovel target), ~$22
S3 (session recordings + backups + Harbor + logs)	~$20	~$150	~$1,200 (storage + cross-region replication egress, §6.2)	(covered by CRR into DR bucket abov
CloudFront + Route53	~$10	~$30	~$500 (egress-driven)	—
Observability (Prometheus/ Grafana/ Loki/Jaeger/ OTel, §7)	~$50 (in-cluster, no extra infra)	~$200	~$800 (in-cluster storage + CloudWatch log export retention)	~$150
Harbor registry (§6.3, in-cluster + S3)	—	~$50	~$300	~$100 (DR replica bucket)
Coder cloud dev environment (§9.10)	~$150 (few active workspaces)	—	—	—
Estimated total (rounded)	**~$830/mo**	**~$4,900/mo**	**~$23,300/mo**	**~$3,940/mo**

Production + DR combined is the platform's steady-state AWS bill — **≈ $27,000/month, planning-order-of-magnitude** — before any of the discounting strategies in §11.2 are applied; those strategies are expected to bring the production figure down materially (Compute Savings Plans alone typically discount steady-state EC2/Fargate spend by 20-40% relative to On-Demand list price).

11.2 Cost-Allocation Tags

Every Terraform resource in §6 MUST carry the same tag schema so cost is attributable per environment, per team, and per service without a manual reconciliation pass:

```
# infrastructure/terraform/shared/tags.tf
locals {
  mandatory_tags = {
    Project     = "helix-terminator"
    Environment = var.environment          # dev | staging |
production | dr
    CostCenter  = var.cost_center           # e.g. "platform-eng",
set per environment/team
    Team        = var.owning_team           # e.g. "platform",
"ssh-domain", "vault-domain"
    ManagedBy   = "terraform"
  }
}

provider "aws" {
  region = var.aws_region
  default_tags {
    tags = local.mandatory_tags
  }
}
```

`default_tags` on the AWS provider block (Terraform native, not a custom wrapper) means every resource created by `module.eks`, `aws_db_instance`, `aws_elasticache_replication_group`, `aws_msk_cluster`, `aws_mq_broker`, and every `aws_s3_bucket` in §6 inherits the tag set automatically — no per-resource tag block to remember or drift out of sync.

11.3 Rightsizing, Savings Plans, and Spot Strategy

Workload	Strategy	Rationale
EKS `system` + `general` node groups (§6.2)	**1-year, No-Upfront Compute Savings Plan** sized to the p50 steady-state vCPU-hour usage from the first 60 days of Cost Explorer data	Baseline, always-on capacity; a Savings Plan (not RIs) covers EC2 + Fargate + Lambda uniformly if any service moves to Fargate later

Workload	Strategy	Rationale
EKS `gpu` node group (§6.2, ML anomaly detection)	**Spot** with the AWS Node Termination Handler + Karpenter consolidation	Batch/interruptible session-analysis workload (min_size = 0); a 2-minute Spot interruption notice is acceptable because jobs checkpoint to Kafka (§8.5) and resume
RDS PostgreSQL prod primary (§6.2)	**1-year Reserved Instance** (No Upfront) on `db.r6g.2xlarge` once the instance class is confirmed stable post-launch	Always-on, cannot Spot; RI is the correct always-on discount instrument for RDS (Savings Plans do not cover RDS)
RDS reporting read replica + cross-region replica	On-Demand initially, re-evaluate for RI after 90 days of stable usage	Newer/less proven usage pattern; avoid committing to a 1-year term before the read-replica sizing is confirmed correct
MSK, Amazon MQ, ElastiCache	On-Demand (no RI/ Savings-Plan equivalent as of this pin)	AWS does not offer a reservation instrument for these three services at time of writing — re-check the AWS pricing page at each cost review (§11.4 no-guessing: do not assume a discount instrument exists without checking)
Coder dev workspaces (§9.10)	**On-Demand + 2-hour idle auto-stop**	Bursty, short-lived, per-developer — reservation makes no sense; the auto-stop TTL is the actual cost control

Rightsizing cadence. AWS Compute Optimizer (EC2/EBS/Lambda) and `kubecost`/`OpenCost` (in-cluster, namespace/pod-level cost attribution, deployed alongside the Prometheus stack in §7.1) are reviewed monthly; any node group or RDS instance class running below 40% average CPU utilization for 30 consecutive days is a rightsizing candidate, tracked the same way any other workable item is tracked in this project's governance model.

11.4 Budget Alerts

```
# infrastructure/terraform/shared/budgets.tf
resource "aws_budgets_budget" "helixterm_prod_monthly" {
  name         = "helixterm-prod-monthly"
  budget_type  = "COST"
  limit_amount = "26000"   # planning ceiling, ~12% headroom over
the §11.1 production estimate
  limit_unit   = "USD"
  time_unit    = "MONTHLY"

  cost_filter {
    name   = "TagKeyValue"
    values = ["user:Environment$production"]
  }

  dynamic "notification" {
    for_each = [50, 80, 100, 120]
    content {
      comparison_operator        = "GREATER_THAN"
      threshold                  = notification.value
      threshold_type             = "PERCENTAGE"
      notification_type          = "ACTUAL"
      subscriber_sns_topic_arns  =
[aws_sns_topic.finops_alerts.arn]
    }
  }
}

resource "aws_sns_topic" "finops_alerts" {
  name = "helixterm-finops-alerts"
}

# SNS -> Slack via the same Lambda pattern the observability stack
(§7) uses
```

```
# for PagerDuty/Slack routing — not duplicated here, see §7.2
alert-routing.
```

Budget alerts fire at 50/80/100/120% of the monthly ceiling into the `#finops-alerts` Slack channel (same routing infrastructure as the production alert rules in §7.2), so a cost anomaly (runaway autoscaling, an orphaned Coder workspace stuck below its idle-detector threshold, an accidental Multi-AZ enable on a staging resource) is caught within the same billing cycle it occurs, not discovered a month later on the AWS invoice.

Appendix A: helix-deps.yaml

The `helix-deps.yaml` file at the repository root declares all submodule versions and external service version locks:

```yaml
# helix-deps.yaml
apiVersion: helix/v1
kind: DependencyManifest
metadata:
  name: helix-terminator
  version: 1.0.0

submodules:
  containers:
    repo: https://github.com/digital-vasic/containers
    ref: v2.3.1
    path: submodules/containers
    go_module: digital.vasic.containers

  docs_chain:
    repo: https://github.com/digital-vasic/docs_chain
    ref: v1.8.0
    path: submodules/docs_chain
    go_module: digital.vasic.docs_chain

  security:
    repo: https://github.com/digital-vasic/security
    ref: v3.1.0
    path: submodules/security
    go_module: digital.vasic.security

  auth:
```

```
    repo: https://github.com/digital-vasic/auth
    ref: v4.0.2
    path: submodules/auth
    go_module: digital.vasic.auth

  helixqa:
    repo: https://github.com/HelixDevelopment/helixqa
    ref: v1.2.0
    path: submodules/helixqa

  constitution:
    repo: https://github.com/HelixTerminator/constitution
    ref: v1.1.0
    path: constitution

infrastructure:
  kubernetes: "1.31"
  helm: "3.16"
  kafka: "3.9.0"
  postgresql: "17.2"
  redis: "8.0"
  rabbitmq: "4.0"
  prometheus: "3.0.0"
  grafana: "11.3.0"
  jaeger: "1.62"
  loki: "3.2.0"
  otel_collector: "0.110.0"
  cert_manager: "1.16.0"
  ingress_nginx: "1.11.0"
  falco: "0.39.0"
  sealed_secrets: "2.16.0"
```

Appendix B: Container Runtime Abstraction

The `digital.vasic.containers` module (`ContainerRuntime` interface) abstracts Docker, Podman, and Kubernetes operations behind a unified API. All HelixTerminator services use this interface when spawning or managing containers (e.g., for SSH session isolation):

```
// submodules/containers/runtime.go (interface definition)
package containers

import (
```

```
    "context"
    "io"
    "time"
)

// ContainerRuntime is the primary interface for container
    lifecycle management.
// Implementations: DockerRuntime, PodmanRuntime,
    KubernetesRuntime
type ContainerRuntime interface {
    // Create creates a new container with the given
        configuration.
    Create(ctx context.Context, cfg ContainerConfig)
        (ContainerID, error)

    // Start starts a stopped container.
    Start(ctx context.Context, id ContainerID) error

    // Stop gracefully stops a running container.
    Stop(ctx context.Context, id ContainerID, timeout
        time.Duration) error

    // Remove removes a container (must be stopped first).
    Remove(ctx context.Context, id ContainerID, force bool) error

    // Exec runs a command inside a running container.
    Exec(ctx context.Context, id ContainerID, cmd ExecConfig)
        (ExecResult, error)

    // Attach attaches stdin/stdout/stderr to a running container.
    Attach(ctx context.Context, id ContainerID) (AttachConn,
        error)

    // Inspect returns detailed state for a container.
    Inspect(ctx context.Context, id ContainerID) (ContainerInfo,
        error)

    // Logs returns a stream of container logs.
    Logs(ctx context.Context, id ContainerID, opts LogOptions)
        (io.ReadCloser, error)

    // PullImage pulls a container image from a registry.
    PullImage(ctx context.Context, ref ImageRef, opts
        PullOptions) error

    // Runtime returns the name of the underlying runtime (docker,
        podman, kubernetes).
```

```
    Runtime() RuntimeType
}
```

Local development uses `PodmanRuntime` (default if Docker socket not found) or `DockerRuntime`. Production Kubernetes deployments use `KubernetesRuntime`, which creates ephemeral Pods for each SSH session with strict resource quotas and security contexts.

Document version: 1.0.0 — Generated for HelixTerminator v1.0 release
Last updated: 2026-06-28 Owner: Platform Engineering Team